- It can be used for more fine-grained routing
- It can both authenticate and authorize requests before routing them into the service mesh

To benefit from these advantages, we will replace the Kubernetes Ingress controller with the Istio ingress gateway. The Istio ingress gateway is used by creating Gateway and VisualService objects, as described previously in the *Introducing Istio API objects* section.

The definition of the previously used Ingress objects has been removed from the dev-env and prod-env Helm charts in kubernetes/helm/environments. The definition files for Istio Gateway and VirtualService objects will be explained in the *Creating the service mesh* section.

The Istio ingress gateway is reached using a different IP address from the IP address used to access the Kubernetes Ingress controller, so we also need to update the IP address mapped to the hostname, minikube.me, which we use when running tests. This is handled in the *Setting up access to Istio services* section.

Replacing the Zipkin server with Istio's Jaeger component

As mentioned in the *Introducing Istio* section, Istio comes with built-in support for distributed tracing using Jaeger. Using Jaeger, we can offload and simplify the microservice landscape in Kubernetes by removing the Zipkin server we introduced in *Chapter 14, Understanding Distributed Tracing.* We will also change the way trace and span IDs are propagated between the microservices from using the default W3C trace context headers to using OpenZipkin's B3 headers. See the *Introducing distributed tracing with Micrometer Tracing and Zipkin* section of *Chapter 14, Understanding Distributed Tracing,* for more information.

The following changes have been applied to the source code:

- The following dependencies have been replaced in all microservice build files, build.gradle:

```
implementation 'io.micrometer:micrometer-tracing-bridge-otel'
implementation 'io.opentelemetry:opentelemetry-exporter-zipkin'
```

The dependencies have been replaced with the following:

```
implementation 'io.micrometer:micrometer-tracing-bridge-brave'
implementation 'io.zipkin.reporter2:zipkin-reporter-brave'
```

- The management.zipkin.tracing.endpoint property in the common configuration file, config-repo/application.yml, points to the Jaeger component in Istio. It has the host name jaeger-collector.istio-system.
- The definition of the Zipkin server in the three Docker Compose files, docker-compose.yml, docker-compose-partitions.yml, and docker-compose-kafka.yml, has been retained to be able to use distributed tracing outside of Kubernetes and Istio, but the Zipkin server has been given the same hostname as the Jaeger component in Istio, jaeger-collector.istio-system.
- The Helm chart for the Zipkin server has been removed.

Jaeger will be installed in the Deploying Istio in a Kubernetes cluster section coming up.

With these simplifications of the microservice landscape explained, we are ready to deploy Istio in a Kubernetes cluster.

Deploying Istio in a Kubernetes cluster

In this section, we will learn how to deploy Istio in a Kubernetes cluster and how to access the Istio services in it.

We will use Istio's CLI tool, istioctl, to install Istio using a demo configuration of Istio that is suitable for testing Istio in a development environment, that is, with most features enabled but configured for minimalistic resource usage.

This configuration is unsuitable for production usage and for performance testing.

For other installation options, see https://istio.io/latest/docs/setup/install/.

To deploy Istio, perform the following steps:

1. Ensure that your Minikube instance from the previous chapter is up and running with the following command:

```
minikube status
```

Expect a response along the lines of the following, provided it is up and running:

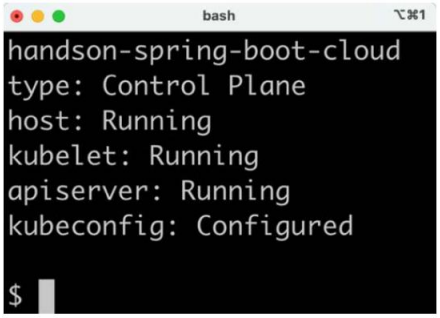


Figure 18.4: Minikube status OK

2. Run a precheck to verify that the Kubernetes cluster is ready for Istio to be installed in it:

```
istioctl experimental precheck
```

Expect a response like the following:

Figure 18.5: Istio precheck OK

3. Install Istio using the demo profile with the following command:

```
cd $BOOK_HOME/Chapter18
istioctl install --skip-confirmation \
    --set profile=demo \
    --set meshConfig.accessLogFile=/dev/stdout \ --set
    meshConfig.accessLogEncoding=JSON \
    --set values.pilot.env.PILOT_JWT_PUB_KEY_REFRESH_INTERVAL=15s \ -f
    kubernetes/istio-tracing.yml
```

The command parameters do the following:

- The accessLog parameters are used to enable the Istio proxies to log requests that are processed. Once Pods are up and running with Istio proxies installed, the access logs can be inspected with the command kubectl logs <MY-POD> -c istio-proxy.

- The PILOT_JWT_PUB_KEY_REFRESH_INTERVAL parameter configures Istio's daemon, istiod, to refresh the fetched JWKS public keys every 15 seconds. The usage of this parameter will be explained in the Deploying v1 and v2 versions of the microservices with routing to the v1 version section.
- The configuration file kubernetes/istio-tracing.yml enables the creation of trace spans used for distributed tracing. It also configures Istio to propagate trace spans between requests using OpenZipkin's B3 headers, as mentioned in the Re-placing the Zipkin server with Istio's Jaeger component section. It looks like this:

```
apiVersion: install.istio.io/v1alpha1
kind: IstioOperator
spec:
  meshConfig:
    enableTracing: true
    defaultConfig:
      tracing: {} # disable legacy MeshConfig tracing options
    extensionProviders:
    - name: "zipkin"
      zipkin:
        service: zipkin.istio-system.svc.cluster.local
        port: 9411
```

The zipkin field declares the tracer service to which all trace spans are sent using the Zipkin API. The hostname specified in the service field, zipkin.istio-system.svc.cluster.
local, points to the Jaeger component and the port (9411) where Jaeger receives trace spans using the Zipkin API.

If you inspect the Zipkin service with the kubectl get svc -n istio-system zipkin -o yaml command, you will see that its selector field points to Pods labeled with app: jaeger, meaning that the Zipkin service will direct all calls to one of the Jaeger component's Pods.

4. By default, trace spans are only created for one percent of the requests. In a production environment, it seems to be a reasonable default value to avoid being overwhelmed by trace spans. But in our test environment, we want to be able to see distributed traces for every request we send. To update the sampling rate used by Istio, we must create a Telemetry object. The configuration file kubernetes/istio-telemetry.yml defines the object as follows:

```
apiVersion: telemetry.istio.io/v1
kind: Telemetry
metadata:
  name: mesh-default
spec:
  tracing:
  - providers:
    - name: "zipkin"
    randomSamplingPercentage: 100.00
```

The Telemetry object configures the tracing provider Zipkin to have a 100% random sampling percentage, meaning that it will create distributed traces for all incoming requests.

Create the Telemetry object with the following command:

```
kubectl -n istio-system apply -f kubernetes/istio-telemetry.yml
```

5. Wait for the Deployment objects and their Pods to be available with the following command:

```
kubectl -n istio-system wait --timeout=600s --
for=condition=available deploy --all
```

6. Next, install the extra components described in the *Introducing Istio* section – Kiali, Jaeger, Prometheus, and Grafana – with these commands:

```
istio_version=$(istioctl version --remote=false -o json | jq -r .clientVersion.version)

echo "Installing integrations for Istio v$istio_version"

kubectl apply -n istio-system -f https://raw.githubusercontent.com/
istio/istio/${istio_version}/samples/addons/kiali.yaml

kubectl apply -n istio-system -f https://raw.githubusercontent.com/
```

```
istio/istio/${istio_version}/samples/addons/jaeger.yaml

kubectl apply -n istio-system -f https://raw.githubusercontent.com/
istio/istio/${istio_version}/samples/addons/prometheus.yaml

kubectl apply -n istio-system -f https://raw.githubusercontent.com/
istio/istio/${istio_version}/samples/addons/grafana.yaml
```

If any of these commands fail, try rerunning the failing command. Errors can occur due to timing issues, which can be resolved by running commands again. Specifically, the installation of Kiali can result in error messages starting with unable to recognize. Rerunning the command makes these error messages go away.

7. Wait a second time for the extra components to be available with the following command:

```
kubectl -n istio-system wait --timeout=600s --
for=condition=available deploy --all
```

8. Finally, run the following command to see what was installed:

```
kubectl -n istio-system get deploy
```

Expect an output similar to this:

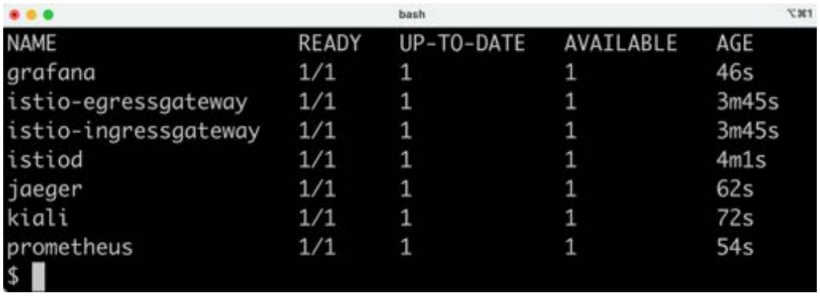


Figure 18.6: Deployments in the Istio Namespace

Istio is now deployed in Kubernetes, but before we move on and create the service mesh, we need to learn a bit about how to access the Istio services in a Minikube environment.

Setting up access to Istio services

The demo configuration used in the previous section to install Istio comes with a few connectivity -related issues that we need to resolve. The Istio ingress gateway is configured as a load-balanced Kubernetes service; that is, its type is LoadBalancer. To be able to access the gateway, we need to run a load balancer in front of the Kubernetes cluster.

Minikube contains a command that can be used to simulate a local load balancer, minikube tunnel. This command assigns an external IP address to each load-balanced Kubernetes service, including the Istio ingress gateway. The hostname, minikube.me, that we use in our tests needs to be translated into the external IP address of the Istio ingress gateway. To simplify access to the web UIs of components such as Kiali and Jaeger, we will also add hostnames dedicated to these services, for example, kiali.minikube.me.

We will also register a hostname to the external health endpoint, as described in the *Observing the service mesh* section. Finally, a few hostnames for services installed and used in subsequent chapters will also be registered so that we don't need to add new hostnames in the following chapters. The services that we will install in the next chapters are Kibana, Elasticsearch, and a mail server.

To enable external access using these hostnames to the Istio services, a Helm chart has been created; see kubernetes/helm/environments/istio-system. The chart contains a Gateway, VirtualService, and DestinationRule object for each Istio component. To protect requests to these hostnames from eavesdropping, only HTTPS requests are allowed. cert-manager, which was introduced in the previous chapter, is used by the chart to automatically provision a TLS certificate for the hostnames and store it in a Secret named hands-on-certificate. All Gateway objects are configured to use this Secret in their configuration of the HTTPS protocol. All definition files can be found in the Helm charts templates folder.

The use of these API objects will be described in more detail in the *Creating the service mesh* and *Protecting external endpoints with HTTPS and certificates* sections.

Run the following command to apply the Helm chart:

```
helm upgrade --install istio-hands-on-addons kubernetes/helm/environments/
istio-system -n istio-system --wait
```

This will result in the gateway being able to route requests for the following hostnames to the corresponding Kubernetes Service:

- kiali.minikube.me requests are routed to kiali:20001
- tracing.minikube.me requests are routed to tracing:80

- prometheus.minikube.me requests are routed to prometheus:9000
- grafana.minikube.me requests are routed to grafana:3000

To verify that the certificate and secret objects have been created, run the following commands:

```
kubectl -n istio-system get secret hands-on-certificate kubectl -n istio-system
get certificate hands-on-certificate
```

Expect an output like this:

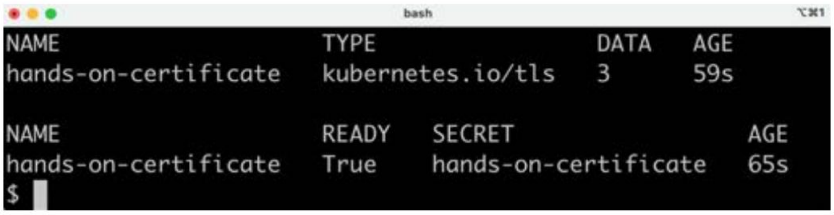


Figure 18.7: The cert-manager has delivered both a TLS Secret and a certificate

The following diagram summarizes how the components can be accessed:

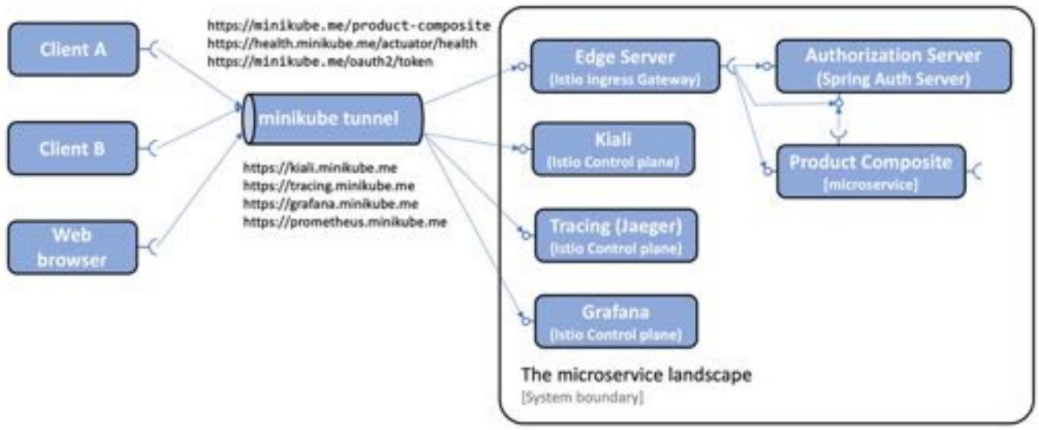


Figure 18.8: Hostnames to be used for accessing components through the Minikube tunnel

Perform the following steps to set up the Minikube tunnel and register the hostnames:

1. Run the following command in a separate terminal window (the command locks the terminal window when the tunnel is up and running):

```
minikube tunnel
```

Note that this command requires that your user has sudo privileges and that you enter your password during startup. It can take a couple of seconds before the command asks for the password, so it is easy to miss!

Once the tunnel is up and running, it will list the istio-ingressgateway as one of the services it exposes (the only one in our case).

2. Configure the hostnames to be resolved to the IP address of the Istio ingress gateway. Start by getting the IP address exposed by the minikube tunnel command for the Istio ingress gateway and save it in an environment variable named INGRESS_IP:

```
INGRESS_IP=$(kubectl -n istio-system get service istio-ingressgateway
-o jsonpath='{.status.loadBalancer.ingress[0].ip}')

echo $INGRESS_IP
```

The echo command will print an IP address. Since we use Minikube's Docker driver, it will always be 127.0.0.1.

3. Update /etc/hosts so that all minikube.me hostnames will use the IP address of the Istio ingress gateway:

```
MINIKUBE_HOSTS="minikube.me grafana.minikube.me kiali.minikube.
me prometheus.minikube.me tracing.minikube.me kibana.minikube.me
elasticsearch.minikube.me mail.minikube.me health.minikube.me"
echo 127.0.0.1 $MINIKUBE_HOSTS | sudo tee -a /etc/hosts
```

4. On Windows, we also need to update the Windows hosts file:

    to. In Windows, open a PowerShell terminal.

    b. Open the Windows hosts file in Visual Code Studio with the following command:

    ```
    code C:\Windows\System32\drivers\etc\hosts
    ```

    c. Add the following line to the Windows hosts file:

    ```
    127.0.0.1 minikube.me grafana.minikube.me kiali.minikube.me prometheus.minikube.me
    tracing.minikube.me kibana.minikube.me elasticsearch.minikube.me mail.minikube.me
    health.minikube.me
    ```

d. When you try to save it, you will get an error regarding Insufficient permissions. Click on the **Retry as Admin...** button to update the hosts file as an administrator.

e. Verify the update:

```
cat C:\Windows\System32\drivers\etc\hosts
```

By default, the /etc/hosts file is overwritten by the content in the Windows hosts file when WSL is restarted. Restarting WSL takes a long time as it also restarts Docker. Restarting Docker, in turn, results in the Minikube instance being stopped, so it needs to be restarted manually. To prevent this slow and tedious restart process, we simply updated both files.

5. Remove the line in /etc/hosts where minikube.me points to only the IP address of the Minikube instance (127.0.0.1). Verify that /etc/hosts only contains one line that translates minikube.me and that it points to the IP address of the Istio ingress gateway, 127.0.0.1:

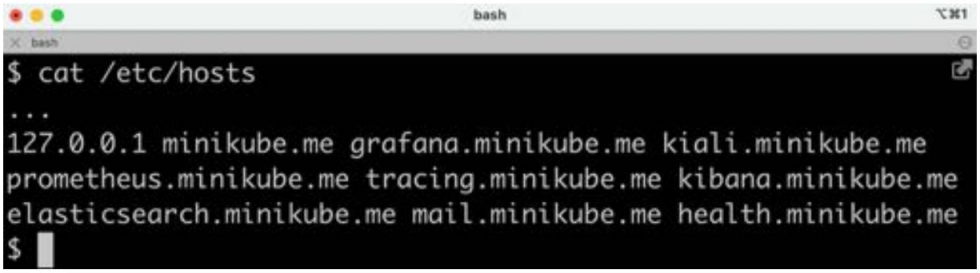


Figure 18.9: /etc/hosts file updated

6. Verify that Kiali, Jaeger, Grafana, and Prometheus can be reached through the tunnel with the following commands:

```
curl -o /dev/null -sk -L -w "%{http_code}\n" https://kiali.minikube.
me/kiali/

curl -o /dev/null -sk -L -w "%{http_code}\n" https://tracing.
minikube.me

curl -o /dev/null -sk -L -w "%{http_code}\n" https://grafana.
minikube.me

curl -o /dev/null -sk -L -w "%{http_code}\n" https://prometheus.
minikube.me/graph#/
```

Each command should return 200 (OK). If the request sent to Kiali doesn't return 200, it often means that its internal initialization is not complete. Wait a minute and try again in that case.

The minikube tunnel command will stop running if, for example, your computer or the Minikube instance is paused or restarted. It needs to be restarted manually in these cases. So, if you fail to call APIs on any of the minikube.me hostnames, always check whether the Minikube tunnel is running and restart it if required.

With the Minikube tunnel in place, we are now ready to create the service mesh.

Creating the service mesh

With Istio deployed, we are ready to create the service mesh. The steps required to create the service mesh are the same as those we used in *Chapter 17, Implementing Kubernetes Features to Simplify the System Landscape* (refer to the *Testing with Kubernetes ConfigMaps, Secrets, Ingress, and cert-manager* section). Let's first see what additions have been made to the Helm templates to set up the service mesh before we run the commands to create the service mesh.

Source code changes

To be able to run the microservices in a service mesh managed by Istio, the dev-env Helm chart brings in two new named templates from the common chart, _istio_base.yaml and _istio_dr_mutual_tls.yaml. Let's go through them one by one.

Content in the _istio_base.yaml template

_istio_base.yaml defines a number of Kubernetes manifests that will be used by both environment charts, dev-env and prod-env. First, it defines three Istio-specific security-related manifests:

- An AuthorizationPolicy manifest named product-composite-require-jwt
- A PeerAuthentication manifest named default
- A RequestAuthentication manifest named product-composite-request-authentication

These three manifests will be explained in the *Securing a service mesh* section.

The remaining four manifests will be discussed here. They are two pairs of Gateway and VirtualService manifests that are used to configure access to and routing from the minikube.me and health.minikube.me hostnames. Gateway objects will be used to define how to receive external traffic, and VirtualService objects are used to describe how to route the incoming traffic inside the service mesh.

The Gateway manifest for controlling access to minikube.me looks like this:

```
apiVersion: networking.istio.io/v1beta1 kind: Gateway

metadata:
  name: hands-on-gw
spec:
  selector:
    istio: ingressgateway
  servers:
  - hosts:
    - minikube.me
    port:
      name: https
      number: 443
      protocol: HTTPS
    tls:
      credentialName: hands-on-certificate
      mode: SIMPLE
```

Here is an explanation of the source code:

- The gateway is named hands-on-gw; this name is used by the virtual services underneath.
- The selector field specifies that the gateway object will be handled by the default Istio ingress gateway, named ingressgateway.
- The hosts and port fields specify that the gateway will handle incoming requests for the minikube.me hostname using HTTPS over port 443.
- The tls field specifies that the Istio ingress gateway can find the certificate and private key used for HTTPS communication in a TLS Secret named hands-on-certificate. Refer to the *Protecting external endpoints with HTTPS and certificates* section for details on how these certificate files are created. The SIMPLE mode denotes that normal TLS semantics will be applied.

The VirtualService manifest for routing requests sent to minikube.me appears as follows:

```
apiVersion: networking.istio.io/v1beta1
kind: VirtualService
metadata:
  name: hands-on-vs
spec:
```

```
  gateways:
  - hands-on-gw
  hosts:
  - minikube.me
  http:
  - match:
    - uri:
        prefix: /oauth2
    route:
    - destination:
        host: auth-server
  – match:
    ...
```

Explanations of the preceding manifest are as follows:

- The gateways and hosts fields specify that the virtual service will route requests that are sent to the minikube.me hostname through the hands-on-gw gateway.
- Within the http element is an array of match and route blocks that specify how URL paths will be forwarded to the associated Kubernetes service. In the preceding manifest, only the first pair of match and route elements is shown. They map requests sent to minikube.me using the /oauth2 path to the auth-server service. This mapping should be familiar from how we specified routing rules in both Spring Cloud Gateway and Ingress objects in the previous chapters. The remaining pairs of match and route elements configure the same routing rules as we have seen for Spring Cloud Gateway and Ingress objects:
  - /login ÿ auth-server
  - /error ÿ auth-server
  - /product-composite ÿ product-composite
  - /openapi ÿ product-composite

For more details, see kubernetes/helm/common/templates/_istio_base.yaml.

In the preceding source code, the destination host is specified using its short name, in other words, product-composite. This works, since the example is based on Kubernetes definitions from the same Namespace, hands-on. If that is not the case, it is recommended in the Istio documentation to use the host's **fully qualified domain name (FQDN)** instead. In this case, it is product-composite.hands-on.svc.cluster.local.

Content in the _istio_dr_mutual_tls.yaml template _istio_dr_mutual_tls.yaml defines a template for specifying a number of DestinationRule objects. It is used to specify that mTLS should be used when routing a request to its corresponding service. It can also be used optionally to specify subsets, something that we will use in the prod-env chart in the *Performing zero-downtime updates* section. The template looks like this:

```
{{- define "common.istio_dr_mutual_tls" -}} {{- range $idx,
$dr := .Values.destinationRules }} apiVersion: networking.istio.io/
v1beta1
kind: DestinationRule
metadata:
    name: {{ $dr.name }}
spec:
    host: {{ $dr.name }}
{{- if $dr.subsets }} {{- with
$dr.subsets }}
    subsets:
{{ toYaml .          | indent 2 }}
{{- end }} {{-
end }}
    trafficPolicy:
        tls:
            mode: ISTIO_MUTUAL
---
{{- end -}} {{-
end -}}
```

Here are some comments about the preceding template:

- The range directive loops over the elements defined in the destinationRules variable
- The host field in the spec part of the manifest is used to specify the name of the Kubernetes Service that this DestinationRule applies to
- A subsets section is only defined if a corresponding element is found in the current element, $dr, in the destinationRules list
- A trafficPolicy is always used to require mTLS

The template is used in the dev-end Helm chart by specifying the destinationRules variable in the values.yaml file as follows:

```
destinationRules:
  - name: product-composite
  - name: auth-server
  - name: product
  - name: recommendation
  - name: review
```

The files can be found at kubernetes/helm/common/templates/_istio_dr_mutual_tls.yaml and kubernetes/helm/environments/dev-env/values.yaml.

With these changes in the source code in place, we are now ready to create the service mesh.

Running commands to create the service mesh

Create the service mesh by running the following commands:

1. Build Docker images from the source code with the following commands:

```
cd $BOOK_HOME/Chapter18
eval $(minikube docker-env -u)
./gradlew build
eval $(minikube docker-env)
docker compose build
```

The eval $(minikube docker-env -u) command ensures that the ./gradlew build command uses the host's Docker engine and not the Docker engine in the Minikube instance. The build command uses Docker to run test containers.

2. Recreate the hands-on Namespace and set it as the default Namespace:

```
kubectl delete namespace hands-on
kubectl apply -f kubernetes/hands-on-namespace.yml
kubectl config set-context $(kubectl config current-context) --namespace=hands-on
```

Note that the hands-on-namespace.yml file creates the hands-on Namespace labeled with istio-injection: enabled. This means that Popds created in this Namespace will get istio-proxy containers injected as sidecars automatically.

3. Resolve the Helm chart dependencies with the following commands:

    to. First, we update the dependencies in the components folder:

    ```
    for f in kubernetes/helm/components/*; do helm dep up $f;
    done
    ```

    b. Next, we update the dependencies in the environments folder:

    ```
    for f in kubernetes/helm/environments/*; do helm dep up $f; donated
    ```

4. Deploy the system landscape using Helm and wait for all Deployments to complete:

    ```
    helm install hands-on-dev-env \
      kubernetes/helm/environments/dev-env \
      -n hands-on --wait
    ```

5. Once the Deployment is complete, verify that we have two containers in each of the microservice Pods:

    ```
    kubectl get pods
    ```

    Expect a response along the lines of the following:

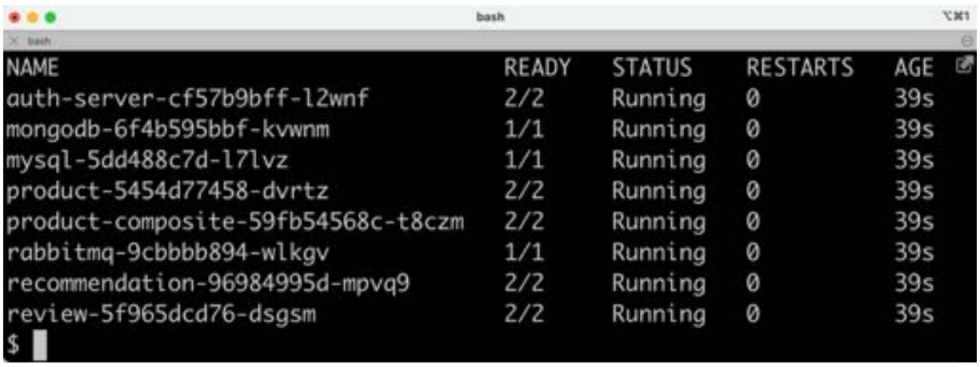


Figure 18.10: Pods up and running

    Note that the Pods that run our microservices report two containers per Pod; that is, they have the Istio proxy injected as a sidecar!

6. Run the usual tests with the following command:

    ```
    ./test-em-all.bash
    ```

The default values for the test-em-all.bash script have been updated from previous chapters to accommodate Kubernetes running in Minikube.

Expect the output to be similar to what we have seen in previous chapters:

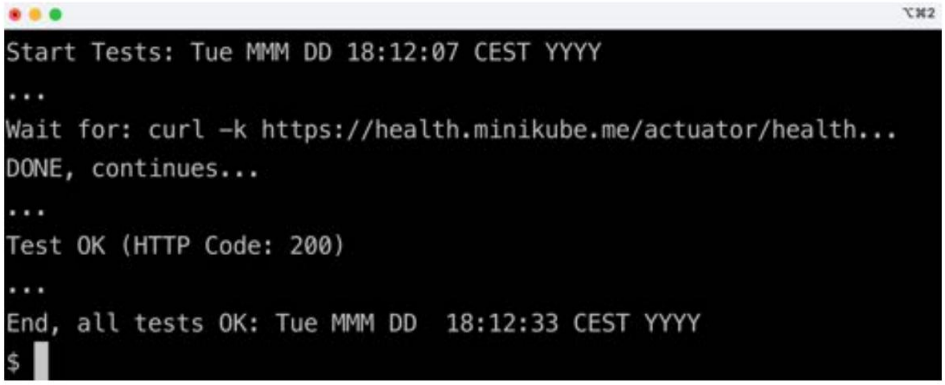


Figure 18.11: Tests running successfully

Before we start to try out Istio and its various components, let's see how we can log the propagation of trace and span IDs using the B3 headers mentioned in the *Replacing the Zipkin server with Istio's Jaeger component* section.

Logging propagation of trace and span IDs

We can see the trace and span IDs in the outgoing requests from the product-composite microservice as we did in the *Sending a successful API request* section of *Chapter 14, Understanding Distributed Tracing.* Since we now run the microservices in Kubernetes, we need to change the log configuration in a ConfigMap and then delete the running Pod to make it affect the microservice:

1. Edit the ConfigMap with the following command:

```
kubectl edit cm product-composite
```

Look for the following lines:

```
# To see tracing headers, uncomment the following two lines and restart the
product-composite service
# spring.codec.log-request-details: true
# logging.level.org.springframework.web.reactive.function.client.
ExchangeFunctions: TRACE
```

2. Uncomment the last two of these lines and exit the editor.
3. Restart the product-composite microservice by deleting its Pod with this command:

```
kubectl delete pod -l app=product-composite
```

4. Print the log output to a terminal window with the following command:

```
kubectl logs -f -l app=product-composite
```

5. Acquire an access token and make a request using the access token:

```
unset ACCESS_TOKEN
ACCESS_TOKEN=$(curl -k https://writer:secret-writer@minikube.me/
oauth2/token -d grant_type=client_credentials -d scope="product:read product:write" -s | jq
-r .access_token)
echo $ACCESS_TOKEN

curl -H "Authorization: Bearer $ACCESS_TOKEN" -k https://minikube.
me/product-composite/1 -w "%{http_code}\n" -o /dev/null -s
```

   Verify that the command returns the HTTP status code for success, 200.

6. In the log output, lines like the following should be seen:

```
YYYY-MM-DDT09:18:52.014Z TRACE 1 ---                    parallel-6] [product-
[ composite,9f96e8379863d637a0d7301b48cee91b,163bbf8f529f8
1a3] oswrfclient.ExchangeFunctions HTTP GET                    : [741082ec]
http://product/product/1?delay=0&faultPercent=0, headers=[X-B3-
TraceId:"9f96e8379863d637a0d7301b48cee91b", X-B3-
SpanId:"163bbf8f529f81a3", X-B3-ParentSpanId:"252a9d9d278d206b", X-B3-
Sampled:"1"]
```

   In the example log output, we can see the standard B3 headers such as X-B3-TraceId and X-B3-SpanId.

7. Revert to not logging trace and span IDs by adding the comments back in the ConfigMap and restart the microservice by deleting its Pod.

With the service mesh up and running, let's see how we can observe what's going on in it using Kiali!

Observing the service mesh

In this section, we will use Kiali together with Jaeger to observe what's going on in the service mesh.

Before we do that, we need to understand how to get rid of some noise created by the health checks performed by Kubernetes' liveness and readiness probes. In the previous chapters, they used the same port as the API requests. This means that Istio will collect metrics for the usage of both health checks and requests sent to the API. This will cause the graphs shown by Kiali to become unnecessarily cluttered. Kiali can filter out traffic that we are not interested in, but a simpler solution is to use a different port for the health checks.

Microservices can be configured to use a separate port for requests sent to the actuator endpoints, for example, health checks sent to the /actuator/health endpoint. The following line has been added to the common configuration file for all microservices, config-repo/application.yml:

```
management.server.port: 4004
```

This will make all microservices use port 4004 to expose the health endpoints. The values.yaml file in the common Helm chart has been updated to use port 4004 in the default liveness and readiness probes. See kubernetes/helm/common/values.yaml.

The product-composite microservice exposes its management port not only to the Kubernetes probes but also externally for health checks, for example, performed by test-em-all.bash. This is done through Istio's ingress gateway, and therefore port 4004 is added to the product-composite microservice Deployment and Service manifests. See the ports and service.ports definitions in kubernetes/helm/components/product-composite/values.yaml.

Spring Cloud Gateway (which is retained so we can run tests in Docker Compose) will continue to use the same port for requests to the API and the health endpoint. In the config-repo/gateway.yml configuration file, the management port is changed to the port used for the API:

```
management.server.port: 8443
```

To simplify external access to the health check exposed by the product-composite microservice, a route is configured for the health.minikube.me hostname to the management port on the product-composite microservice. Refer to the explanation of the _istio_base.yaml template.

With the requests sent to the health endpoint out of the way, we can start to send some requests through the service mesh.

We will start a low-volume load test using siege, which we learned about in *Chapter 16, Deploying Our Microservices to Kubernetes.* After that, we will go through some of the most important parts of Kiali to see how it can be used to observe a service mesh in a web browser. We will also see how Jaeger is used for distributed tracing.

Since the certificate we use is self-signed, web browsers will not rely on it automatically . Most web browsers let you visit the web page if you assure them that you understand the security risks. If the web browser refuses, opening a private window helps in some cases.

Specifically, regarding Chrome, if it does not let you visit the web page, saying **Your connection is not private,** you can click the **Advanced** button and then click on the **Proceed to... (unsafe)** link.

Start the test client with the following commands:

```
ACCESS_TOKEN=$(curl https://writer:secret-writer@minikube.me/oauth2/token -d
grant_type=client_credentials -d scope="product:read product:write" -ks | jq .access_token -r)


echo ACCESS_TOKEN=$ACCESS_TOKEN

siege https://minikube.me/product-composite/1 -H "Authorization: Bearer $ACCESS_TOKEN" -c1 -d1
-v
```

The first command will get an OAuth 2.0/OIDC access token that will be used in the next command , where siege is used to submit one HTTP request per second to the product-composite API.

Expect output from the siege command as follows:

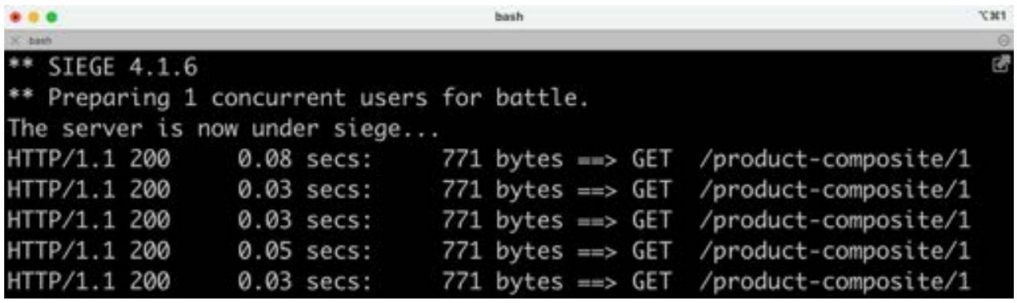


Figure 18.12: System landscape under siege

Use a web browser of your choice that accepts self-signed certificates and proceed with the following steps:

1. Open Kiali's web UI using the https://kiali.minikube.me URL. By default, you will be logged in as an anonymous user. Expect a web page similar to the following:

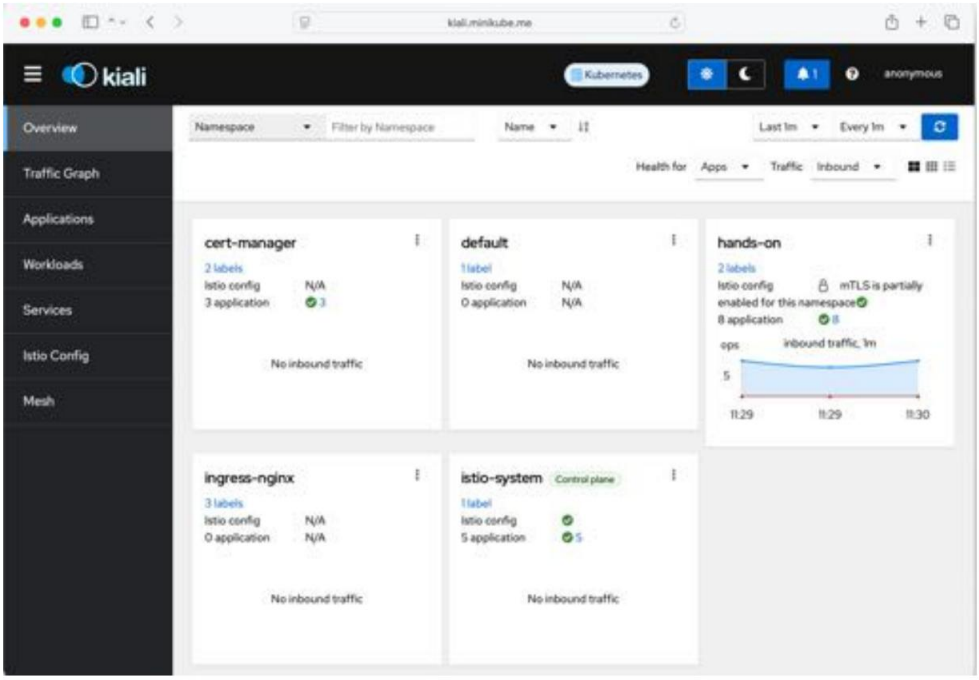


Figure 18.13: Kiali web UI

2. Click on the **Overview** tab if it is not already active.
3. Click on the menu (three vertical dots in the top-right corner) in the box named **hands-on** and select **Graph.** Expect a graph to be shown that represents the current traffic flowing through the service mesh.
4. Click on the **Display** button and deselect all options except for **Response Time, Median,** and **Traffic Animation.**
5. In the **Hide…** field, specify name = jaeger to avoid cluttering the view with traces sent to Jaeger.
6. Kiali now displays a graph representing requests that are currently sent through the service mesh, where active requests are represented by small moving circles along the arrows, as follows:

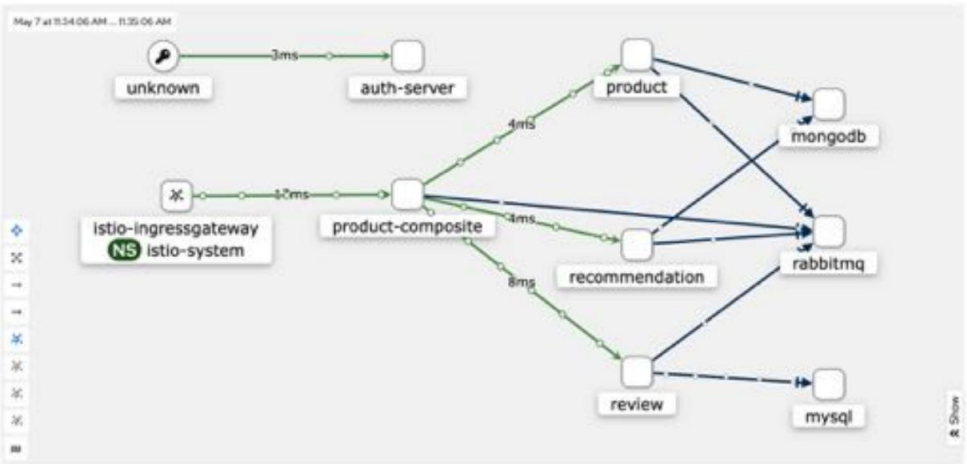


Figure 18.14: Kiali graph showing the hands-on Namespace

7. The traffic from **unknown** to the **auth-server** represents calls to the authorization server to get the JWKS public keys.

   This gives a pretty good initial overview of what's going on in the service mesh!

8. Let's now look at some distributed tracing using Jaeger. Open the web UI using the https://tracing.minikube.me URL. Click on the **Service** dropdown in the menu to the left and select the **istio-ingressgateway.istio-system** service. Click on the **Find Trace** button; you should see a result like this:

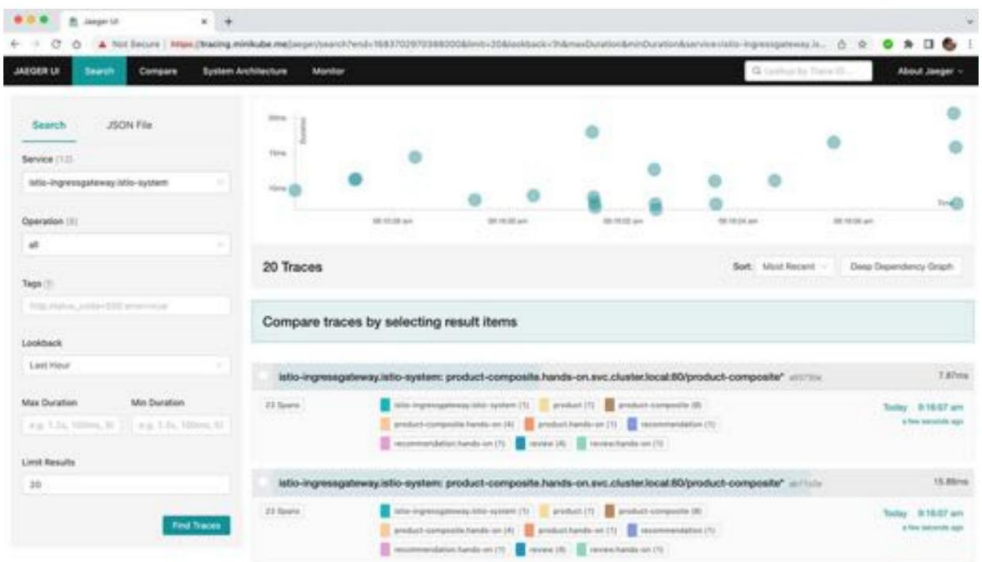


Figure 18.15: Distributed traces visualized by Jaeger

9. Click on one of the traces that is reported to contain **23 Spans** to examine it. Expect a web page such as the following:

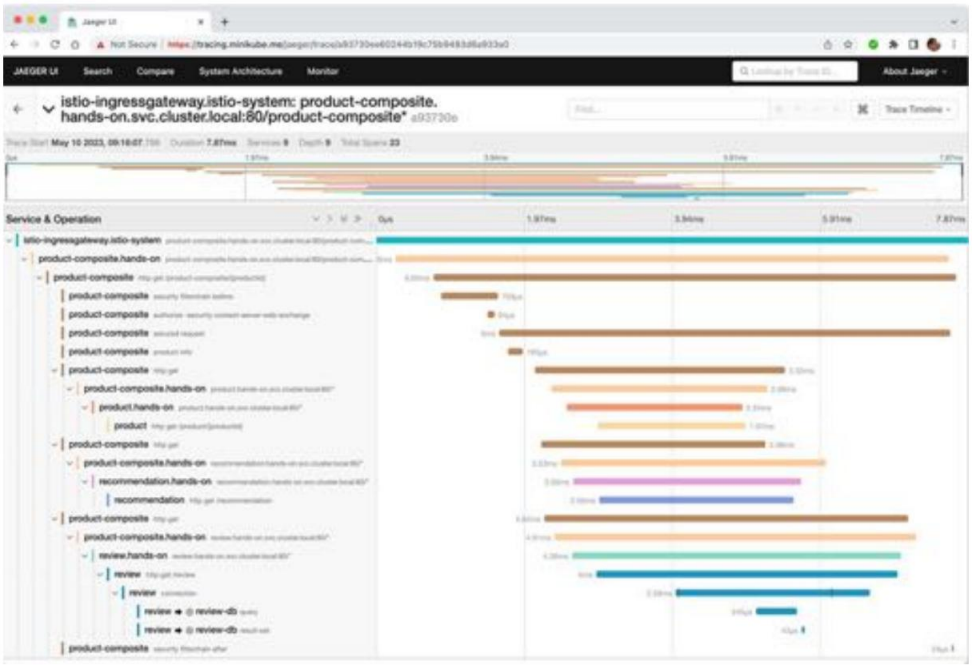


Figure 18.16: View a full trace call tree in Jaeger

This is basically the same tracing information as Zipkin made available in *Chapter 14, Understanding Distributed Tracing.* Note that we can see trace information from both the Istio proxies and the microservices themselves. The spans reported by Istio proxies are suffixed with the Kubernetes Namespace, that is, .istio-system and .hands-on.

There is much more to explore, but this is enough by way of an introduction. Feel free to explore the web UI in Kiali and Jaeger on your own.

Be aware that the access token acquired for the test client, siege, is only valid for an hour. If the traffic drops unexpectedly, check the output from siege; if it reports 4XX instead of 200, it's time to renew the access token!

Let's move on and learn how Istio can be used to improve security in the service mesh!

Securing a service mesh

In this section, we will learn how to use Istio to improve the security of a service mesh. We will cover the following topics:

- How to protect external endpoints with HTTPS and certificates
- How to require that external requests are authenticated using OAuth 2.0/OIDC access tokens
- How to protect internal communication using **mutual authentication (mTLS)**

Let's now understand each of these in the following sections.

Protecting external endpoints with HTTPS and certificates

In the *Setting up access to Istio services* and *Content in the _istio_base.yaml template* sections, we learned that the gateway objects use a TLS certificate stored in a Secret named hands-on-certificate for their HTTPS endpoints.

The Secret is created by the cert-manager based on the configuration in the istio-system Helm chart. The chart's template, selfsigned-issuer.yaml, is used to define an internal self-signed CA and has the following content:

```
apiVersion: cert-manager.io/v1
kind: Issuer
metadata:
  name: selfsigned-issuer
spec:
  selfSigned: {}
---
apiVersion: cert-manager.io/v1
kind: Certificate
metadata:
  name: ca-cert
spec:
  isCA: true
  commonName: hands-on-ca
  secretName: ca-secret
  issuerRef:
    name: selfsigned-issuer
---
```

```
apiVersion: cert-manager.io/v1
kind: Issuer
metadata:
  name: ca-issuer
spec:
  AC:
    secretName: ca-secret
```

From the preceding manifests, we can see the following:

- A self-signed issuer named selfsigned-issuer.
- This issuer is used to create a self-signed certificate named ca-cert. • The certificate is given the common name hands-on-ca.
- Finally, a self-signed CA, ca-issuer, is defined using the certificate, ca-cert, as its root certificate. This CA will be used to issue the certificate used by the gateway objects.

The chart's template, hands-on-certificate.yaml, defines this certificate as follows:

```
apiVersion: cert-manager.io/v1
kind: Certificate
metadata:
  name: hands-on-certificate
spec:
  commonName: minikube.me
  subject:
    ...
  dnsNames:
  - minikube.me
  - health.minikube.m
  - dashboard.minikube.me
  - kiali.minikube.me
  - tracing.minikube.me -
  prometheus.minikube.me -
  grafana.minikube.me
  - kibana.minikube.me
  - elasticsearch.minikube.me
  - mail.minikube.me
  issuerRef:
    name: ca-issuer
  secretName: hands-on-certificate
```

From this manifest, we can see the following:

- The certificate is named hands-on-certificate
- Its common name is set to minikube.me
- It specifies a few optional extra details about its subject (left out for clarity)
- All other hostnames are declared as **Subject Alternative Names** in the certificate
- It will use the issuer named ca-issuer declared previously
- The cert-manager will store the TLS certificate in a Secret named hands-on-certificate

When the istio-system Helm chart was installed, these templates were used to create the corresponding API objects in Kubernetes. This triggered the cert-manager to create the certificates and Secrets.

The template files can be found in the kubernetes/helm/environments/istio-system/templates folder.

To verify that it is these certificates that are used by the Istio ingress gateway, we can run the following command:

```
keytool -printcert -sslserver minikube.me | grep -E "Owner:|Issuer:"
```

Expect the following output:

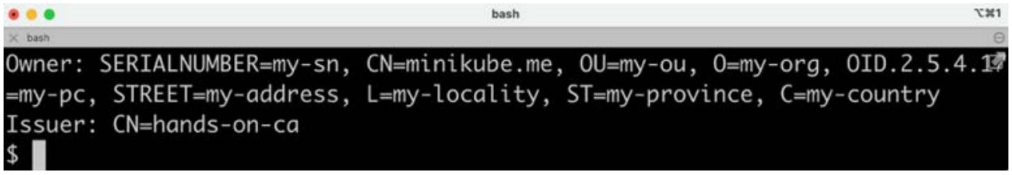


Figure 18.17: Inspecting the certificate for minikube.me

The output shows that the certificate is issued for the common name minikube.se and that it is issued by our own CA, using its root certificate with the common name hands-on-ca.

As mentioned in *Chapter 17, Implementing Kubernetes Features to Simplify the System Landscape* (refer to the *Automating certificate provisioning* section), this self-signed CA needs to be replaced for production use cases with, for example, Let's Encrypt or another CA that the cert-manager can use to provide trusted certificates.

With the certificate configuration verified, let's move on to see how the Istio ingress gateway can protect microservices from unauthenticated requests.

Authenticating external requests using OAuth 2.0/OIDC access tokens

An Istio ingress gateway can require and validate JWT-based OAuth 2.0/OIDC access tokens, in other words, protecting the microservices in the service mesh from external unauthenticated requests. For a recap on JWT, OAuth 2.0, and OIDC, refer to *Chapter 11, Securing Access to APIs* (see the *Protecting APIs using OAuth 2.0 and OpenID Connect* section). Istio can also be configured to perform authorization but, as mentioned in the *Introducing Istio API objects* section, we will not use it.

This is configured in the common Helm chart's template, _istio_base.yaml. The two manifests look like this:

```
apiVersion: security.istio.io/v1beta1 kind:
RequestAuthentication
metadata:
  name: product-composite-request-authentication
spec:
  jwtRules:
  - forwardOriginalToken: true issuer:
    http://auth-server
    jwksUri: http://auth-server.hands-on.svc.cluster.local/oauth2/jwks
  selector:
    matchLabels:
      app.kubernetes.io/name:product-composite

---


apiVersion: security.istio.io/v1beta1
kind: AuthorizationPolicy
metadata:
  name: product-composite-require-jwt
spec:
  action: ALLOW
  rules:
  - {}
  selector:
    matchLabels:
      app.kubernetes.io/name:product-composite
```

From the manifests, we can see the following:

- The RequestAuthentication named product-composite-request-authentication re- quires a valid JWT-encoded access token for requests sent to the product-composite service:
  - It selects services that it performs request authentication for based on a label selector, app.kubernetes.io/name: product-composite.
  - It allows tokens from the issuer, http://auth-server.
  - It will use the http://auth-server.hands-on.svc.cluster.local/oauth2/jwks URL to fetch a JWKS. The key set is used to validate the digital signature of the access tokens.
  - It will forward the access token to the underlying services, in our case, the product-com-posite microservice.
- The AuthorizationPolicy named product-composite-require-jwt is configured to allow all requests to the product-composite service; it will not apply any authorization rules.

It can be a bit hard to understand whether Istio's RequestAuthentication is validating the access tokens or whether it is only the product-composite service that is performing the validation. One way to ensure that Istio is doing its job is to change the configuration of RequestAuthentication so that it always rejects access tokens.

To verify that RequestAuthentication is in action, apply the following commands:

1. Make a normal request:

```
ACCESS_TOKEN=$(curl https://writer:secret-writer@minikube.me/oauth2/
token -d grant_type=client_credentials -d scope="product:read product:write" -ks |
jq .access_token -r)
echo ACCESS_TOKEN=$ACCESS_TOKEN
curl -k https://minikube.me/product-composite/1 -H "Authorization: Bearer
$ACCESS_TOKEN" -i
```

   Verify that it returns an HTTP response status code 200 (OK).

2. Edit the RequestAuthentication object and temporarily change the issuer, for example, to http://auth-server-x:

```
kubectl edit RequestAuthentication product-composite-request
authentication
```

3. Verify the change:

```
kubectl get RequestAuthentication product-composite-request-authentication -o yaml
```

Verify that the issuer has been updated, in my case to http://auth-server-x.

4. Make the request again. It should fail with the HTTP response status code 401 (Unauthorized) and the error message Jwt issuer is not configured:

```
curl -k https://minikube.me/product-composite/1 -H "Authorization: Bearer $ACCESS_TOKEN" -i
```

Since it takes a few seconds for Istio to propagate the change, the new name of the issuer, you might need to repeat the command a couple of times before it fails.

This proves that Istio is validating the access tokens!

5. Revert the changed name of the issuer to http://auth-server:

```
kubectl edit RequestAuthentication product-composite-request-authentication
```

6. Verify that the request works again. First, wait a few seconds for the change to be propagated. Then run the following command:

```
curl -k https://minikube.me/product-composite/1 -H "Authorization: Bearer $ACCESS_TOKEN"
```

Suggested additional exercise: Try out the Auth0 OIDC provider, as described in *Chapter 11, Securing Access to APIs* (refer to the *Testing with an external OpenID Connect provider* section). Add your Auth0 provider to jwt- authentication-policy.yml. In my case, it appears as follows:

```
- jwtRules:
    issuer: “https://dev-magnus.eu.auth0.com/”
    jwksUri: “https://dev-magnus.eu.auth0.com/.well-
“known/jwks.json”
```

Now, let's move on to the last security mechanism that we will cover in Istio: the automatic protection of internal communication in the service mesh using mutual authentication with mTLS.

Protecting internal communication using mutual authentication with mTLS

In this section, we will learn how Istio can be configured to automatically protect internal com-munication within the service mesh using **mTLS.** When using mutual authentication, not only does the service provide its identity by exposing a certificate but the clients also prove their identity to the service by exposing a client-side certificate. This provides a higher level of security compared to normal TLS/HTTPS usage, where only the identity of the service is proven. Setting up and maintaining mutual authentication, that is, providing new certificates and rotating outdated certificates for the clients, is known to be complex and is therefore rarely used. Istio fully automates the provisioning and rotation of certificates for the mutual authentication used

for internal communication within the service mesh. This makes it much easier to use mutual authentication compared to setting it up manually.

So, why should we use mutual authentication? Isn't it sufficient to protect external APIs with HTTPS and OAuth 2.0/OIDC access tokens?

As long as the attacks come through the external API, it might be sufficient. But what if a Pod inside the Kubernetes cluster becomes compromised? For example, if an attacker gains control over a Pod, they can start listening to traffic between other Pods in the Kubernetes cluster. If the internal communication is sent as plaintext, it will be very easy for the attacker to gain access to sensitive information sent between Pods in the cluster. To minimize the damage caused by such an intrusion, mutual authentication can be used to prevent an attacker from eavesdropping on internal network traffic.

To enable the use of mutual authentication managed by Istio, Istio needs to be configured both on the server side using a policy called PeerAuthentication, and on the client side using a DestinationRule.

The policy is configured in the common Helm chart's template, _istio_base.yaml. The manifest looks like this:

```
apiVersion: security.istio.io/v1beta1
kind: PeerAuthentication
metadata:
  name: default
spec:
  mtls:
    mode: PERMISSIVE
```

As mentioned in the *Introducing Istio API objects* section, the PeerAuthentication policy is configured to allow both mTLS and plain HTTP requests using PERMISSIVE mode. This enables Ku-bernetes to call liveness and readiness probes using plain HTTP.

We have also already met the DestinationRule manifests, in the *Content in the _istio_dr_mutual_tls.yaml template* section. The central part of the DestinationRule manifests for requiring mTLS looks like this:

```
  trafficPolicy:
    tls:
      mode: ISTIO_MUTUAL
```

To verify that the internal communication is protected by mTLS, perform the following steps:

1. Ensure that the load tests started in the preceding *Observing the service mesh* section are still running and report 200 (OK).
2. Go to the Kiali graph in a web browser (https://kiali.minikube.me).
3. Click on the **Display** button and enable the **Security** label. The graph will show a padlock on all communication links that are protected by Istio's automated mutual authentication, as follows:

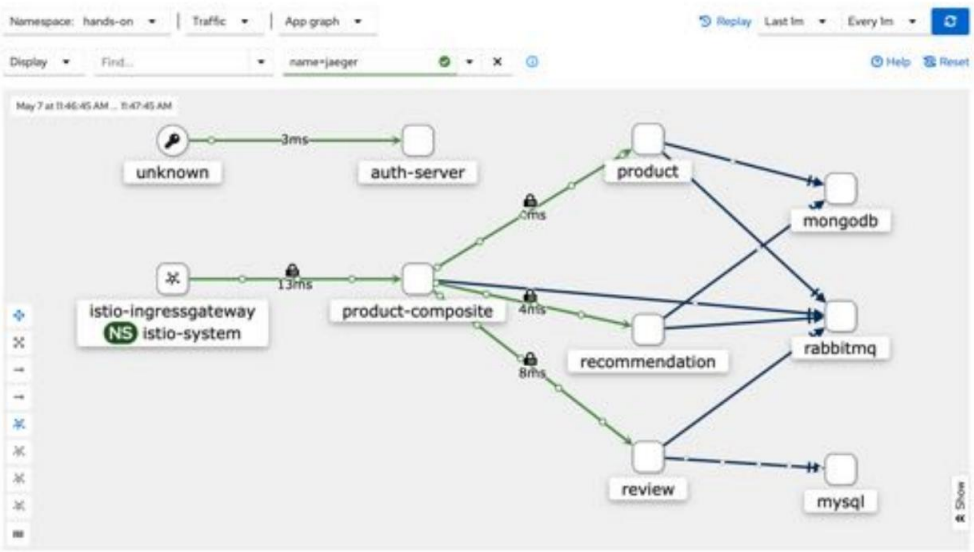


Figure 18.18: Inspecting mTLS settings in Kiali

Expect a padlock on all links.

Calls to RabbitMQ, MySQL, and MongoDB are not handled by Istio proxies and therefore require manual configuration to be protected using TLS, if required.

With this, we have seen all three security mechanisms in Istio in action, and it is now time to see how Istio can help us to verify that a service mesh is resilient.

Ensuring that a service mesh is resilient

In this section, we will learn how to use Istio to ensure that a service mesh is resilient, that is, that it can handle temporary faults in a service mesh. Istio comes with mechanisms similar to what the Spring Framework offers in terms of timeouts, retries, and a type of circuit breaker called **out -lier detection** to handle temporary faults. When it comes to deciding whether language-native mechanisms should be used to handle temporary faults or whether this should be delegated to a service mesh such as Istio, I tend to favor using language-native mechanisms, as in the examples in *Chapter 13, Improving Resilience Using Resilience4j.* In many cases, it is important to keep the logic for handling errors, for example, handling fallback alternatives for a circuit breaker, together with other business logic for a microservice. Keeping the logic for handling temporary faults in the source code also makes it easier to test it using, for example, JUnit and testcontainers, something that becomes much more complex if handling temporary faults is delegated to a service mesh such as Istio.

There are cases when the corresponding mechanisms in Istio could be of great help. For example, if a microservice is deployed and it is determined that it can't handle temporary faults that occur in production from time to time, then it can be very convenient to add a timeout or a retry mechanism using Istio instead of waiting for a new release of the microservice with corresponding error handling features put in place.

Another capability in the area of resilience that comes with Istio is the capability to inject faults and delays into an existing service mesh. Why might we want to do that?

Injecting faults and delays in a controlled way is very useful to verify that the resilient capabil-ities in the microservices work as expected! We will try them out in this section, verifying that the retry, timeout, and circuit breaker in the product-composite microservice work as expected.

In *Chapter 13, Improving Resilience Using Resilience4j* (refer to the *Adding programmable delays and random errors* section), we added support for injecting faults and delays into the microservices source code. That source code should preferably be replaced by using Istio's capabilities for injecting faults and delays at runtime, as demonstrated in the following subsections.

We will begin by injecting faults to see whether the retry mechanisms in the product-composite microservice work as expected. After that, we will delay the responses from the product service and verify that the circuit breaker handles the delay as expected.

Testing resilience by injecting faults

Let's make the product service throw random errors and verify that the microservice landscape handles this correctly. We expect the retry mechanism in the product-composite microservice to kick in and retry the request until it succeeds or its limit of the maximum number of retries is reached. This will ensure that a short-lived fault does not affect the end user more than the

delay introduced by the retry attempts. Refer to the *Adding a retry mechanism* section of *Chapter 13, Improving Resilience Using Resilience4j,* for a recap on the retry mechanism in the product- composite microservice.

Faults can be injected using a virtual service such as kubernetes/resilience-tests/product-virtual-service-with-faults.yml. This appears as follows:

```
apiVersion: networking.istio.io/v1beta1
kind: VirtualService
metadata:
  name: product
spec:
  hosts:
    - product
  http:
  - route:
    - destination:
        host: product
    fault:
      abort:
        httpStatus: 500
        percentage:
          value: 20
```

The definition says that 20% of the requests sent to the product service will be aborted with the HTTP status code 500 (Internal Server Error).

Perform the following steps to test this:

1. Ensure that the load tests using siege, as started in the *Observing the service mesh* section, are running.
2. Apply the fault injection with the following command:

```
kubectl apply -f kubernetes/resilience-tests/product-virtual-service-with-faults.yml
```

3. Monitor the output from the siege load testing tool. Expect output similar to the following:

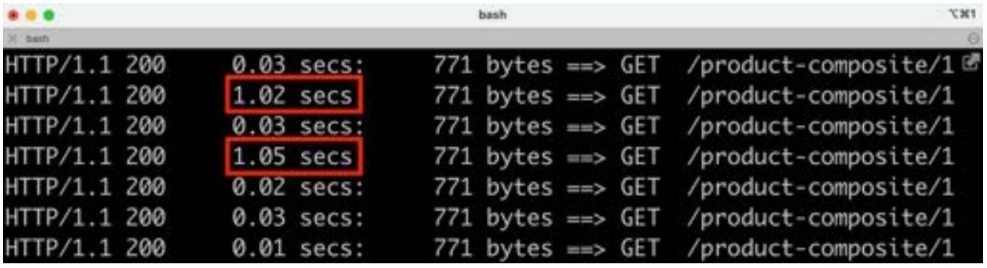


Figure 18.19: Observing the retry mechanism in action

From the output, we can see that all requests are still successful (in other words, status 200 (OK) is returned); However, some of them (20%) take an extra second to complete. This indicates that the retry mechanism in the product-composite microservice has kicked in and has retried a failed request to the product service.

4. Conclude the tests by removing the fault injection with the following command:

```
kubectl delete -f kubernetes/resilience-tests/product-virtual-service-with-faults.yml
```

Let's now move on to the next section, where we will inject delays to trigger the circuit breaker.

Testing resilience by injecting delays

As we learned in *Chapter 13, Improving Resilience Using Resilience4j,* a circuit breaker can be used to prevent problems due to the slow or complete lack of response of services after accepting requests.

Let's verify that the circuit breaker in the product-composite service works as expected by injecting a delay into the product service using Istio. A delay can be injected using a virtual service.

Refer to kubernetes/resilience-tests/product-virtual-service-with-delay.yml. Its code appears as follows:

```
apiVersion: networking.istio.io/v1beta1
kind: VirtualService
metadata:
  name: product
spec:
  hosts:
    - product
  http:
  - route:
    - destination:
        host: product
    fault:
      delay:
        fixedDelay: 3s
        percent: 100
```

This definition says that all requests sent to the product service will be delayed by 3 seconds.

Requests sent to the product service from the product-composite service are configured to time out after two seconds. The circuit breaker is configured to open its circuit if three consecutive requests fail. When the circuit is open, it will fail fast; In other words, it will immediately throw an exception without attempting to call the underlying service. The business logic in the product-composite microservice will catch this exception and apply fallback logic. For a recap, see *Chapter 13, Improving Resilience Using Resilience4j* (refer to the *Adding a circuit breaker and a time limiter* section).

Perform the following steps to test the circuit breaker by injecting a delay:

1. Stop the load test by pressing *Ctrl* + *C* in the terminal window where siege is running.
2. Create a temporary delay in the product service with the following command:

```
kubectl apply -f kubernetes/resilience-tests/product-virtual-service-with-delay.yml
```

3. Acquire an access token as follows:

```
ACCESS_TOKEN=$(curl https://writer:secret-writer@minikube.me/oauth2/
token -d grant_type=client_credentials -d scope="product:read product:write" -ks |
jq .access_token -r)

echo ACCESS_TOKEN=$ACCESS_TOKEN
```

4. Send six requests in a row:

```
for i in {1..6}; do time curl -k https://minikube.me/product-composite/1 -H "Authorization:
Bearer $ACCESS_TOKEN"; donated
```

Expect the following:

- The circuit opens up after the first three failed calls.
- The circuit breaker applies fast-fail logic for the last three calls.
- A fallback response is returned for the last three calls.

The responses from the first three calls are expected to be a timeout-related error message with a response time of 2 seconds (in other words, the timeout time). Expect responses for the first three calls along the lines of the following:

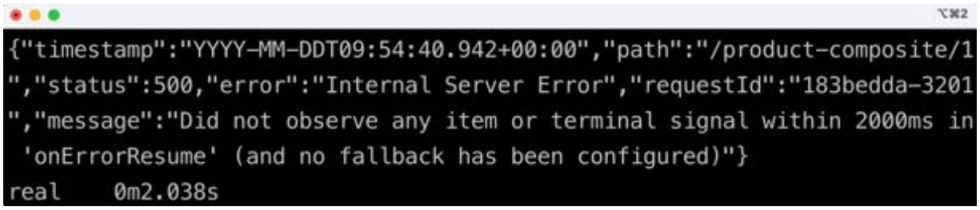


Figure 18.20: Observing timeouts

The responses from the last three calls are expected to come from the fallback logic with a short response time. Expect responses for the last three calls as follows:

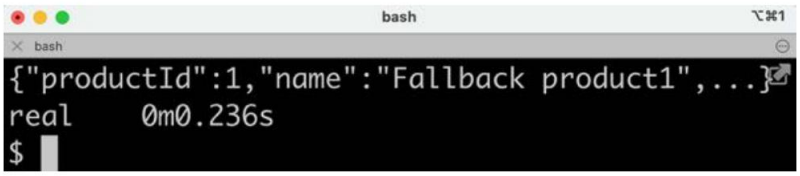


Figure 18.21: Fallback method in action

5. Simulate the delay problem being fixed by removing the temporary delay with the following command:

```
kubectl delete -f kubernetes/resilience-tests/product-virtual-service-with-delay.yml
```

6. Verify that the correct answers are returned again, and without any delay, by sending a new request using the for loop command in *step* 4.

If you want to check the state of the circuit breaker, you can do it with the following command:

```
curl -ks https://health.minikube.me/actuator/health | jq
-r .components.circuitBreakers.details.product.details.state
```

It should report CLOSED, OPEN, or HALF_OPEN, depending on its state.

This proves that the circuit breaker reacts as expected when we inject a delay using Istio. This concludes testing the features in Istio that can be used to verify that the microservice landscape is resilient. The final feature we will explore in Istio is its support for traffic management; we will see how it can be used to enable deployments with zero downtime.

Performing zero-downtime updates

As mentioned in *Chapter 16, Deploying Our Autonomous Microservices to Kubernetes,* being able to deploy an update without downtime becomes crucial with a growing number of microservices that are updated independently of one another.

In this section, we will learn about Istio's traffic management and routing capabilities and how they can be used to perform deployments of new versions of microservices without requiring any downtime. In *Chapter 15, Introduction to Kubernetes,* we learned that Kubernetes can be used to perform a rolling upgrade without requiring any downtime. Using the Kubernetes rolling upgrade mechanism automates the entire process, but, unfortunately, it provides no option to test the new version before all users are routed to it.

Using Istio, we can deploy the new version but initially route all users to the existing version (called the **old** version in this chapter). After that, we can use Istio's fine-grained routing mechanism to control how users are routed to the new and the old versions. We will see how two popular upgrade strategies can be implemented using Istio:

- **Canary deployments:** When using canary deployments, all users are routed to the old version, except for a group of selected test users who are routed to the new version. When the test users have approved the new version, regular users can be routed to the new version using a blue-green deploy.
- **Blue-green deployments:** Traditionally, a blue-green deploy means that all users are switched to either the blue or the green version, one being the new version and the other being the old version. If something goes wrong when switching over to the new version, it is very simple to switch back to the old version. Using Istio, this strategy can be refined by gradually shifting users over to the new version, for example, starting with 20% of the users and then slowly increasing the percentage. At all times, it is very easy to route all users back to the old version if a fatal error is revealed in the new version.

As already stated in *Chapter 16,* it is important to remember that a prerequisite for these types of upgrade strategies is that the upgrade is **backward-compatible.** Such an upgrade is compatible both in terms of APIs and message formats, which are used to communicate with other services and database structures. If the new version of the microservice requires changes to external APIs, message formats, or database structures that the old version can't handle, these upgrade strategies can't be applied.

We will go through the following deployment scenario:

1. We will start by deploying the v1 and v2 versions of the microservices, with routing configured to send all requests to the v1 version of the microservices.
2. Next, we will allow a test group to run canary tests; that is, we'll verify the new v2 versions of the microservices. To simplify the tests somewhat, we will only deploy new versions of the core microservices, that is, the product, recommendation, and review microservices.
3. Finally, we will start to move regular users over to the new versions using a blue-green deploy; initially, a small percentage of users and then, over time, more and more users until, eventually, they are all routed to the new version. We will also see how we can quickly switch back to the v1 version if a fatal error is detected in the new v2 version.

Let's first see what changes need to be applied to the source code to be able to deploy and route traffic to two concurrent versions, v1 and v2, of the core microservices.

Source code changes

To be able to run multiple versions of a microservice concurrently, the Deployment objects and their corresponding Pods must have different names, for example, product-v1 and product-v2. There must, however, be only one Kubernetes Service object per microservice. All traffic to a specific microservice always goes through the same Service object, irrespective of what version of the Pod the request will be routed to in the end. To configure the current routing rules for canary tests and blue-green deployments, Istio's VirtualService and DestinationRule objects are used. Finally, the values.yaml file in the prod-env Helm chart is used to specify the versions of each microservice that will be used in the production environment.

Let's go through the details for each definition in the following subsections:

- Virtual services and destination rules
- Deployments and services
- Tying things together in the prod-env Helm chart

Virtual services and destination rules

To split the traffic between two versions of a microservice, we need to specify the weight distribution bution between the two versions in a virtual service on the sender side. The virtual service will spread the traffic between two subsets, called old and new. The exact meaning of the new and old subsets is defined in a corresponding DestinationRule on the receiver side. It uses labels to determine which Pods run the old and new versions of the microservice.

To support canary tests, a routing rule is required in the virtual services that always routes the canary testers to the new subset. To identify canary testers, we will assume that requests from a canary tester contain an HTTP header named X-group with the value test.

A template has been added to the common Helm chart for creating a set of virtual services that can split the traffic between two versions of a microservice. The template is named _istio_vs_ green_blue_deploy.yaml and looks like this:

```
{{- define "common.istio_vs_green_blue_deploy" -}}
{{- range $name := .Values.virtualServices }}
apiVersion: networking.istio.io/v1beta1
kind: VirtualService
metadata:
    name: {{ $name }}
spec:
```

```
    hosts:
    - {{ $name }}
    http:
    - match:
      - headers:
          X-group:
            exact: test
      route:
      - destination:
          host: {{ $name }}
          subset: new
    - route:
      - destination:
          host: {{ $name }}
          subset: old
        weight: 100
      - destination:
          host: {{ $name }}
          subset: new
        weight: 0
---
{{- end -}}
{{- end -}}
```

From the template, we can see the following:

- The range directive loops over the elements defined in the virtualServices variable
- The hosts field in the spec part of the manifest is used to specify the names of the Kubernetes service that this VirtualService will apply to
- In the http section, two routing destinations are declared, along with a weight:
  - One route matching the canary testers' HTTP header, X-group, set to test. This route always sends the requests to the new subset.
  - One route destination for the old subset and one for the new subset.
- The weight is specified as a percentage, and the sum of the weights will always be 100.
- All traffic is initially routed to the old subset.

To be able to route canary testers to the new versions based on header-based routing, the product -composite microservice has been updated to forward the HTTP header, X-group. Refer to the getCompositeProduct() method in the se.magnus.microservices.composite.product.services.ProductCompositeServiceImpl class for details.

For the destination rules, we will reuse the template introduced in the *Content in the _istio_dr_mutual_tls.yaml template* section. This template will be used by the prod-env Helm chart to specify the versions of the microservices to be used. This is described in the *Tying things together in the prod-env Helm chart* section.

Deployments and services

To make it possible for a destination rule to identify the version of a Pod based on its labels, a version label has been added in the template for deployments in the common Helm chart, _deployment.yaml. Its value is set to the tag of the Pod's Docker image. We will use the Docker image tags v1 and v2, so that will also be the value of the version label. The added line looks like this:

```
        version: {{ .Values.image.tag }}
```

To give the Pods and their Deployment objects names that contain their version, their default names have been overridden in the prod-env chart. In their values.yaml files, the fullnameOverride field is used to specify a name that includes version info. This is done for the three core microservices and looks like this:

```
product:
  fullnameOverride: product-v1

recommendation:
  fullnameOverride: recommendation-v1

review:
  fullnameOverride: review-v1
```

An undesired side effect of this is that the corresponding Service objects will also get a name that includes the version info. As explained previously, we need to have one service that can route requests to the different versions of the Pods. To avoid this naming problem, the Service template, _service.yaml, in the common Helm chart, is updated to use the common.name template instead of the common.fullname template used previously in *Chapter 17.*

Finally, to be able to deploy multiple versions of the three core microservices, their Helm charts have been duplicated in the kubernetes/helm/components folder. The name of the new charts is suffixed with -green. The only difference compared to the existing charts is that they don't include the Service template from the common chart, avoiding the creation of two Service objects per core microservice. The new charts are named product-green, recommendation-green, and review-green.

Tying things together in the prod-env Helm chart

The prod-env Helm chart includes the _istio_vs_green_blue_deploy.yaml template from the common Helm chart, as well as the templates included by the dev-env chart; see the *Creating the service mesh* section.

The three new *-green Helm charts for the core microservices are added as dependencies to the Chart.yaml file.

In its values.yaml file, everything is tied together. From the previous section, we have seen how the v1 versions of the core microservices are defined with names that include version info.

For the v2 versions, the three new *-green Helm charts are used. The values are the same as for the v1 versions except for the name and Docker image tag. For example, the configuration of the v2 version of the product microservice looks like this:

```
product-green:
  fullnameOverride: product-v2
  image:
    tag: v2
```

To declare virtual services for the three core microservices, the following declaration is used:

```
virtualServices:
  - product
  - recommendation
  - review
```

Finally, the destination rules are declared in a similar way to the dev-env Helm chart. The main difference is that we now use subsets to declare the actual versions that should be used when traffic is routed by the virtual services to either the old or the new subset. For example, the destination rule for the product microservice is declared like this:

```
destinationRules:
  - ...
  - name: product
    subsets:
    - labels:
        version: v1
      name: old
    - labels:
        version: v2
      name: new
...
```

From the preceding declaration, we can see that traffic sent to the old subset is directed to v1 Pods of the product microservice and to v2 Pods for the new subset.

For details, see the file in the prod-env chart available in the kubernetes/helm/environments/prod-env folder.

Note that this is where we declare for the production environment what the existing (old) and the coming (new) versions are, v1 and v2 in this scenario. In a future scenario, where it is time to upgrade v2 to v3, the old subset should be updated to use v2 and the new subset should use v3.

Now, we have seen all the changes to the source code and we are ready to deploy the v1 and v2 versions of the microservices.

Deploying the v1 and v2 versions of the microservices with routing to the v1 version

To be able to test the v1 and v2 versions of the microservices, we need to remove the development environment we used earlier in this chapter and create a production environment where we can deploy the v1 and v2 versions of the microservices.

To achieve this, run the following commands:

1. Uninstall the development environment:

```
helm uninstall hands-on-dev-env
```

2. To monitor the termination of Pods in the development environment, run the following command until it reports No resources found in hands-on namespace.:

```
kubectl get pods
```

3. Start MySQL, MongoDB, and RabbitMQ outside of Kubernetes:

```
eval $(minikube docker-env)
docker compose up -d mongodb mysql rabbitmq
```

4. Tag the Docker images with v1 and v2 versions:

```
docker tag hands-on/auth-server hands-on/auth-server:v1
docker tag hands-on/product-composite-service hands-on/product-composite-
service:v1
docker tag hands-on/product-service hands-on/product-service:v1
docker tag hands-on/recommendation-service hands-on/recommendation-
service:v1
docker tag hands-on/review-service hands-on/review-service:v1

docker tag hands-on/product-service hands-on/product-service:v2
docker tag hands-on/recommendation-service hands-on/recommendation-
service:v2
docker tag hands-on/review-service hands-on/review-service:v2
```

The v1 and v2 versions of the microservices will be the same versions of the microservices in this test. But it doesn't matter to Istio, so we can use this simplified approach to test Istio's routing capabilities.

5. Deploy the system landscape using Helm and wait for all deployments to complete:

```
helm install hands-on-prod-env \
    kubernetes/helm/environments/prod-env \
    -n hands-on --wait
```

6. Once the deployment is complete, verify that we have the v1 and v2 Pods up and running for the three core microservices with the following command:

```
kubectl get pods
```

Expect a response like this:

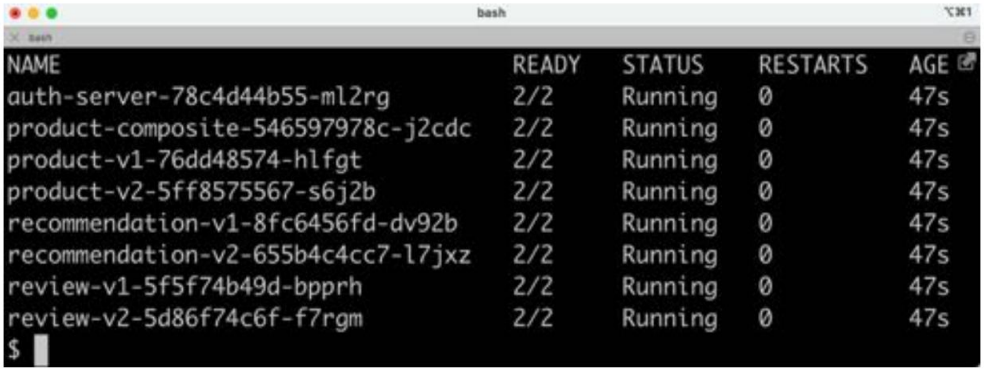


Figure 18.22: v1 and v2 Pods deployed at the same time

7. Run the usual tests to verify that everything works:

```
./test-em-all.bash
```

Unfortunately, the tests will fail initially with an error message like this:

```
- Response Body: Jwks doesn't have key to match kid or something from Jwt
```

This error is caused by the Istio daemon, istiod, caching the JWKS public keys from the auth server in the development environment. The auth server in the production environment will have new JWKS keys but the same identity as istiod, so it tries to reuse the old JWKS public keys, causing this failure. Istio caches JWKS public keys for 20 minutes by default, but when installing Istio, we lowered the refresh interval to 15 seconds; see the *Deploying Istio in a Kubernetes cluster* section. So, after waiting a short while, up to a minute depending on how quickly the refreshed keys are propagated, you should be able to run the tests successfully. The tests might still fail once the issue with cached JWKS has disappeared, with errors such as this:

```
Test FAILED, EXPECTED VALUE: 3, ACTUAL VALUE: 0, WILL ABORT
```

Then, simply rerun the command, and it should run fine! These errors are secondary failures caused by the original error caused by the JWKS cache.

Expect output that is similar to what we have seen in the previous chapters:

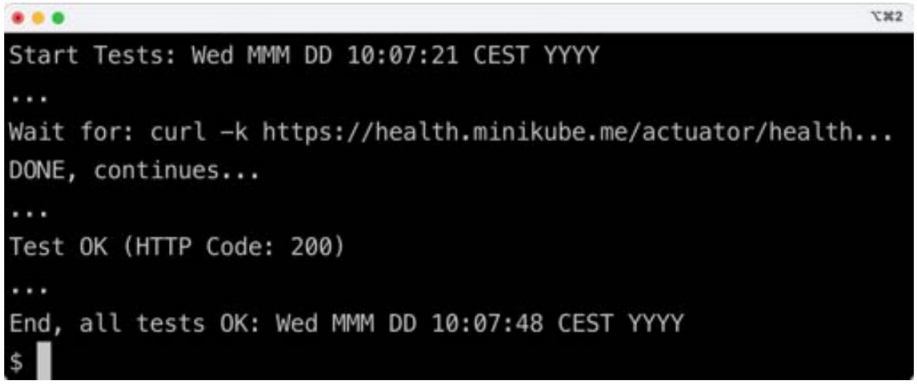


Figure 18.23: Tests run successfully

We are now ready to run some **zero-downtime deployment** tests. Let's begin by verifying that all traffic goes to the v1 version of the microservices!

Verifying that all traffic initially goes to the v1 version of the microservices

To verify that all requests are routed to the v1 version of the microservices, we will start up the load test tool, siege, and then observe the traffic that flows through the service mesh using Kiali.

Perform the following steps:

1. Get a new access token and start the siege load test tool with the following commands:

```
ACCESS_TOKEN=$(curl https://writer:secret-writer@minikube.me/oauth2/
token -d grant_type=client_credentials -d scope="product:read product:write" -ks |
jq .access_token -r)

echo ACCESS_TOKEN=$ACCESS_TOKEN

siege https://minikube.me/product-composite/1 -H "Authorization: Bearer $ACCESS_TOKEN"
-c1 -d1 -v
```

2. Go to the **Graph** view in Kiali's web UI (https://kiali.minikube.me):

    to. Click on the **Display** menu button and select **Namespace Boxes.**

    b. Click on the **App graph** menu button and select **Versioned app graph.**

c. Expect only traffic to the **v1** version of the microservices, as follows:

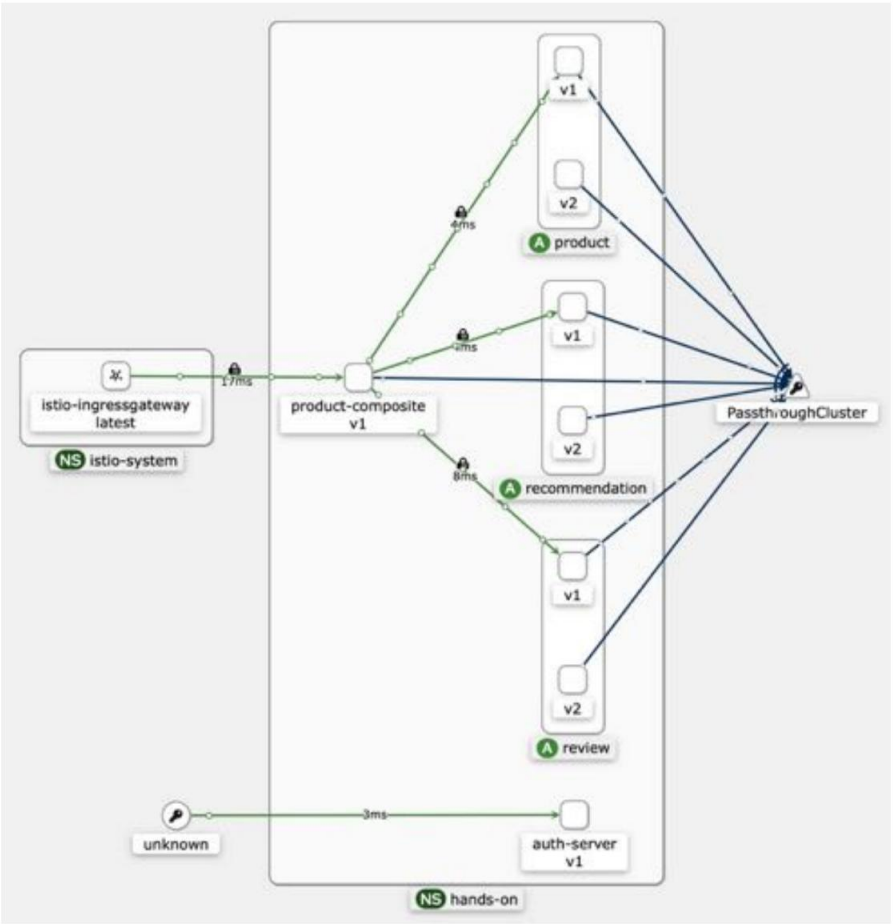


Figure 18.24: All requests go to the v1 Pods

This means that even though the v2 versions of the microservices are deployed, they do not get any traffic routed to them. Let's now try out canary tests where selected test users are allowed to try out the v2 versions of the microservices!

Running Canarian tests

To run a canary test so that some users are routed to the new versions while all other users are still routed to the old versions of the deployed microservices, we need to add the X-group HTTP header set to the value test in our requests sent to the external API.

To see which version of a microservice served a request, the serviceAddresses field in the response can be inspected. The serviceAddresses field contains the hostname of each service that took part in creating the response. The hostname is equal to the name of the Pod, so we can find the version in the hostname; for example, product-v1-... for a product service of version v1, and product-v2-... for a product service of version v2.

Let's begin by sending a normal request and verifying that it is the v1 versions of the microservices that respond to our request. Next, we'll send a request with the X-group HTTP header set to the value test and verify that the new v2 versions are responding.

To do this, perform the following steps:

1. Perform a normal request to verify that the request is routed to the v1 version of the microservices by using jq to filter out the serviceAddresses field in the response:

```
ACCESS_TOKEN=$(curl https://writer:secret-writer@minikube.me/oauth2/
token -d grant_type=client_credentials -d scope="product:read product:write" -ks |
jq .access_token -r)

echo ACCESS_TOKEN=$ACCESS_TOKEN

curl -ks https://minikube.me/product-composite/1 -H "Authorization: Bearer $ACCESS_TOKEN"
| jq .serviceAddresses
```

Expect a response along the lines of the following:

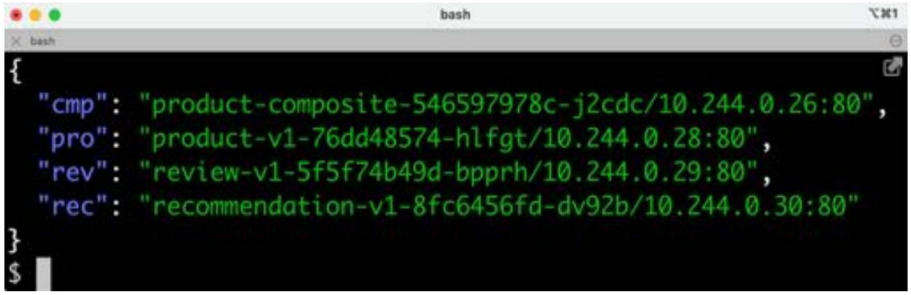


Figure 18.25: All requests go to the v1 Pods

As expected, all three core services are v1 versions of the microservices.

2. If we add the X-group=test header, we expect the request to be served by v2 versions of the core microservices. Run the following command:

```
curl -ks https://minikube.me/product-composite/1 -H "Authorization: Bearer $ACCESS_TOKEN"
-H "X-group: test" | jq .serviceAddresses
```

Expect a response similar to the following:

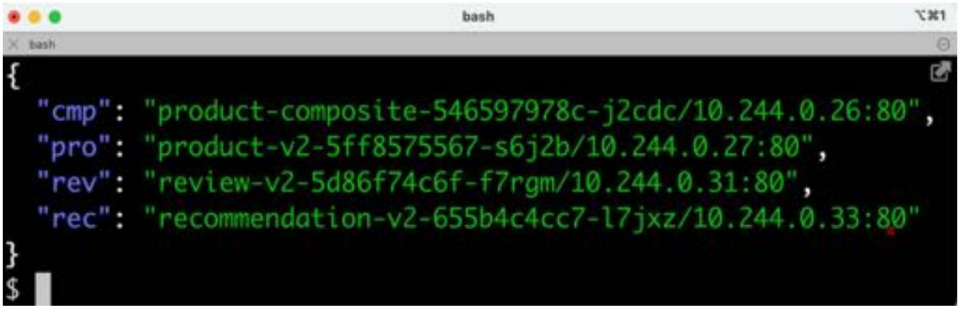


Figure 18.26: Setting the HTTP header to X-group=test makes the requests go to the v2 Pods

As expected, all three core microservices that respond are now v2 versions; as a canary tester, we are routed to the new v2 versions!

Given that the canary tests returned the expected results, we are ready to allow normal users to be routed to the new v2 versions using a blue-green deployment.

Running a blue-green deployment

To route a portion of the normal users to the new v2 versions of the microservices, we need to modify the weight distribution in the virtual services. They are currently 100/0; in other words, all traffic is routed to the old v1 versions. We can achieve this, as we did before, by editing the manifest files of the virtual services and executing a kubectl apply command to make the changes take effect. As an alternative, we can use the kubectl patch command to change the weight distribution directly on the virtual service objects in the Kubernetes API server.

I find the patch command useful when making a number of changes to the same objects to try something out, for example, to change the weight distribution in the routing rules. In this section, we will use the kubectl patch command to quickly change the weight distribution in the routing rules between the v1 and v2 versions of the microservices. To get the state of a virtual service after a few kubectl patch commands have been executed, a command such as kubectl get vs NNN -o yaml can be issued. For example, to get the state of the virtual service of the product microservice, issue the following command: kubectl get vs product -o yaml.

Since we haven't used the kubectl patch command before and it can be a bit involved to start with, let's undertake a short introduction to see how it works before we perform the blue-green deployment.

A short introduction to the kubectl patch command

The kubectl patch command can be used to update specific fields in an existing object in the Kubernetes API server. We will try the patch command on the virtual service for the review micro- service, named review. The relevant parts of the definition of the virtual service, review, appear as follows:

```
spec:
   http:
   - match:
      ...
   - route:
      - destination:
           host: review
           subset: old
        weight: 100
      - destination:
           host: review
           subset: new
        weight: 0
```

An example patch command that changes the weight distribution of the routing to the v1 and v2 Pods in the review microservice appears as follows:

```
kubectl patch virtualservice review --type=json -p='[
   {"op": "add", "path": "/spec/http/1/route/0/weight", "value": 80},
   {"op": "add", "path": "/spec/http/1/route/1/weight", "value": 20}
]'
```

The command will configure the routing rules of the review microservice to route 80% of the requests to the old version and 20% of the requests to the new version.

To specify that the weight value should be changed in the review virtual service, the /spec/http/1/route/0/weight path is given for the old version, and the /spec/http/1/route/1/weight path is for the new version.

The 0 and 1 in the path are used to specify the index of array elements in the definition of the virtual service. For example, http/1 means the second element in the array under the http element. See the definition of the preceding review virtual service.

From the definition, we can see that the first element with index 0 is the match element, which we will not change. The second element is the route element, which we want to change.

Now that we know a bit more about the kubectl patch command, we are ready to carry out a blue-green deployment.

Performing the blue-green deployment

It is time to gradually move more and more users to the new versions using a blue-green deployment. To perform the deployment, run the following steps:

1. Ensure that the load test tool, siege, is still running. Note that it was started in the pre-ceding *Verifying that all traffic initially goes to the v1 version of the microservices* section.
2. To allow 20% of users to be routed to the new v2 version of the review microservice, we can patch the virtual service and change the weights with the following command:

```
kubectl patch virtualservice review --type=json -p='[
    {"op": "add", "path": "/spec/http/1/route/0/weight", "value":
    80},
    {"op": "add", "path": "/spec/http/1/route/1/weight", "value":
    20}
]'
```

3. To observe the change in the routing rule, go to the Kiali web UI (https://kiali.minikube.me) and select the **Graph** view.
4. Click on the **Display** menu and change the edge labels to **Traffic Distribution.**

5. Wait for a minute before the metrics are updated in Kiali so that we can observe the change. Expect the graph in Kiali to show something like the following:

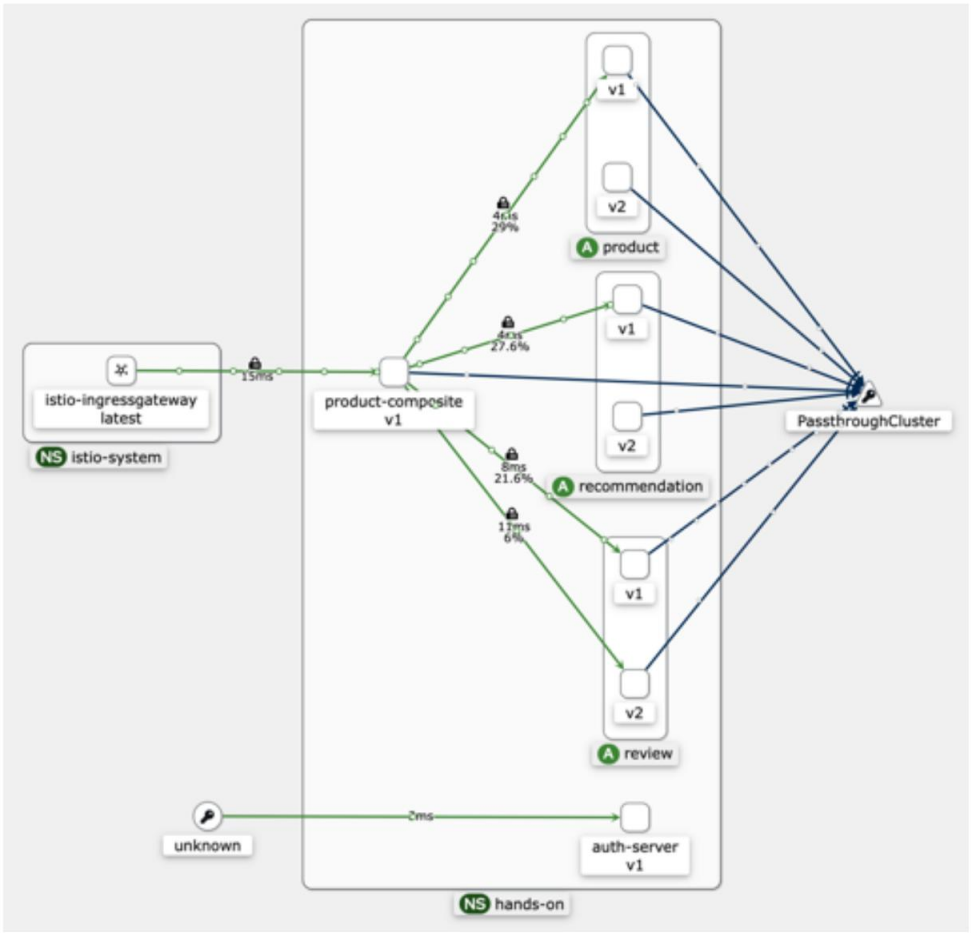


Figure 18.27: 80% goes to v1 services and 20% goes to v2 services

Depending on how long you have waited, the graph might look a bit different! In the screenshot, we can see that Istio now routes traffic to both the v1 and v2 versions of the review microservice.

Of the traffic that is sent to the review microservice from the product-composite microservice, 6% is routed to the new v2 Pod, and 21.6% to the old v1 Pod. This means that 6/(6 + 21.6) = 22% of the requests are routed to the v2 Pod, and 78% to the v1 Pod. This is in line with the 20/80 distribution we have requested.

Please feel free to try out the preceding kubectl patch command to affect the routing rules for the other core microservices, product and recommendation.

To simplify changing the weight distribution for all three core microservices, the./kubernetes/routing-tests/split-traffic-between-old-and-new-services.bash script can be used. For example, to route all traffic to the v2 version of all microservices, run the following script, feeding it with the weight distribution 0 100:

```
./kubernetes/routing-tests/split-traffic-between-old-and-new-services.bash
0 100
```

You have to give Kiali a minute or two to collect metrics before it can visualize the changes in routing, but remember that the change in the current routing is immediate!

Expect that requests are routed only to the v2 versions of the microservices in the graph after a while:

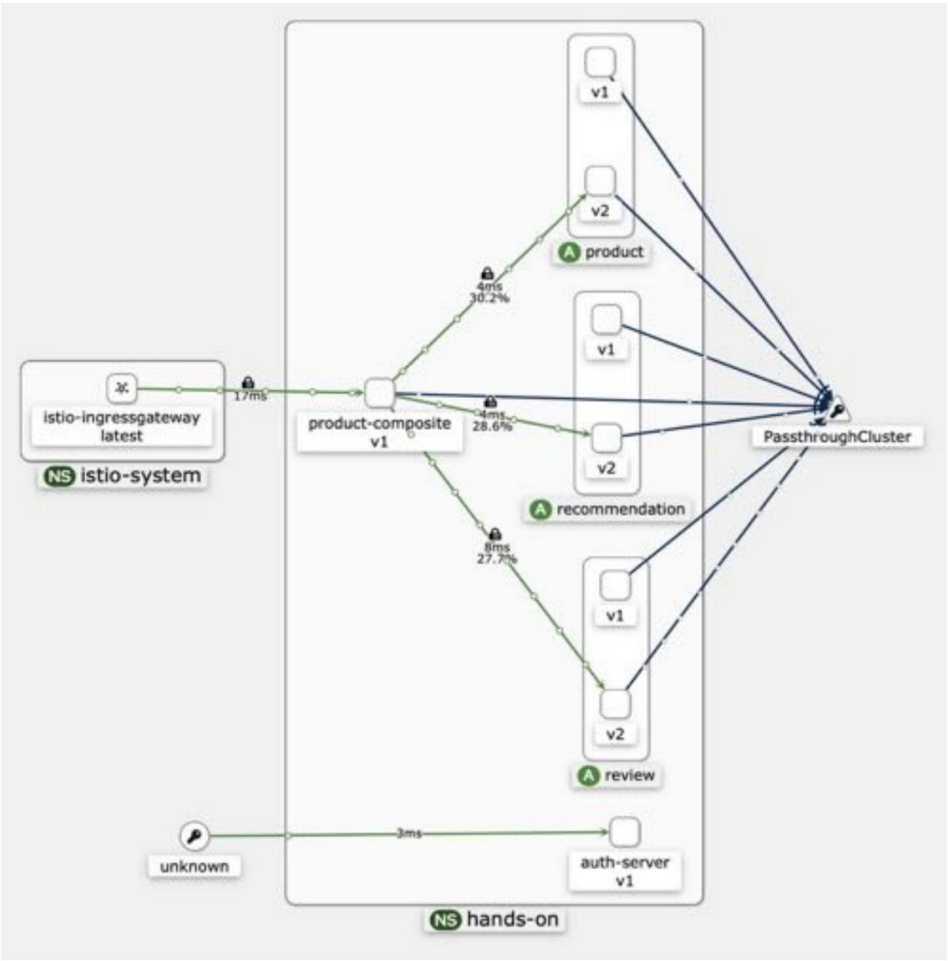


Figure 18.28: All traic goes to v2 services

Depending on how long you have waited, the graph might look a bit different!

If something goes terribly wrong following the upgrade to v2, the following command can be executed to revert all traffic to the v1 version of all microservices:

```
./kubernetes/routing-tests/split-traffic-between-old-and-new-services.bash
100 0
```

After a short while, the graph in Kiali should look like the screenshot in the previous *Verifying that all traffic initially goes to the v1 version of the microservices* section, showing all requests going to the v1 version of all microservices again.

This concludes the introduction to the service mesh concept and Istio as an implementation of it.

Before we wrap up the chapter, let's recap how we can run tests in Docker Compose to ensure that the source code of our microservices does not rely on either the deployment in Kubernetes or the presence of Istio.

Running tests with Docker Compose

As mentioned a few times now, it is important to ensure that the source code of the microservices doesn't become dependent on a platform such as Kubernetes or Istio from a functional perspective.

To verify that the microservices work as expected without the presence of Kubernetes and Istio, run the tests as described in *Chapter 17* (refer to the *Testing with Docker Compose* section). Since the default values of the test script, test-em-all.bash, have been changed, as described previously in the *Running commands to create the service mesh* section, the following parameters must be set when using Docker Compose: USE_K8S=false HOST=localhost PORT=8443 HEALTH_URL=https://localhost:8443. For example, to run the tests using the default Docker Compose file, docker-compose.yml, run the following command:

```
USE_K8S=false HOST=localhost PORT=8443 HEALTH_URL=https://localhost:8443 ./test-em-
all.bash start stop
```

The test script should, as before, begin by starting all containers; it should then run the tests, and, finally, stop all containers. For details of the expected output, see *Chapter 17* (refer to the *Verifying that the microservices work without Kubernetes* section).

After successfully executing the tests using Docker Compose, we have verified that the microservices are dependent on neither Kubernetes nor Istio from a functional perspective. These tests conclude the chapter on using Istio as a service mesh.

Summary

In this chapter, we learned about the service mesh concept and Istio, an open source implementation of the concept. A service mesh provides capabilities for handling challenges in a system landscape of microservices in areas such as security, policy enforcement, resilience, and traffic management. A service mesh can also be used to make a system landscape of microservices observable by visualizing the traffic that flows through the microservices.

For observability, Istio can be integrated with Kiali, Jaeger, and Grafana (more on Grafana and Prometheus in *Chapter 20, Monitoring Microservices).* When it comes to security, Istio can be configured to use a certificate to protect external APIs with HTTPS and require that external re-quests contain valid JWT-based OAuth 2.0/OIDC access tokens. Finally, Istio can be configured to automatically protect internal communication using mutual authentication (with mTLS).

For resilience and robustness, Istio comes with mechanisms for handling retries, timeouts, and an outlier detection mechanism similar to a circuit breaker. In many cases, it is preferable to implement these resilience capabilities in the source code of the microservices, if possible. The ability in Istio to inject faults and delays is very useful for verifying that the microservices in the service mesh work together as a resilient and robust system landscape. Istio can also be used to handle zero-downtime deployments. Using its fine-grained routing rules, both canary and blue-green deployments can be performed.

One important area that we haven't covered yet is how to collect and analyze log files created by all microservice instances. In the next chapter, we will see how this can be done using a popular stack of tools known as the EFK stack, based on Elasticsearch, Fluentd, and Kibana.

Questions

1. What is the purpose of a proxy component in a service mesh?
2. What's the difference between a control plane and a data plane in a service mesh?
3. What is the istioctl kube-inject command used for?
4. What is the minikube tunnel command used for?
5. What tools are Istio integrated with for observability?
6. What configuration is required to make Istio protect communication within the service mesh using mutual authentication?
7. What can the abort and delay elements in a virtual service be used for?
8. What configuration is required to set up a blue-green deployment scenario?

19

Centralized Logging with the EFK Stack

In this chapter, we will learn how to collect and store log records from microservice instances, as well as how to search and analyze log records. As we mentioned in *Chapter* 1, *Introduction to Microservices,* it is difficult to get an overview of what is going on in a system landscape of microservices when each microservice instance writes log records to its local filesystem. We need a component that can collect the log records from the microservice's local filesystem and store them in a central database for analysis, search, and visualization. A popular open source-based solution for this is based on the following tools:

- **Elasticsearch,** a distributed database with great capabilities for searching and analyzing large datasets
- **Fluentd,** a data collector that can be used to collect log records from various sources, filter and transform the collected information, and finally send it to various consumers (for example, Elasticsearch)
- **Kibana,** a graphical frontend to Elasticsearch that can be used to visualize search results and run analyzes of the collected log records

Together, these tools are called the **EFK stack,** named after the initials of each tool.

The following topics will be covered in this chapter:

- Configuring Fluentd
- Deploying the EFK stack on Kubernetes for development and test usage
- Analyzing the collected log records

- Discovering log records from microservices and finding related log records
- Performing root cause analysis

Technical requirements

For instructions on how to install the tools used in this book and how to access the source code

For this book, see the following chapters:

- *Chapter 21, Installation Instructions for macOS*
- *Chapter 22, Installation Instructions for Microsoft Windows with WSL 2 and Ubuntu*

The code examples in this chapter all come from the source code in $BOOK_HOME/Chapter19.

If you want to view the changes applied to the source code in this chapter (that is, see the changes we made so that we can use the EFK stack for centralized log analysis), you can compare it with the source code for *Chapter 18, Using a Service Mesh to Improve Observability and Management.* You can use your favorite diff tool and compare the two folders, $BOOK_HOME/Chapter18 and $BOOK_HOME/Chapter19.

Introducing Fluentd

In this section, we will learn the basics of how to configure Fluentd. Before we do that, let's learn a bit about the background of Fluentd and how it works at a high level.

Overview of Fluentd

Historically, one of the most popular open source stacks for handling log records has been the ELK stack from Elastic (https://www.elastic.co), based on Elasticsearch, Logstash (used for log collection and transformation), and Kibana. Since Logstash runs on a Java VM, it requires a relatively large amount of memory. Over the years, a number of open source alternatives have been developed that require significantly less memory than Logstash, one of them being Fluentd (https://www.fluentd.org).

Fluentd is managed by the **Cloud Native Computing Foundation (CNCF)** (https://www.cncf.io), the same organization that manages the Kubernetes project. Therefore, Fluentd has become a natural choice as an open source-based log collector that runs in Kubernetes. Together with Elastic and Kibana, it forms the EFK stack.

CNCF maintains a list of alternative products for several categories, for example, for logging. For alternatives to Fluentd listed by CNCF, see https://landscape.cncf.io/card-mode?category=logging&grouping=category.

Fluentd is written in a mix of C and Ruby, using C for the performance-critical parts and Ruby where flexibility is of more importance, for example, allowing the simple installation of third-party plugins using Ruby's gem install command.

A log record is processed as an event in Fluentd and consists of the following information:

- A time field describing when the log record was created
- A tag field that identifies what type of log record it is – the tag is used by Fluentd's routing engine to determine how a log record will be processed
- A record field that contains the current log information, which is stored as a JSON object

A Fluentd configuration file is used to tell Fluentd how to collect, process, and finally send log records to various targets, such as Elasticsearch. A configuration file consists of the following types of core elements:

- <source>: Source elements describe where Fluentd will collect log records; for example, tailing log files that have been written to by Docker containers.

Tailing a log file means monitoring what is written to a log file. A frequently used Unix/Linux tool for monitoring what is appended to a file is named tile.

Source elements typically tag the log records, describing the type of log record. They could, for example, be used to tag log records to state that they come from containers running in Kubernetes.

- <filter>: Filter elements are used to process the log records. For example, a filter element can parse log records that come from Spring Boot-based microservices and extract interesting parts of the log message into separate fields in the log record. Extracting information into separate fields in the log record makes the information searchable by Elasticsearch.
  A filter element selects the log records to process based on their tags.
- <match>: Match elements decide where to send log records, acting as output elements. They are used to perform two main tasks:
  - Sending processed log records to targets such as Elasticsearch.
  - Routing to decide how to process log records. A routing rule can rewrite the tag and re-emit the log record into the Fluentd routing engine for further processing.
    A routing rule is expressed as an embedded <rule> element inside the <match> element. Output elements decide what log records to process, in the same way as a filter, based on the tag of the log records.

Fluentd comes with a number of built-in and external third-party plugins that are used by the source, filter, and output elements. We will see some of them in action when we walk through the configuration file in the next section. For more information on the available plugins, see Fluentd's documentation, which is available at https://docs.fluentd.org.

With this overview of Fluentd out of the way, we are ready to see how Fluentd can be configured to process the log records from our microservices.

Configuring Fluentd

The configuration of Fluentd is based on the configuration files from a Fluentd project on GitHub, fluentd-kubernetes-daemonset. The project contains Fluentd configuration files for how to collect log records from containers that run in Kubernetes and how to send them to Elasticsearch once they have been processed. We will reuse this configuration without changes, and it will simplify our own configuration to a great extent. The Fluentd configuration files can be found at https://github.com/fluent/fluentd-kubernetes-daemonset/tree/master/archived-image/
v1.4/debian-elasticsearch/conf.

The configuration files that provide this functionality are kubernetes.conf and fluent.conf. The kubernetes.conf configuration file contains the following information:

- Source elements that tail container log files and log files from processes that run outside of Kubernetes; for example, kubelet and the Docker daemon. The source elements also tag the log records from Kubernetes with the full name of the log file, with / replaced by . and prefixed with kubernetes. Since the tag is based on the full filename, the name contains the name of the namespace, Pod, and container, among other things. So, the tag is very useful for finding log records of interest by matching the tag.

  For example, the tag from the product-composite microservice could be something like kubernetes.var.log.containers.product-composite-7...s_hands-on_comp- e...b.log, while the tag for the corresponding istio-proxy microservice in the same Pod could be something like kubernetes.var.log.containers.product-composite- 7...s_hands-on_istio-proxy-1...3.log.

- A filter element that enriches the log records that come from containers running inside Kubernetes, along with Kubernetes-specific fields that contain information such as the names of the containers and the namespace they run in.

The main configuration file, fluent.conf, contains the following information:

- @include statements for other configuration files; for example, the kubernetes.conf file we described previously. It also includes custom configuration files that are placed in a specific folder, making it very easy for us to reuse these configuration files without any changes and provide our own configuration file that only handles processing related to our own log records. We simply need to place our own configuration file in the folder specified by the fluent.conf file.
- An output element that sends log records to Elasticsearch.

As described in the *Deploying Fluentd* section later on, these two configuration files will be package-aged into the Docker image we will build for Fluentd.

What's left to cover in our own configuration file is the following:

- Detecting and parsing Spring Boot-formatted log records from our microservices.
- Handling multiline stack traces. Stack traces are written to log files using multiple lines. This makes it hard for Fluentd to handle a stack trace as a single log record.
- Separating log records from the istio-proxy sidecars from the log records that were created by the microservices running in the same Pod. The log records that are created by istio-proxy don't follow the same pattern as the log patterns that are created by our Spring Boot-based microservices. Therefore, they must be handled separately so that Fluentd doesn't try to parse them as Spring Boot-formatted log records.

To achieve this, the configuration is, to a large extent, based on using the rewrite_tag_filter plugin. This plugin can be used for routing log records based on the concept of changing the name of a tag and then re-emitting the log record to the Fluentd routing engine.

This processing is summarized by the following UML activity diagram.

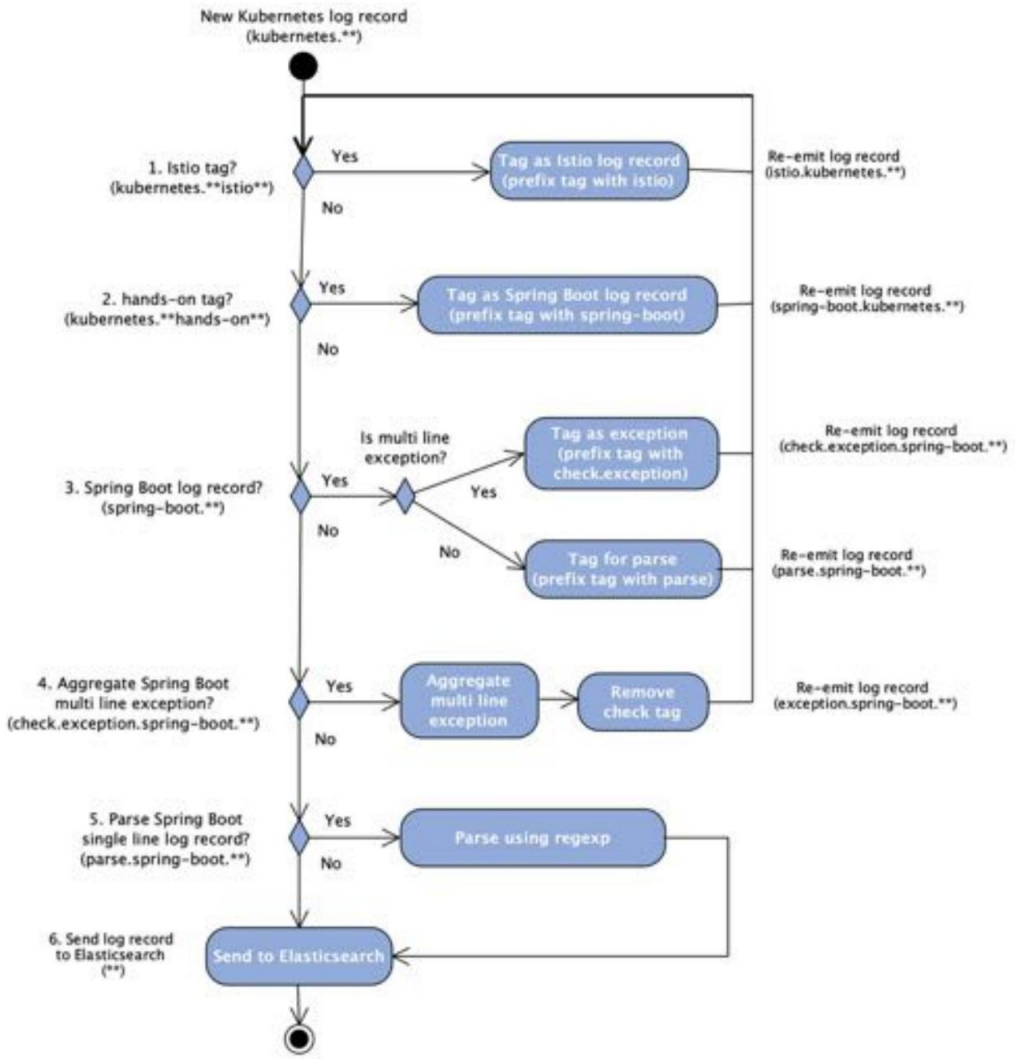
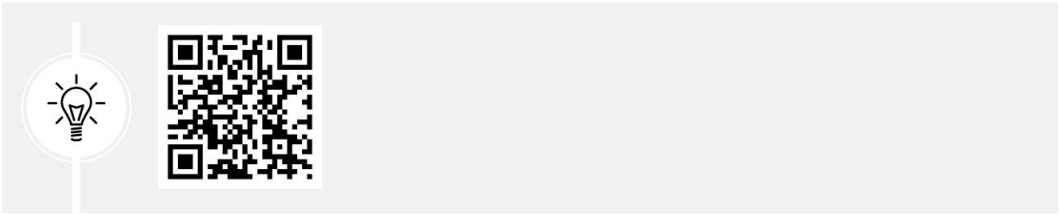


Figure 19.1: Fluent processing of log records



At a high level, the design of the configuration file looks as follows:

- The tags of all log records from Istio, including istio-proxy, are prefixed with istio so that they can be separated from the Spring Boot-based log records.
- The tags of all log records from the hands-on namespace (except for the log records from istio-proxy) are prefixed with spring-boot.
- The log records from Spring Boot are checked for the presence of multiline stack traces. If the log record is part of a multiline stack trace, it is processed by the third-party detect-exceptions plugin to recreate the stack trace. Otherwise, it is parsed using a regular expression to extract information of interest. See the *Deploying Fluentd* section for details on this third-party plugin.

The fluentd-hands-on.conf configuration file implements this activity diagram. The configuration file is placed inside a Kubernetes ConfigMap (see kubernetes/efk/fluentd-hands-on-configmap.yml). Let's go through this step by step, as follows:

1. First comes the definition of the ConfigMap and the filename of the configuration file, fluentd-hands-on.conf. It looks as follows:

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: fluentd-hands-on-config
  namespace: kube-system
data:
  fluentd-hands-on.conf: |
```

   We can see that the data element will contain the configuration of Fluentd. It starts with the filename and uses a vertical bar, |, to mark the beginning of the embedded configuration file for Fluentd.

2. The first <match> element matches the log records from Istio; that is, tags that are prefixed with Kubernetes and contain istio as either part of their namespace or part of their container name. It looks like this:

```
<match kubernetes.**istio**>
  @type rewrite_tag_filter
  <rule>
    key log
    pattern ^(.*)$
    istio tag.${tag}
  </rule>
</match>
```

Let's explain the preceding source code:

- The <match> element matches any tags that follow the kubernetes.**istio** pattern; that is, tags that start with Kubernetes and then contain the word istio somewhere in the tag name. The word istio can come from the name of either the namespace or the container; both are part of the tag.
- The <match> element contains only one <rule> element, which prefixes the tag with istio. The ${tag} variable holds the current value of the tag.
- Since this is the only <rule> element in the <match> element, it is configured to match all log records.
- Since all log records that come from Kubernetes have a log field, the key field is set to log; that is, the rule looks for a log field in the log records.
- To match any string in the log field, the pattern field is set to the ^(.*)$ regular expression. The ^ part marks the beginning of a string, while $ marks the end of a string. The (.*) part matches any number of characters, except for line breaks.
- The log records are re-emitted to the Fluentd routing engine. Since no other elements in the configuration file match tags starting with istio, the log records will be sent directly to the output element for Elasticsearch, which is defined in the fluent.conf file we described previously.

3. The second <match> element matches all log records from the hands-on namespace; that is, the log records that are emitted by our microservices. It looks like this:

```
<match kubernetes.**hands-on**>
  @type rewrite_tag_filter
```

```
        <rule>
          key log
          pattern ^(.*)$
          spring-boot tag.${tag}
        </rule>
      </match>
```

From the source code, we can see the following:

- The log records emitted by our microservices use formatting rules for the log message defined by Spring Boot, so their tags are prefixed with spring-boot. Then, they are re-emitted for further processing.
- The <match> element is configured in the same way as the <match kubernetes.**istio**> element we looked at previously, to match all records.

4. The third <match> element matches spring-boot log records and determines whether they are ordinary Spring Boot log records or part of a multiline stack trace. Since Spring Boot 3, Project Reactor has added extra information to stack traces to clarify what caused an exception . (For details, see https://projectreactor.io/docs/core/release/reference/debugging.html#reading-a-stack-trace-in-debug-mode.) To be able to parse the current stack trace, we will filter out this information. The <match> element looks like this:

```
      <match spring-boot.**>
        @type rewrite_tag_filter
        <rule>
          key log
          pattern /^\d{4}-\d{2}-\d{2}T\d{2}:\d{2}:\d{2}\.\d{3}
([-+]\d{2}:\d{2}|Z).*/
          tag parse.${tag}
        </rule>
        # Get rid of Reactor debug info:
        #
        # Suppressed: reactor.core.publisher.
FluxOnAssembly$OnAssemblyException:
        # Error has been observed at the following site(s):
        # *__checkpoint 'á¢ Handler se.magnus.microservices.core.
product.services.ProductServiceImpl#getProduct(HttpHeaders, int, int, int)
[DispatcherHandler]
        # *__checkpoint 'á¢ org.springframework.web.filter.reactive.
```

```
ServerHttpObservationFilter [DefaultWebFilterChain] # *__checkpoint 'â¢ HTTP
        GET "/product/1?faultPercent=100"
[ExceptionHandlingWebHandler]
        # Original Stack Trace:
        <rule>
            key log
            pattern /^\s+Suppressed:.*$/
            tag skip.${tag} </rule>

        <rule>
            key log
            pattern /^Error has been observed at the following site.*/ tag skip.${tag}

        </rule>
        <rule>
            key log
            pattern /^\s+\*__checkpoint.*/
            tag skip.${tag} </rule>

        <rule>
            key log
            pattern /^Original Stack Trace:.*/ tag skip.${tag}

        </rule>
        <rule>
            key log
            pattern /^.*/ tag
            check.exception.${tag}
        </rule>
    </match>
```

As seen in the source code, this is determined by using six <rule> elements:

- The first uses a regular expression to check whether the log field in the log element starts with a timestamp or not.
- If the log field starts with a timestamp, the log record is treated as an ordinary Spring Boot log record and its tag is prefixed with parse.

- Next follows four rule elements that are used to filter out the extra information added by Project Reactor; they all prefix the tag with skip.
- Otherwise, the last <rule> element will match, and the log record is handled as a multiline log record. Its tag is prefixed with check.exception.
- The log record is re-emitted in either case, and its tag will either start with check.exception.spring-boot, skip.spring-boot, or parse.spring-boot after this processing.

5. The fourth <match> element is used to get rid of the log output from Project Reactor; that is, match tags starting with skip.spring-boot. The <match> element applies the null output plugin that throws away the events. It looks like this:

```
<match skip.spring-boot.**>
  @type null
</match>
```

6. In the fifth <match> element, the selected log records have a tag that starts with check.exception.spring-boot; that is, log records that are part of a multiline stack trace. It looks like this:

```
<match check.exception.spring-boot.**>
  @type detect_exceptions
  Java languages
  remove_tag_prefix check
  message log
  multiline_flush_interval 5
</match>
```

The detect_exceptions plugin works like this:

- The detect_exceptions plugin is used to combine multiple one-line log records into a single log record that contains a complete stack trace
- Before a multiline log record is re-emitted into the routing engine, the check prefix is removed from the tag to prevent a never-ending processing loop of the log record

7. Finally, the configuration file consists of a <filter> element that parses Spring Boot log messages using a regular expression, extracting information of interest. It looks like this:

```
<filter parse.spring-boot.**>
  @type parser
  key_name log
  time_key time
  time_format %Y-%m-%dT%H:%M:%S.%N
  reserve_data true
  format /^(?<time>\d{4}-\d{2}-\d{2}
  T\d{2}:\d{2}:\d{2}\.\d{3}([-+]\d{2}:\d{2}|Z))\s+
  (?<spring.level>[^\s]+)\s+(?<spring.pid>\d+)\s+---\s+
  \[\s*(?<spring.thread>[^\]]+)\]\s+
  (\[(?<spring.service>[^,]*),(?<spring.trace>[^,]*),
  (?<spring.span>[^\]]*)]*\])\s+
  (?<spring.class>[^\s]+)\s*:\s+(?<log>.*)$/                    </filter>
```

Note that filter elements don't re-emit log records; instead, they just pass them on to the next element in the configuration file that matches the log record's tag.

The following fields are extracted from the Spring Boot log message that's stored in the log field in the log record:

- <time>: The timestamp for when the log record was created
- <spring.level>: The log level of the log record: FATAL, ERROR, WARN, INFO, DEBUG, or TRACE
- <spring.service>: The name of the microservice
- <spring.trace>: The trace ID used to perform distributed tracing
- <spring.span>: The span ID, the ID of the part of the distributed processing that this microservice executed
- <spring.pid>: The process ID
- <spring.thread>: The thread ID
- <spring.class>: The name of the Java class
- <log>: The current log message

The names of Spring Boot-based microservices are specified using the spring.application.name property. This property has been added to each microservice -vice-specific property file in the config repository, in the config-repo folder.

Getting regular expressions right can be challenging, to say the least. Fortunately, there are sev-eral websites that can help. When it comes to using regular expressions together with Fluentd, I recommend using the following site: https://fluentular.herokuapp.com/.

Now that we have been introduced to how Fluentd works and how the configuration file is constructed, we are ready to deploy the EFK stack.

Deploying the EFK stack on Kubernetes

Deploying the EFK stack on Kubernetes will be done in the same way as we have deployed our own microservices: using Kubernetes manifest files for objects such as Deployments, Services, and ConfigMaps.

The deployment of the EFK stack is divided into three parts:

- Deploying Elasticsearch and Kibana
- Deploying Fluentd
- Setting up access to Elasticsearch and Kibana

But first, we need to build and deploy our own microservices.

Building and deploying our microservices

Building, deploying, and verifying the deployment using the test-em-all.bash test script is done in the same way as it was done in *Chapter 18, Using a Service Mesh to Improve Observability and Management,* in the *Running commands to create the service mesh* section. These instructions assume that cert-manager and Istio are installed as instructed in *Chapters 17* and *18.*

Run the following commands to get started:

1. First, build the Docker images from the source with the following commands:

```
cd $BOOK_HOME/Chapter19
eval $(minikube docker-env -u)
./gradlew build
eval $(minikube docker-env)
docker compose build
```

The eval $(minikube docker-env -u) command ensures that the ./gradlew build command uses the host's Docker engine and not the Docker engine in the Minikube instance. The build command uses Docker to run test containers.

2. Recreate the namespace, hands-on, and set it as the default namespace:

```
kubectl delete namespace hands-on
kubectl apply -f kubernetes/hands-on-namespace.yml
kubectl config set-context $(kubectl config current-context) --namespace=hands-on
```

3. Resolve the Helm chart dependencies with the following commands.

   First, we update the dependencies in the components folder:

```
for f in kubernetes/helm/components/*; do helm dep up $f; donated
```

   Next, we update the dependencies in the environments folder:

```
for f in kubernetes/helm/environments/*; do helm dep up $f; donated
```

4. Deploy the system landscape using Helm and wait for all deployments to complete:

```
helm install hands-on-dev-env \
    kubernetes/helm/environments/dev-env \
    -n hands-on --wait
```

5. Start the Minikube tunnel in a separate terminal window, if it's not already running (see *Chapter 18,* the *Setting up access to Istio services* section, for a recap if required):

```
minikube tunnel
```

   Remember that this command requires that your user has sudo privileges and that you enter your password during startup. It takes a couple of seconds before the command asks for the password, so it is easy to miss!

6. Run the normal tests to verify the deployment with the following command:

```
./test-em-all.bash
```

Expect the output to be similar to what we saw in the previous chapters:

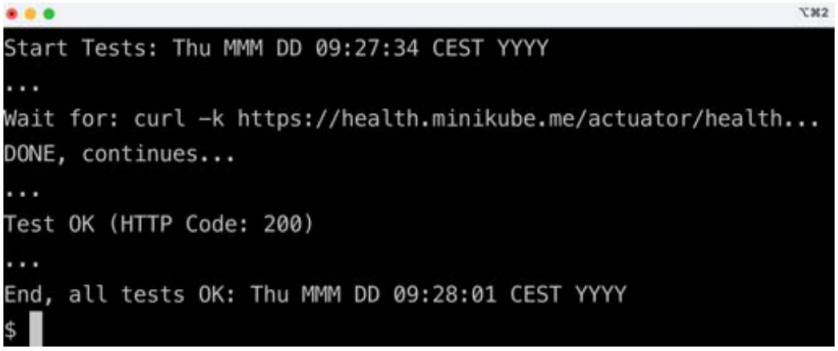


Figure 19.2: Tests running fine

7. You can also try out the APIs manually by running the following commands:

```
ACCESS_TOKEN=$(curl -k https://writer:secret-writer@minikube.me/
oauth2/token -d grant_type=client_credentials -d scope="product:read product:write" -s |
jq .access_token -r)

echo ACCESS_TOKEN=$ACCESS_TOKEN

curl -ks https://minikube.me/product-composite/1 -H "Authorization: Bearer $ACCESS_TOKEN"
| jq .productId
```

Expect the requested product ID, 1, in the response.

With the microservices deployed, we can move on and deploy Elasticsearch and Kibana!

Deploying Elasticsearch and Kibana

We will deploy Elasticsearch and Kibana to their own namespace, registry. Both Elasticsearch and Kibana will be deployed for development and test usage using a Kubernetes Deployment and Service object. The services will expose the standard ports for Elasticsearch and Kibana internally in the Kubernetes cluster; that is, port 9200 for Elasticsearch and port 5601 for Kibana.

To provide external HTTP access to Elasticsearch and Kibana, we will create Istio objects, as we did in *Chapter 18, Using a Service Mesh to Improve Observability and Management,* for Kiali and Jaeger – see the *Setting up access to Istio services* section for a recap, if required. This will result in Elasticsearch and Kibana being available at https://elasticsearch.minikube.me and https://kibana.minikube.me.

The manifest files have been packaged in a Helm chart in the kubernetes/helm/environments/logging folder.

For recommended deployment options for Elasticsearch and Kibana in a production environment on Kubernetes, see https://www.elastic.co/elastic-cloud-kubernetes.

We will use the latest versions that were available for version 7 when this chapter was written:

- Elasticsearch version 7.17.28
- Kibana version 7.17.28

Elasticsearch version 8 is not used, due to limited support in the Fluentd plugin for Elasticsearch; see https://github.com/uken/fluent-plugin-elasticsearch/issues/1005. The fluentd-kubernetes-daemonset Docker base image that we will use in the following section to install Fluentd uses this plugin.

Before we deploy, let's look at the most interesting parts of the manifest files in the Helm chart's template folder.

A walkthrough of the manifest files

The manifest file for Elasticsearch, elasticsearch.yml, contains a standard Kubernetes Deployment and Service object that we have seen multiple times before; for example, in *Chapter 15, Introduction to Kubernetes,* in the *Trying out a sample deployment* section. The most interesting part of the manifest file is the following:

```
apiVersion: apps/v1
kind: Deployment
...
        containers:
        - name: elasticsearch
          image: docker.elastic.co/elasticsearch/elasticsearch:7.17.28
          resources:
            limits:
              CPU: 500m
              memory: 2Gi
            requests:
              CPU: 500m
              memory: 2Gi
```

Let's explain some of this manifest:

- We use an official Docker image from Elastic that's available at docker.elastic.co. The version is set to 7.17.28.
- The Elasticsearch container is allowed to allocate a relatively large amount of memory – 2 GB – to be able to run queries with good performance. The more memory, the better the performance.

The manifest file for Kibana, kibana.yml, also contains a standard Kubernetes Deployment and Service object. The most interesting parts in the manifest file are as follows:

```
apiVersion: apps/v1
kind: Deployment
...
        containers:
        - name: kibana
          image: docker.elastic.co/kibana/kibana:7.17.28
          env:
          - name: ELASTICSEARCH_URL
            value: http://elasticsearch:9200
```

Let's explain some of the manifest:

- For Kibana, we also use an official Docker image from Elastic that's available at docker.elastic.co. The version is set to 7.17.28.
- To connect Kibana with the Elasticsearch Pod, an environment variable, ELASTICSEARCH_URL, is defined to specify the address to the Elasticsearch service, http://elasticsearch:9200.

Finally, the Istio manifests for setting up external access are found in the expose-elasticsearch.yml and expose-kibana.yml files. For a recap on how the Gateway, VirtualService, and DestinationRule objects are used, see the *Creating the service mesh* section in *Chapter 18.* They will provide the following forwarding of external requests:

- https://elasticsearch.minikube.me ÿ http://elasticsearch:9200
- https://kibana.minikube.me ÿ http://kibana:5601

With these insights, we are ready to perform the implementation of Elasticsearch and Kibana.

Running the deploy commands

Deploy Elasticsearch and Kibana by performing the following steps:

1. To make the deployment steps run faster, prefetch the Docker images for Elasticsearch and Kibana with the following commands:

```
eval $(minikube docker-env) docker
pull docker.elastic.co/elasticsearch/elasticsearch:7.17.28 docker pull docker.elastic.co/
kibana/kibana:7.17.28
```

2. Use the Helm chart to create a logging namespace, deploy Elasticsearch and Kibana in it, and wait for the Pods to be ready:

```
helm install logging-hands-on-add-on kubernetes/helm/environments/ logging \ -n logging
--create-
        namespace --wait
```

3. Verify that Elasticsearch is up and running with the following command:

```
curl https://elasticsearch.minikube.me -sk | jq -r .tagline
```

Expect You Know, for Search as a response.

Depending on your hardware, you might need to wait for a minute or two before Elasticsearch responds with this message.

4. Verify that Kibana is up and running with the following command:

```
curl https://kibana.minikube.me \
    -kLs -o /dev/null -w "%{http_code}\n"
```

Expect 200 as the response.

Again, you might need to wait for a minute or two before Kibana is initialized and responds with 200.

With Elasticsearch and Kibana deployed, we can start to deploy Fluentd.

Deploying Fluentd

Deploying Fluentd is a bit more complex compared to deploying Elasticsearch and Kibana. To deploy Fluentd, we will use a Docker image that's been published by the Fluentd project on Docker Hub, fluent/fluentd-kubernetes-daemonset, and the sample Kubernetes manifest files from a Fluentd project on GitHub, fluentd-kubernetes-daemonset. It is located at https://github.com/fluent/fluentd-kubernetes-daemonset. As is implied by the name of the project, Fluentd will be deployed as a DaemonSet, running one Pod per Node in the Kubernetes cluster. Each Fluentd Pod is responsible for collecting log output from processes and containers that run on the same Node as the Pod. Since we are using Minikube with a single Node cluster, we will only have one Fluentd Pod.

To handle multiline log records that contain stack traces from exceptions, we will use a third-party Fluentd plugin provided by Google, fluent-plugin-detect-exceptions, which is available at https://github.com/GoogleCloudPlatform/fluent-plugin-detect-exceptions. To be able to use this plugin, we will build our own Docker image where the fluent-plugin-detect-exceptions plugin will be installed.

Fluentd's Docker image, fluentd-kubernetes-daemonset, will be used as the base image.

We will use the following versions:

- Fluentd version 1.4.2
- fluent-plugin-detect-exceptions version 0.0.12

Before we deploy, let's look at the most interesting parts of the manifest files.

A walkthrough of the manifest files

The Dockerfile that's used to build the Docker image, kubernetes/efk/Dockerfile, looks as follows:

```
FROM fluent/fluentd-kubernetes-daemonset:v1.4.2-debian-elasticsearch-1.1
RUN gem install fluent-plugin-detect-exceptions -v 0.0.12 \
  && gem sources --clear-all \
  && rm -rf /var/lib/apt/lists/* \
                  /home/fluent/.gem/ruby/2.3.0/cache/*.gem
```

Let's explain this in detail:

- The base image is Fluentd's Docker image, fluentd-kubernetes-daemonset. The v1.4.2-debian-elasticsearch-1.1 tag specifies that version 1.4.2 will be used with a package that contains built-in support for sending log records to Elasticsearch. The base Docker image contains the Fluentd configuration files that were mentioned in the *Configuring Fluentd* section.
- The Google plugin, fluent-plugin-detect-exceptions, is installed using Ruby's package manager, gem.

The manifest file of the DaemonSet, kubernetes/efk/fluentd-ds.yml, is based on a sample manifest file in the fluentd-kubernetes-daemonset project, which can be found at https://github.com/fluent/fluentd-kubernetes-daemonset/blob/master/fluentd-daemonset-elasticsearch.yaml.

This file is a bit complex, so let's go through the most interesting parts separately:

1. First, here's the declaration of the DaemonSet:

```
apiVersion: apps/v1
kind: DaemonSet
metadata:
  name: fluentd
  namespace: kube-system
```

   The kind key specifies that this is a DaemonSet. The namespace key specifies that the DaemonSet will be created in the kube-system namespace and not in the logging namespace where Elasticsearch and Kibana are deployed.

2. The next part specifies the template for the Pods that are created by the DaemonSet. The most interesting parts are as follows:

```
spec:
  template:
    spec:
      containers:
      - name: fluentd
        image: hands-on/fluentd:v1
        env:
          - name: FLUENT_ELASTICSSEARCH_HOST
            value: "elasticsearch.logging"
```

```
        - name: FLUENT_ELASTICSSEARCH_PORT
          value: "9200"
```

The Docker image that's used for the Pods is hands-on/fluentd:v1. We will build this Docker image after walking through the manifest files using the Dockerfile we described previously.

A number of environment variables are supported by the Docker image and are used to customize it. The two most important ones are as follows:

- FLUENT_ELASTICSEARCH_HOST, which specifies the hostname of the Elasticsearch service, elasticsearch.logging
- FLUENT_ELASTICSSEARCH_PORT, which specifies the port that's used to communicate with Elasticsearch, 9200

Since the Fluentd Pod runs in a different namespace from Elasticsearch, the hostname cannot be specified using its short name; that is, elasticsearch. Instead, the namespace part of the DNS name must also be specified; that is, elasticsearch.logging. As an alternative, the **fully qualified domain name (FQDN),** elasticsearch.logging.svc.cluster.local, can also be used. But since the last part of the DNS name, svc.cluster.local, is shared by all DNS names inside a Kubernetes cluster, it does not need to be specified.

3. Finally, a number of volumes (that is, filesystems) are mapped to the Pod, as follows:

```
      volumeMounts:
      - name: varlog
        mountPath: /var/log
      - name: varlibdockercontainers
        mountPath: /var/lib/docker/containers
        readOnly: true
      - name: journal
        mountPath: /var/log/journal
        readOnly: true
      - name: fluentd-extra-config
        mountPath: /fluentd/etc/conf.d
    volumes:
    - name: varlog
      hostPath:
```

```
            path: /var/log
        - name: varlibdockercontainers
          hostPath:
            path: /var/lib/docker/containers
        - name: journal
          hostPath:
            path: /run/log/journal
        - name: fluentd-extra-config
          configMap:
            name: "fluentd-hands-on-config"
```

Let's take a look at the source code in detail:

- Three folders on the host (that is, the Node) are mapped to the Fluentd Pod. These folders contain the log files that Fluentd will tail and collect log records from. The folders are /var/log, /var/lib/docker/containers, and /run/log/journal.
- Our own configuration file, which specifies how Fluentd will process log records from our microservices, is mapped using a ConfigMap called fluentd-hands-on- config to the /fluentd/etc/conf.d folder. The base Docker image configures Fluentd to include any configuration file that's found in the /fluentd/etc/conf.d folder. See the *Configuring Fluentd* section for details.

For the full source code of the manifest file for the DaemonSet, see the kubernetes/efk/fluentd- ds.yml file.

Now that we've walked through everything, we are ready to perform the deployment of Fluentd.

Running the deploy commands

To deploy Fluentd, we have to build the Docker image, create a ConfigMap, and finally deploy the DaemonSet. Run the following commands to perform these steps:

1. Build the Docker image and tag it with hands-on/fluentd:v1 using the following command:

```
eval $(minikube docker-env)
docker build -f kubernetes/efk/Dockerfile -t hands-on/fluentd:v1
kubernetes/efk/
```

2. Create a ConfigMap, deploy Fluentd's DaemonSet, and wait for the Pod to be ready with the following commands:

```
kubectl apply -f kubernetes/efk/fluentd-hands-on-configmap.yml
kubectl apply -f kubernetes/efk/fluentd-ds.yml
kubectl wait --timeout=120s --for=condition=Ready pod -l app=fluentd -n kube-system
```

3. Verify that the Fluentd Pod is healthy with the following command:

```
kubectl logs -n kube-system -l app=fluentd --tail=-1 | grep "fluentd worker is now running worker"
```

Expect a response of 2023-05-22 14:59:46 +0000 [info]: #0 fluentd worker is now running worker=0.

As with Elasticsearch and Kibana, you might need to wait for a minute or two before Fluentd responds with this message.

4. Fluentd will start to collect a considerable number of log records from the various containers in the Minikube instance. After a minute or so, you can ask Elasticsearch how many log records have been collected with the following command:

```
curl https://elasticsearch.minikube.me/_all/_count -sk | jq .count
```

5. The command can be a bit slow the first time it is executed but should return a total count of several thousand of log records. In my case, it returned 55607.

This completes the deployment of the EFK stack. Now, it's time to try it out and find out what all the collected log records are about!

Trying out the EFK stack

The first thing we need to do before we can try out the EFK stack is to initialize Kibana so that it knows which indices to use in Elasticsearch.

An **index** in Elasticsearch corresponds to a **database** in SQL concepts. The SQL concepts **table, row,** and **column** correspond to **type, document,** and **property** in Elasticsearch.

Once that is done, we will try out the following common tasks:

1. We will start by analyzing what types of log records Fluentd has collected and stored in Elasticsearch. Kibana has a very useful visualization capability that can be used for this.
2. Next, we will learn how to find all related log records created by the microservices while processing an external request. We will use the **trace ID** in the log records as a correlation ID to find related log records.
3. Finally, we will learn how to use Kibana to perform **root cause analysis,** finding the current reason for an error.

Initializing Kibana

Before we start to use Kibana, we must specify which search indices to use in Elasticsearch and which field in the indices holds the timestamps for the log records.

Just a quick reminder that we are using a certificate created by our own **certificate authority (CA),** meaning that it is not trusted by web browsers! For a recap on how to make web browsers accept our certificate, see the *Observing the service mesh* section of *Chapter 18.*

Perform the following steps to initialize Kibana:

1. Open Kibana's web UI using the https://kibana.minikube.me URL in a web browser.
2. Text box down right, **Your data is not secure** ÿ **Dismiss** button.
3. On the **Welcome to Elastic** page, select **Explore on my own.**
4. On the **Welcome home** page, click on the ÿ hamburger menu (three horizontal lines) in the upper-left corner, and click on **Stack Management** at the bottom of the menu to the left.
5. In the **Management** menu, go to the bottom and select **Index Patterns** under the **Kibana** header.
6. Click on the button named **Create index pattern.**
7. Enter logstash-* as the index pattern name and click on the **Next Step** button.

Indices are, by default, named logstash for historical reasons, even though Fluentd is used for log collection.

8. Click on the drop-down list for the **Timestamp** field and select the only available field, @ timestamp.

9. Click on the **Create index pattern** button.
10. Kibana will show a page that summarizes the fields that are available in the selected indices.

With Kibana initialized, we are ready to examine the collected log records.

Analyzing the log records

From the deployment of Fluentd, we know that it immediately started to collect a significant number of log records. So, the first thing we need to do is get an understanding of what types of log records Fluentd has collected and stored in Elasticsearch.

We will use Kibana's visualization feature to divide the log records by the Kubernetes namespace and then ask Kibana to show us how the log records are divided by the type of container within each namespace. A pie chart is a suitable chart type for this type of analysis. Perform the following steps to create a pie chart:

1. In Kibana's web UI, click on the hamburger menu again and select **Visualize Library** under **Analytics** in the menu.
2. Click on the **Create new visualization** button and select the **Lens** type on the next page. A web page like the following will be displayed:

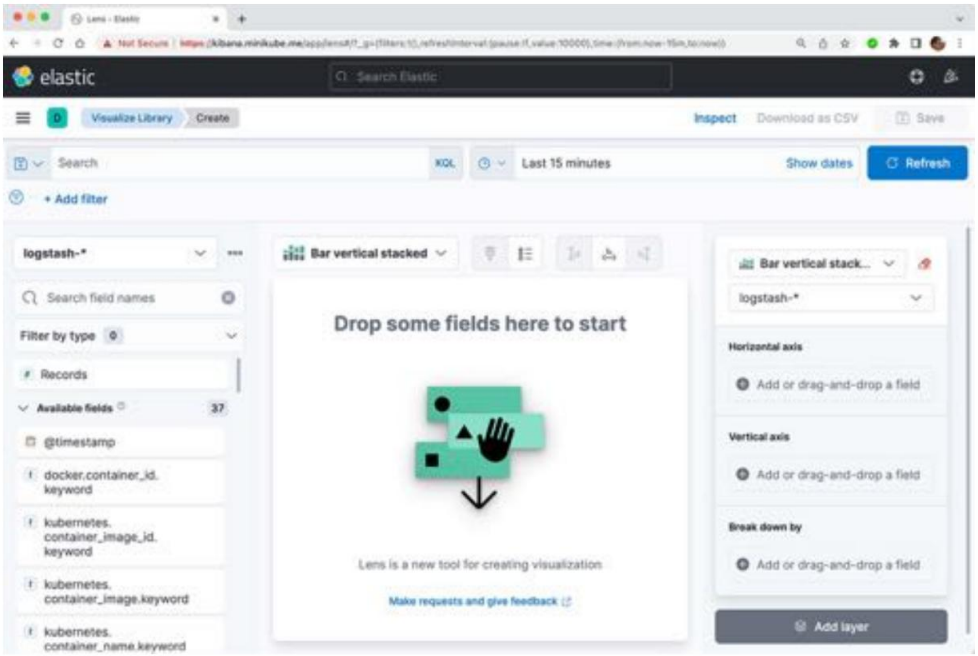


Figure 19.3: Starting to analyze log records in Kibana

3. Verify that **logstash-*** is the selected index pattern in the top-left drop-down menu.
4. In the **Bar vertical stacked** drop-down menu next to the index pattern, select **Pie** as the visualization type.
5. In the time picker (a date interval selector) above the pie chart, set a date interval large enough to cover log records of interest (set to the **Last 15 minutes** in the following screen- shot). Click on its calendar icon to adjust the time interval.
6. In the field named **Search field names** below the index pattern, enter kubernetes. namespace_name.keyword.
7. Under the **Available fields** list, the kubernetes.namespace_name.keyword field is now present. Drag this field into the big box in the middle of the page, named **Drop some fields here to start.** Kibana will immediately start to analyze log records and render a pie chart divided into Kubernetes namespaces. In my case, it looks like this:

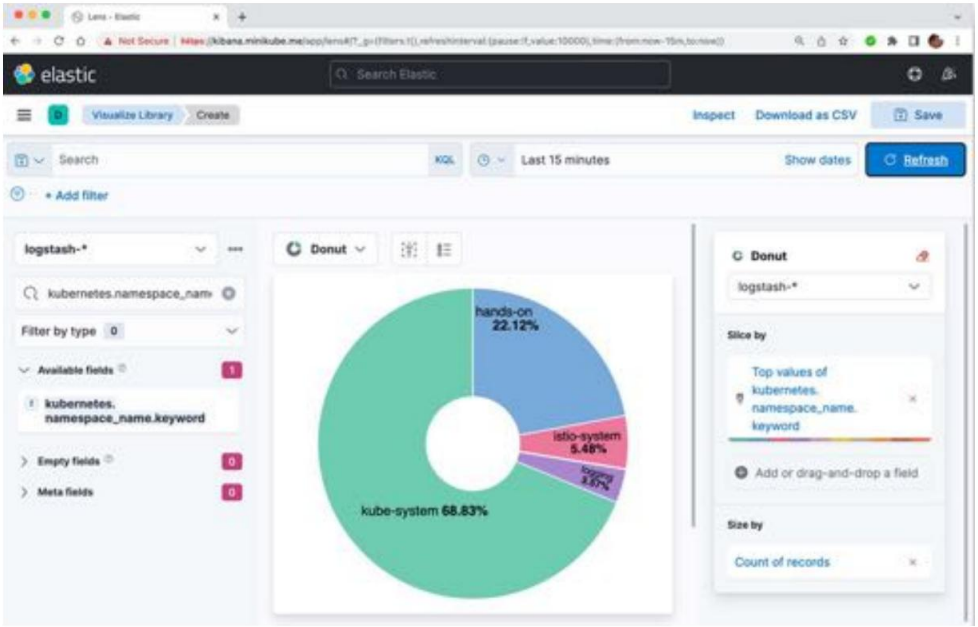


Figure 19.4: Kibana analysis of log records per Kubernetes namespace

We can see that the log records are divided into the namespaces we have been working with in the previous chapters: kube-system, istio-system, logging, and our own hands-on namespace. To see which containers have created the log records per namespace, we need to add a second field.

8. In the **Search field names** field, enter kubernetes.container_name.keyword.

9. In the **Available fields** list, the kubernetes.container_name.keyword field is now present. Drag this field into the big box in the middle of the page showing the foot chart. Kibana will immediately start to analyze log records and render a pie chart divided by Kubernetes namespace and container name.
10. In the result of *step* 9, we can see a lot of log records coming from coredns – 67%, in my case. Since we are not particularly interested in these log records, we can remove them by adding a filter with the following steps:

    to. Click on **+ Add filter** (in the top-left corner).

    b. Select the kubernetes.container_name.keyword field and the is not operator. Finally, enter the coredns value and click on the **Save** button.

11. In my case, the rendered pie chart now looks like this:

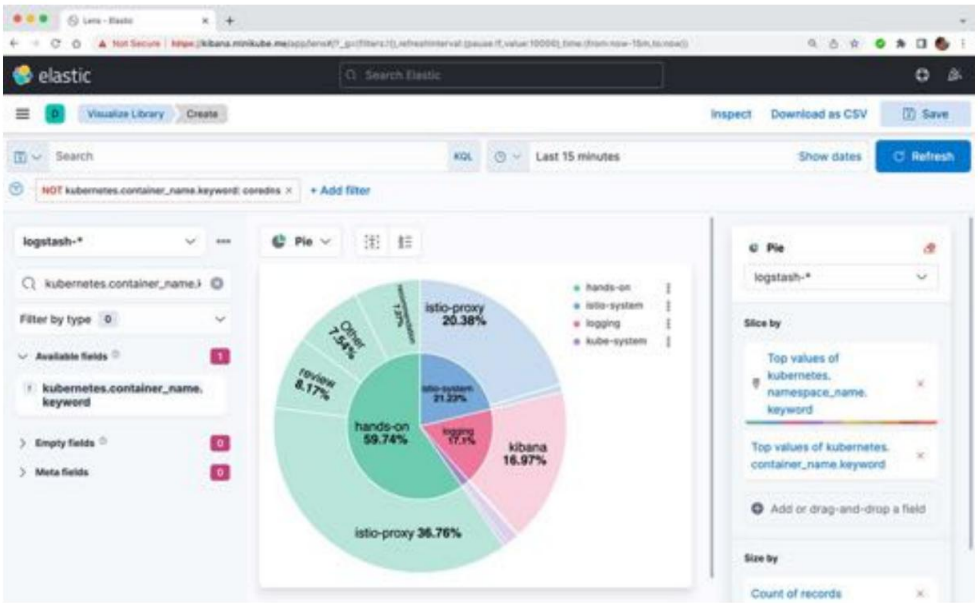


Figure 19.5: Kibana analysis of log records per namespace and container

Here, we can find the log records from our microservices. Most of the log records come from the review and recommendation microservices. The product and product-composite microservices can be found under the **Other** section of the pie chart.

12. Wrap up this introduction to how to analyze what types of log records we have collected by saving this pie chart in a dashboard. Click on the **Save** button in the top-right corner.

13. On the page named **Save Lens visualization,** do the following:

    to. Give it a title; for example, hands-on-visualization.

    b. Enter a description; for example, This is my first visualization in Kibana.

    c. In the **Add to dashboard** box, select **New.** The page should look like this:

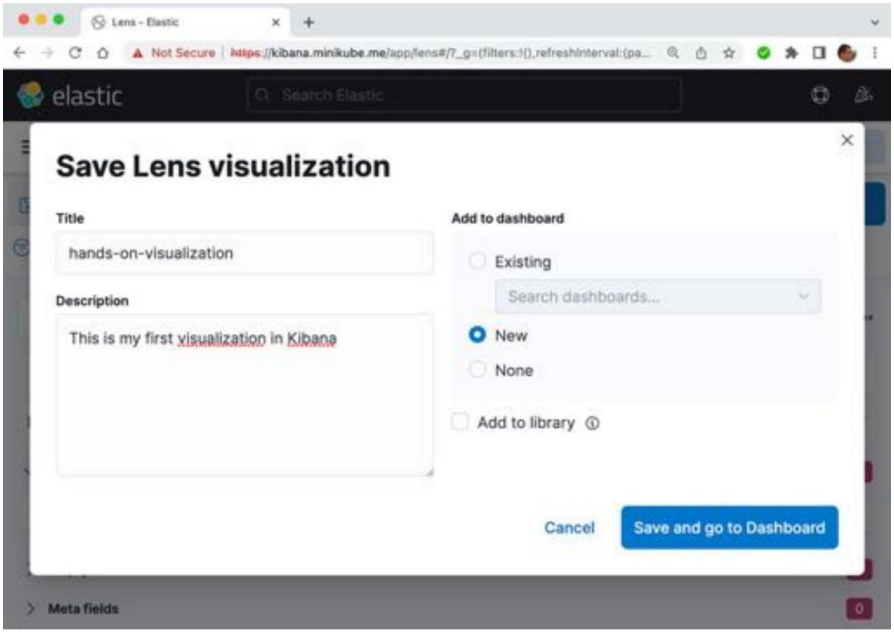


Figure 19.6: Creating a dashboard in Kibana

    d. Click on the button named **Save and go to Dashboard.** A dashboard like the following should be presented:

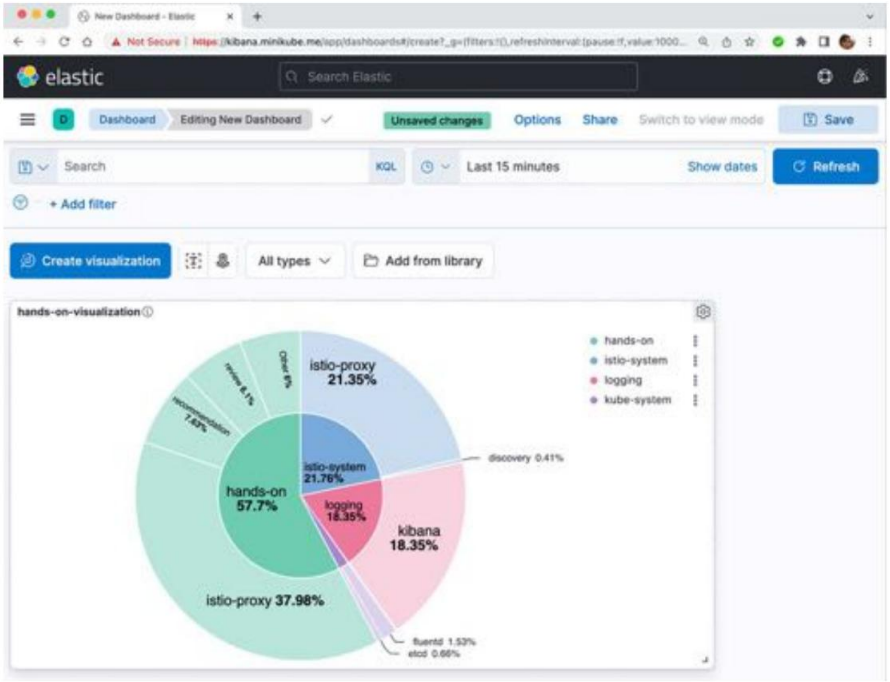


Figure 19.7: The new dashboard in Kibana

14. Click on the **Save** button in the top-right corner, give the dashboard a name (for example, hands-on-dashboard), and click on the **Save** button.

    You can now always go back to this dashboard by selecting **Dashboard** from the hamburger menu.

Kibana contains tons of features for analyzing log records – feel free to try them out on your own. For inspiration, see https://www.elastic.co/guide/en/kibana/7.17/dashboard.html. We will now move on and start to locate the current log records from our microservice.

Discovering the log records from microservices

In this section, we will learn how to utilize one of the main features of centralized logging, finding log records from our microservices. We will also learn how to use the trace ID in the log records to find log records from other microservices that belong to the same process; for example, processing an external request sent to the public API.

Let's start by creating some log records that we can look up with the help of Kibana. We will use the API to create a product with a unique product ID and then retrieve information about the product. After that, we can try to find the log records that were created when retrieving the product information.

The creation of log records in the microservices has been updated a bit from the previous chapter so that product-composite and the three core microservices, product, recommendation, and review, all write a log record with the log level set to INFO when they begin processing a GET request. Let's go over the source code that's been added to each microservice:

- Product composite microservice log creation:

```
LOG.info("Will get composite product info for product.id={}", productId);
```

- Product microservice log creation:

```
LOG.info("Will get product info for id={}", productId);
```

- Recommendation microservice log creation:

```
LOG.info("Will get recommendations for product with id={}", productId);
```

- Review microservice log creation:

```
LOG.info("Will get reviews for product with id={}", productId);
```

For more details, see the source code in the microservices folder.

Perform the following steps to use the API to create log records and, after that, use Kibana to look up the log records:

1. Get an access token with the following command:

```
ACCESS_TOKEN=$(curl -k https://writer:secret-writer@minikube.me/
oauth2/token -d grant_type=client_credentials -d scope="product:read product:write" -s |
jq .access_token -r)

echo ACCESS_TOKEN=$ACCESS_TOKEN
```

2. As mentioned in the introduction to this section, we will start by creating a product with a unique product ID. Create a minimalistic product (without recommendations and reviews) for "productId" :1234 by executing the following command:

```
curl -X POST -k https://minikube.me/product-composite \
    -H "Content-Type: application/json" \
    -H "Authorization: Bearer $ACCESS_TOKEN" \
    --data '{"productId":1234,"name":"product name
1234","weight":1234}'
```

Read the product with the following command:

```
curl -H "Authorization: Bearer $ACCESS_TOKEN" -k 'https://minikube.
me/product-composite/1234' -s | jq.
```

Expect a response similar to the following:

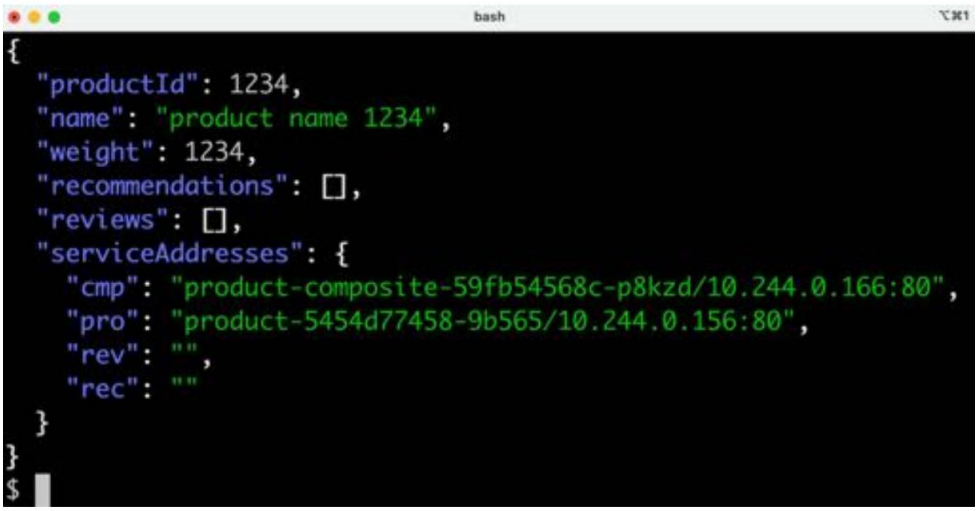


Figure 19.8: Looking up the product with productId = 1234

Hopefully, some log records were created by these API calls. Let's jump over to Kibana and find out!

3. On the Kibana web page, click **Discover** from the hamburger menu. You will see something like the following:

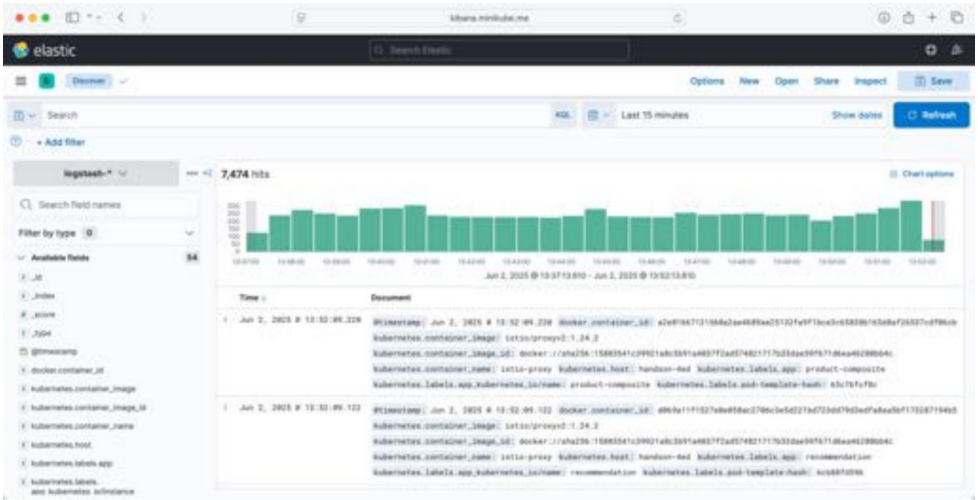


Figure 19.9: Kibana web UI with its major parts

In the top-left corner, we can see that Kibana has found **7,474** log records. The time picker shows that they are from the last 15 minutes. In the histogram, we can see how the log records are spread out over time. Below the histogram is a table showing the most recent log events that were found by the query.

4. If you want to change the time interval, you can use the time picker. Click on its calendar icon to adjust the time interval.
5. To get a better view of the content in the log records, add some fields from the log records as columns in the table under the histogram.

6. To be able to see all available fields, click on the down arrow to the right of the **Filter by type** label, and unselect **Hide empty fields.**
7. Select fields from the **Available fields** list to the left. Scroll down until the field is found.

   To find fields more easily, use the field named **Search field names** to filter the list of available fields.

   Hold the cursor over the field, and a **+** button will appear (a white cross in a blue circle); click on it to add the field as a column in the table. Select the following fields, in order:

   to. spring.level, the log level

   b. kubernetes.namespace_name, the Kubernetes namespace

   c. kubernetes.container_name, the name of the container

   d. spring.trace, the trace ID used for distributed tracing

   and. log, the current log message

   To save some space, you can hide the list of fields by clicking on the collapse icon next to the index pattern field (containing the text logstash-*).

   The web page should look something like the following:

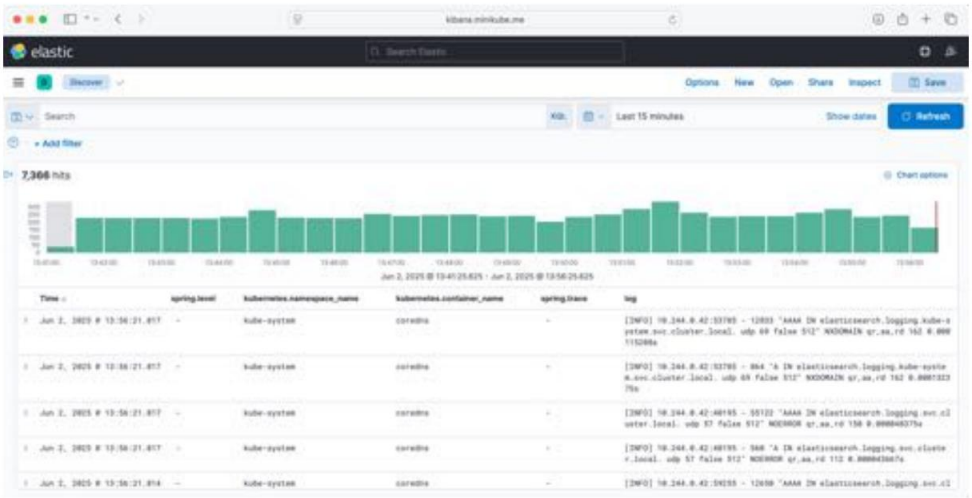


Figure 19.10: Kibana web UI showing log records

The table now contains information that is of interest regarding the log records!

8. To find log records from the call to the GET API, we can ask Kibana to find log records where the log field contains the text info for product.id=1234. This matches the log output from the product-composite microservice that was shown previously.

   This can be done by entering log:"info for product.id=1234" in the top-left **Search** field and clicking on the **Update** button (this button can also be named **Refresh).** Expect one log record to be found:

Figure 19.11: Kibana web UI showing a log record for productId = 1234

9. Verify that the timestamp is from when you call the GET API, and verify that the name of the container that created the log record is product-composite; that is, verify that the log record was sent by the product composite microservice.
10. Now, we want to see the related log records from the other microservices that participated in the process of returning information about the product with product ID 1234. In other words, we want to find log records with the same trace ID as that of the log record we found. To do this, place the cursor over the spring.trace field for the log record. Two small magnifying glasses will be shown to the right of the field, one with a **+** sign and one with a **-** sign. Click on the magnifying glass with the **+** sign to filter on the trace ID.
11. Clear the **Search** field so that the only search criterion is the filter of the trace field. Then, Click on the **Update** button to see the result. Expect a response like the following:

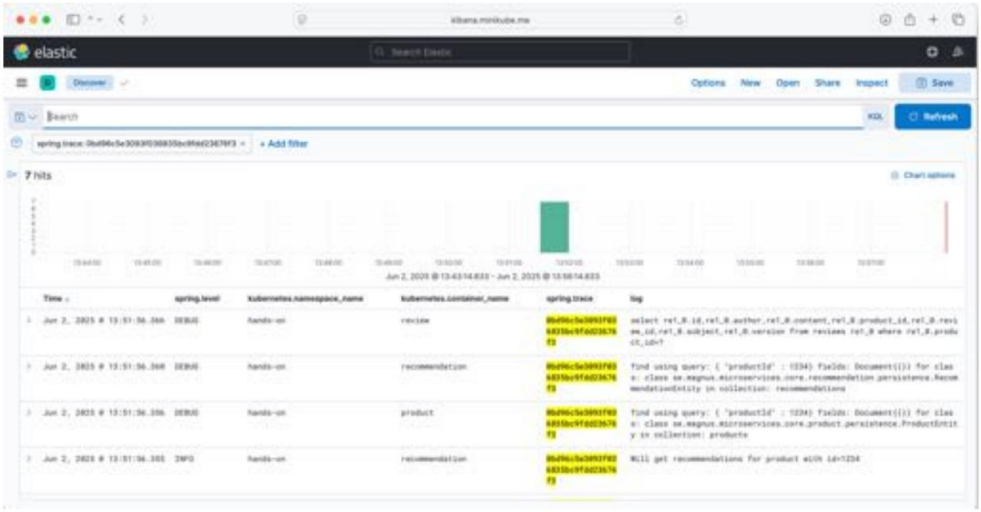


Figure 19.12: Kibana web UI showing log records for a trace ID

We can see some detailed debug messages that clutter the view; let's get rid of them!

12. Place the cursor over a DEBUG value and click on the magnifying glass with the – sign to filter out log records with the log level set to DEBUG.
13. We should now be able to see the four expected log records, one for each microservice involved in the lookup of product information for the product with product ID 1234:

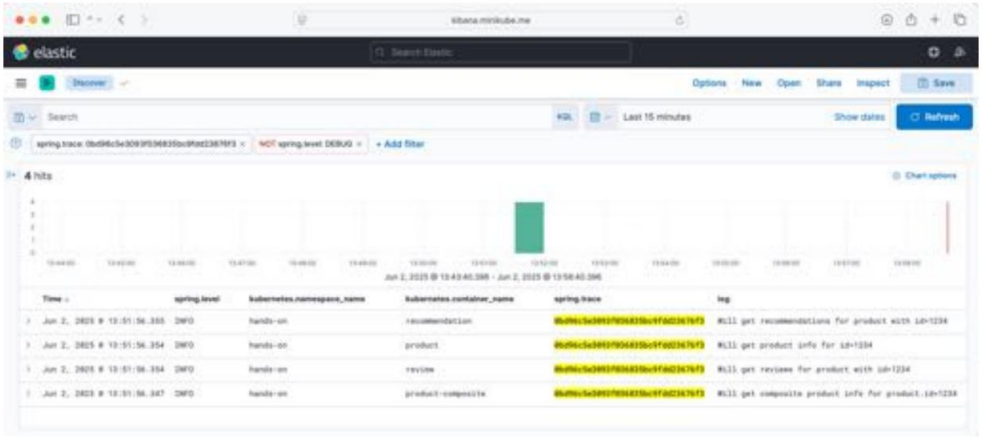


Figure 19.13: Kibana web UI showing log records for a trace ID with log level = INFO

Also, note that the filters that were applied included the trace ID but excluded log records with the log level set to DEBUG.

Now that we know how to find the expected log records, we are ready to take the next step. This will be to learn how to find unexpected log records (that is, error messages), and how to perform root cause analysis to find the reason for these error messages.

Performing root cause analysis

One of the most important features of centralized logging is that it makes it possible to analyze errors using log records from many sources and, based on that, perform root cause analysis, finding the actual reason for the error message.

In this section, we will simulate an error and see how we can find information about it all the way down to the line of source code that caused the error in one of the microservices in the sys-tem landscape. To simulate an error, we will reuse the fault parameter we introduced in *Chapter 13, Improving Resilience Using Resilience4j,* in the *Adding programmable delays and random errors* section. We can use this to force the product microservice to throw an exception. Perform the following steps:

1. Run the following command to generate a fault in the product microservice while searching-ing for product information on the product with product ID 1234:

```
curl -H "Authorization: Bearer $ACCESS_TOKEN" -k https://minikube.
me/product-composite/1234?faultPercent=100 -s | jq.
```

Expect the following error in response:

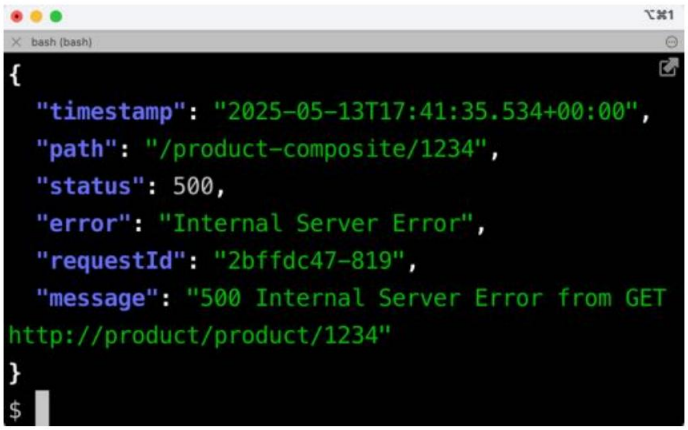


Figure 19.14: A request that caused an error in the processing

Now, we must pretend that we have no clue about the reason for this error! Otherwise, the root cause analysis wouldn't be very exciting, right?

Let's assume that we work in a support organization and have been asked to investigate a problem that just occurred when an end user tried to look up information regarding a product with product ID 1234, but got an error message saying 500 Internal Server Error in response.

2. Before we start to analyze the problem, let's delete the previous search filters in the Kibana web UI so that we can start from scratch. For each filter we defined in the previous section, click on its close icon (an **x** sign) to remove it.
3. Start by using the time picker to select a time interval that includes the point in time when the problem occurred. In my case, 15 minutes is sufficient.
4. Select log records belonging to our namespace, hands-on. This can be done with the following steps:

    to. Expand the list of fields to the left by clicking on the hamburger icon **(ÿ)** in the top-left corner.

    b. Click on the kubernetes.namespace_name field in the **Selected fields** list. A list of the top five namespaces is shown.

    c. Click on the **+** sign after the hands-on namespace.

5. Next, search for log records with the log level set to WARN within this time frame where the log message mentions product ID 1234. This can be done by clicking on the spring.level field in the list of selected fields. When you click on this field, its most used values will be displayed under it. Filter on the WARN value by clicking on its **+** sign. Kibana will now show log records within the selected time frame with their log level set to WARN from the hands-on namespace, like this:

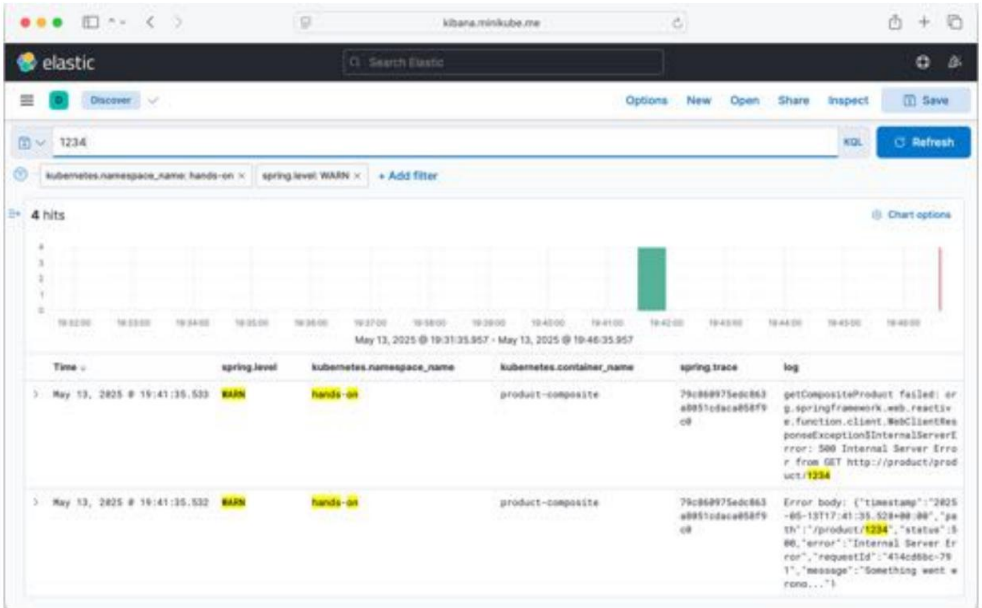


Figure 19.15: Kiali web UI showing log records that report errors

We can see a number of error messages related to product ID 1234. The top log entries have the same trace ID, so this seems like a trace ID of interest to use for further investigation. The first log entry also contains the text reported by the end user, 500 and Internal Server Error, and a Something went wrong… error message, which probably has to do with the root cause of the error.

6. Filter on the trace ID of the first log record as we did in the previous section.
7. Remove the filter from the WARN log level to be able to see all the records belonging to this trace ID. Expect Kibana to respond with a lot of log records looking something like this:

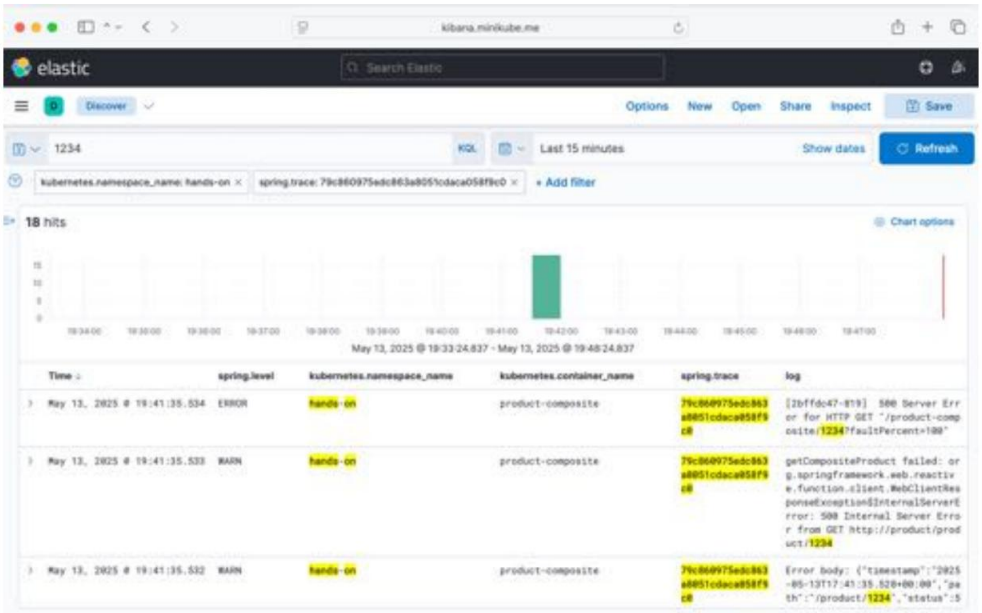


Figure 19.16: Kiali web UI – looking for the root cause

8. Unfortunately, we cannot find any stack trace identifying the root cause by using trace IDs. This is due to a limitation in the Fluentd plugin we use for collecting multiline exceptions, fluent-plugin-detect-exceptions. It cannot relate stack traces to the trace ID that was used. Instead, we can use a feature in Kibana to find surrounding log records that have occurred close in time to a specific log record.

9. Expand the log record that says Error body: {… status":500,"error":"Internal Server Error","message":"Something went wrong..."…} using the arrow to the left of the log record. Detailed information about this specific log record will be revealed:

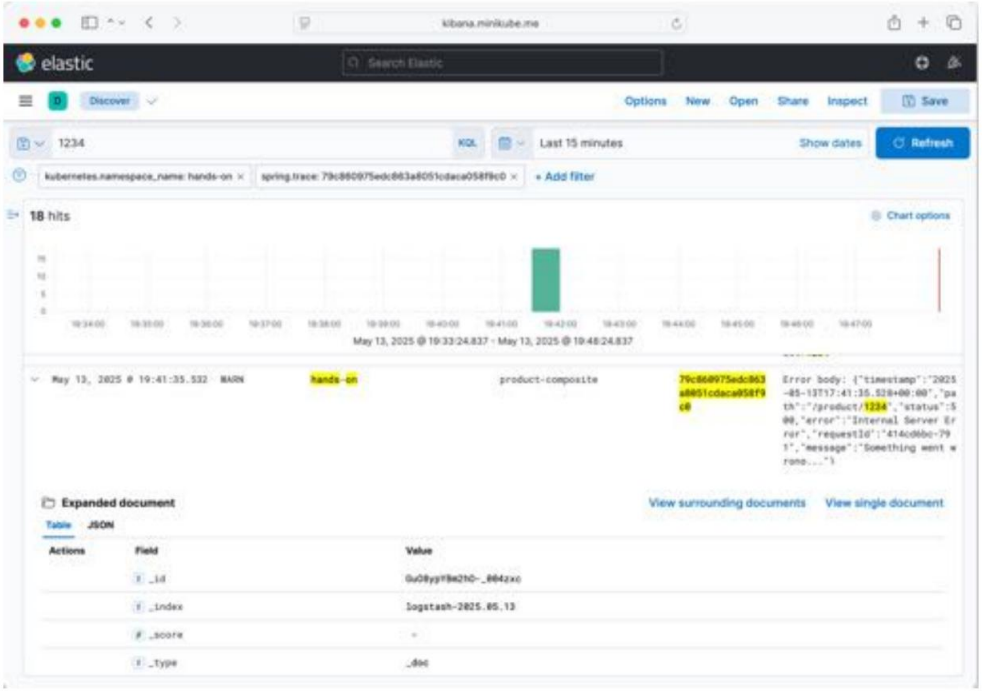


Figure 19.17: Kiali web UI – expanding the log record with the root cause log message

10. There is also a link named **View surrounding documents;** click on it to see nearby log records. Scroll down to the bottom of the page to find a **Load** field where the number of records can be specified. Increase the default value from 5 to 10. Expect a web page like the following:

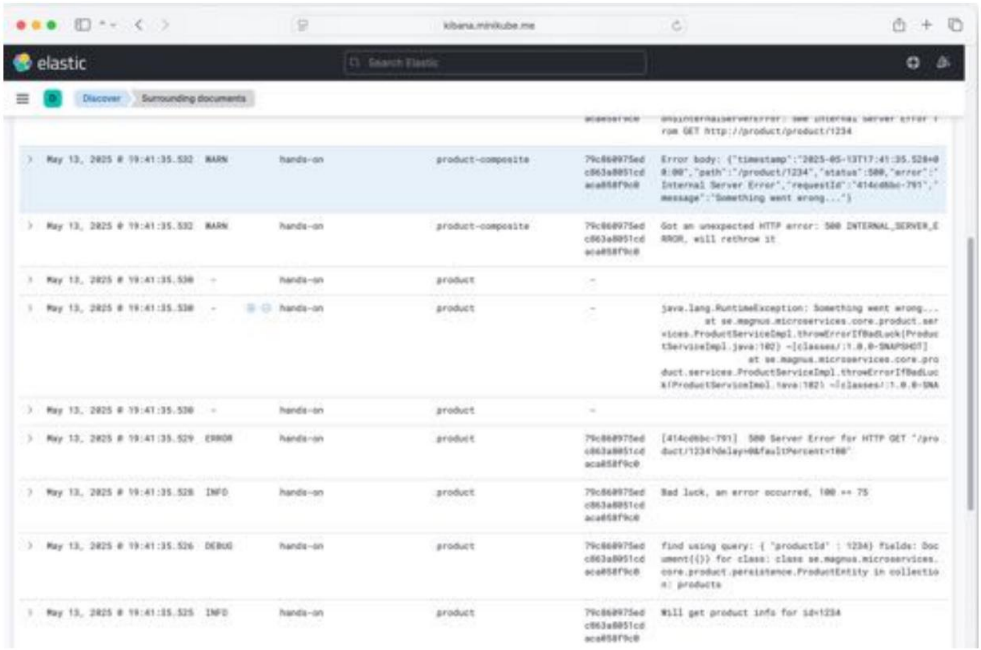


Figure 19.18: Kiali web UI – the root cause found

11. The third log record below the expanded log record contains the stack trace for the Something went wrong... error message. This error message looks interesting. It was logged by the product microservice just 5 milliseconds before the expanded log record. They seem to be related! The stack trace in that log record points to *line 102* in ProductServiceImpl.

    Java. Looking at the source code (see microservices/product-service/src/main/java/se/magnus/microservices/core/product/services/ProductServiceImpl.java), *line 102* looks as follows:

```
throw new RuntimeException("Something went wrong...");
```

This is the root cause of the error. We did know this in advance, but now we have seen how we can navigate to it as well.

In this case, the problem is quite simple to solve; simply omit the faultPercent parameter in the request to the API. In other cases, resolving the root cause can be much difficult to figure out!

12. This concludes the root cause analysis. Click on the back button in the web browser to go back to the main page.
13. To be able to reuse the configuration of the search criteria and table layout, its definition can be saved by Kibana. Select, for example, to filter on log records from the hands-on namespace and click on the **Save** link in the top-right menu. Give the search definition a name and click on the **Save** button. The search definition can be restored when required using the **Open** link in the menu.

This concludes this chapter on using the EFK stack for centralized logging.

Summary

In this chapter, we learned about the importance of collecting log records from microservices in a system landscape into a common centralized database where analysis and searches of the stored log records can be performed. We use the EFK stack, Elasticsearch, Fluentd, and Kibana, to collect, process, store, analyze, and search for log records.

Fluentd was used to collect log records not only from our microservices but also from the various supporting containers in the Kubernetes cluster. Elasticsearch was used as a text search engine. Together with Kibana, we saw how easy it is to get an understanding of what types of log records we have collected.

We also learned how to use Kibana to perform important tasks such as finding related log records from cooperating microservices and how to perform root cause analysis, finding the real reason for an error message.

Being able to collect and analyze log records in this way is an important capability in a production environment, but these types of activities are always done afterwards, once the log record has been collected. Another important capability is to be able to monitor the current health of the microservices, collecting and visualizing runtime metrics in terms of the use of hardware resources, response times, and so on. We touched on this subject in the previous chapter, and in the next chapter, we will learn more about monitoring microservices.

Questions

1. A user searched for ERROR log messages in the hands-on namespace for the last 30 days using the search criteria shown in the following screenshot, but none were found. Why?

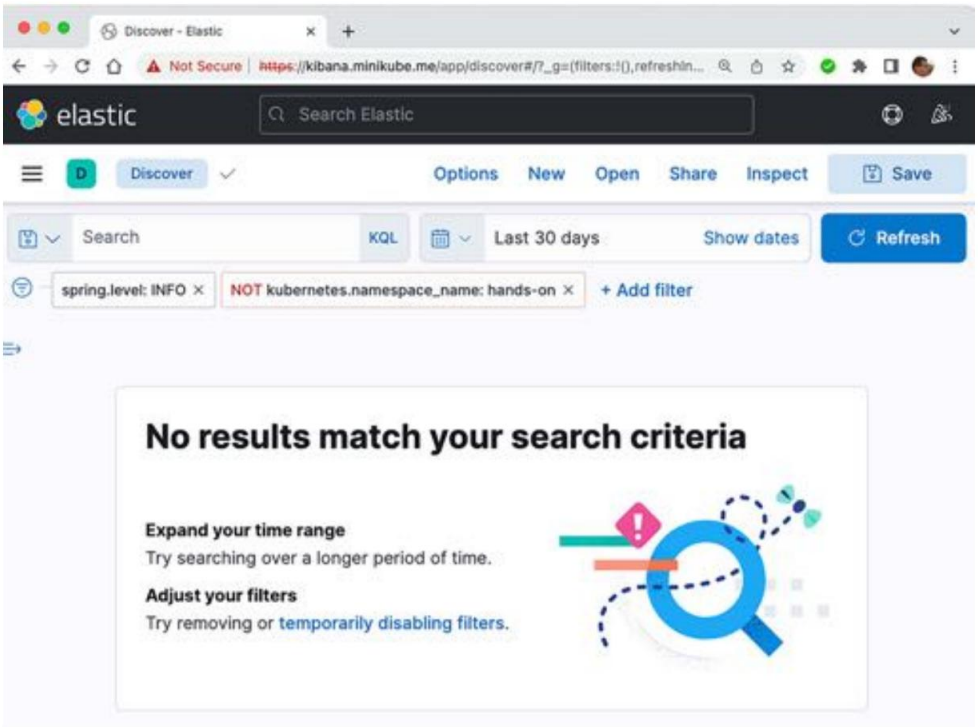


Figure 19.19: Kiali web UI not showing expected log records

2. A user has found a log record of interest (shown in the following screenshot). How can the user find related log records from this and other microservices, for example, that come from processing an external API request?

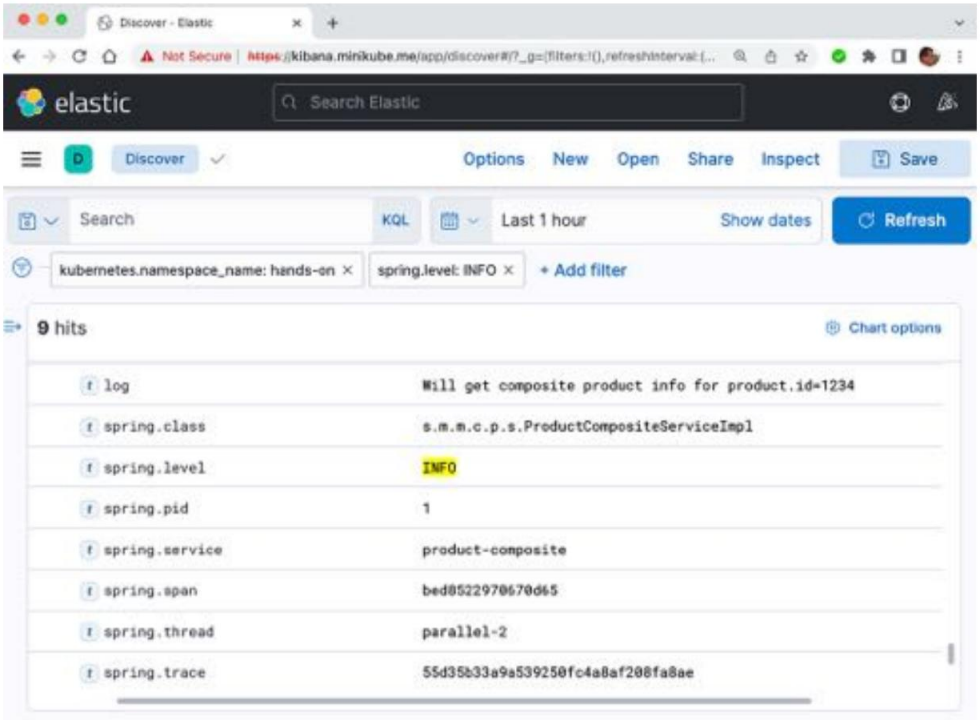


Figure 19.20: Kiali web UI – how do we find related log records?

3. A user has found a log record that seems to indicate the root cause of a problem that was reported by an end user. How can the user find the stack trace that shows where in the source code the error occurred?

Figure 19.21: Kiali web UI – how do we find the root cause?

4. Why doesn't the following Fluentd configuration element work?

```
<match kubernetes.**hands-on**>
  @type rewrite_tag_filter
  <rule>
    key log
    pattern ^(.*)$
    spring-boot tag.${tag}
  </rule>
</match>
```

5. How can you determine whether Elasticsearch is up and running?
6. You suddenly lose connection to Kibana from your web browser. What could have caused this problem?

20

Microservices Monitoring

In this chapter, we will learn how to use Prometheus and Grafana to collect, monitor, and alert us about performance metrics. As we mentioned in *Chapter* 1, *Introduction to Microservices,* in a production environment, it is crucial to be able to collect metrics for application performance and hardware resource usage. Monitoring these metrics is required to avoid long response times or outages for API requests and other processes.

To be able to monitor a system landscape of microservices cost-efficiently and proactively, we must also be able to define alarms that are triggered automatically if the metrics exceed the configured limits.

In this chapter, we will cover the following topics:

- Introduction to application monitoring using Prometheus and Grafana
- Changes in source code to collect application metrics
- Building and deploying the microservices
- Monitoring microservices using Grafana dashboards
- Setting up alarms in Grafana

Technical requirements

For instructions on how to install the tools used in this book and how to access this book's source code, please refer to the following chapters in this book:

- *Chapter 21, Installation Instructions for macOS*
- *Chapter 22, Installation Instructions for Microsoft Windows with WSL 2 and Ubuntu*

The code examples in this chapter all come from the source code in $BOOK_HOME/Chapter19.

If you want to view the changes that will be applied to the source code in this chapter so that you can use Prometheus and Grafana to monitor and alert on performance metrics, you can compare it with the source code for *Chapter 19, Centralized Logging with the EFK Stack.* You can use your favorite diff tool and compare the two folders – that is, $BOOK_HOME/Chapter19 and $BOOK_HOME/
Chapter 20.

Introduction to application monitoring using Prometheus and Grafana

In this chapter, we will reuse the deployment of Prometheus and Grafana that we created in *Chapter 18, Using a Service Mesh to Improve Observability and Management,* in the *Deploying Istio in a Kubernetes cluster* section. Also in that chapter, we were briefly introduced to Prometheus, a popular open source database used to collect and store time series data such as performance metrics. There, we learned about Grafana, an open source tool to visualize performance metrics.
The Grafana deployment comes with a set of Istio-specific dashboards. Kiali can also render some performance-related graphs without the use of Grafana. In this chapter, we will get some hands-on experience with these tools.

The Istio configuration we deployed in *Chapter 18* includes a configuration of Prometheus that automatically collects metrics from Pods in Kubernetes. All we need to do is set up an endpoint in our microservice that produces metrics in a format Prometheus can consume. We also need
to add annotations to the Kubernetes Pods so that Prometheus can find the address of these endpoints. See the *Changes in source code to collect application metrics* section of this chapter for details on how to set this up. To demonstrate Grafana's capabilities to raise alerts, we will also deploy a local mail server.

The following diagram illustrates the relationship between the runtime components we just discussed:

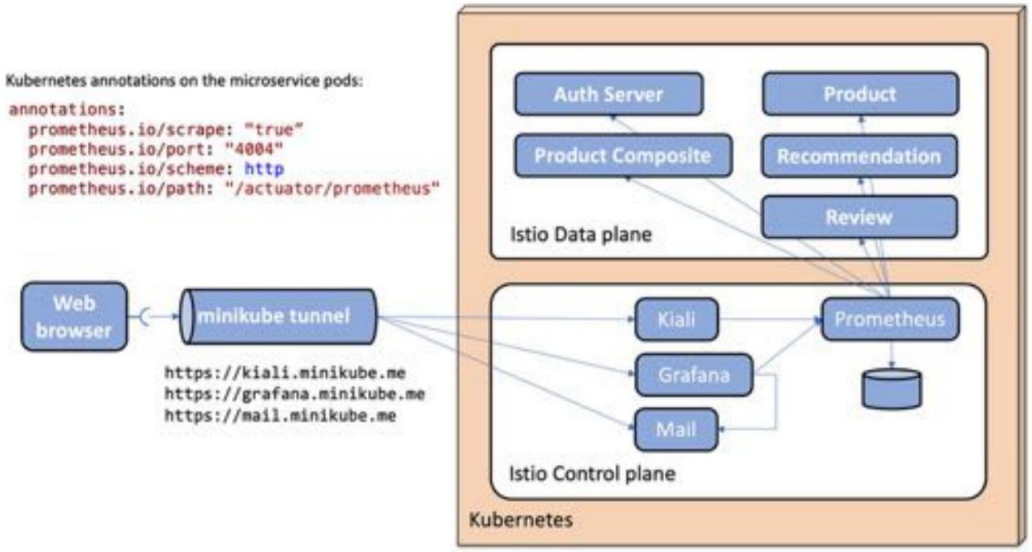


Figure 20.1: Adding Prometheus and Grafana to the system landscape

Here, we can see how Prometheus uses the annotations in the definitions of the Kubernetes Pods to be able to collect metrics from our microservices. It then stores these metrics in its database. A user can access the web UIs of Kiali and Grafana to monitor these metrics in a **web browser.** The web browser uses the **minikube tunnel** that was introduced in *Chapter 18,* in the *Setting up access to Istio services* section, to access Kiali, Grafana, and also a web page from the mail server to see alerts sent out by Grafana.

Please remember that the configuration that was used to deploy Istio in *Chapter 18*

It is only intended for development and testing, not production. For example, perfor-mance metrics stored in the Prometheus database will not survive the Prometheus

Pod being restarted!

The Istio version used in this book, v1.24.2, comes with Grafana v11.2.2 and Prometheus v2.54.1.

In the next section, we will look at what changes have been applied to the source code to make the microservices produce performance metrics that Prometheus can collect.

Changes in source code to collect application metrics

Spring Boot 2 introduced support for producing performance metrics in a Prometheus format using the **Micrometer** library (https://micrometer.io). There's only one change we need to make to the source code of the microservices: we need to add a dependency on the Micrometer library, micrometer-registry-prometheus, to the Gradle build files, build.gradle. The dependency looks like this:

```
implementation 'io.micrometer:micrometer-registry-prometheus'
```

This will make the microservices produce Prometheus metrics on port 4004 using the /actuator/prometheus path.

In *Chapter 18,* we separated the management port, used by the actuator, from the port serving requests to APIs exposed by a microservice. See the *Observing the service mesh* section for a recap, if required.

To let Prometheus know about these endpoints, each microservice's Pod is annotated with the following code:

```
annotations:
  prometheus.io/scrape: "true"
  prometheus.io/port: "4004"
  prometheus.io/scheme: http
  prometheus.io/path: "/actuator/prometheus"
```

Quick tip: Enhance your coding experience with the **AI Code Explainer** and **Quick Copy** features. Open this book in the next-gen Packt Reader. Click the **Copy** button

(1) to quickly copy code into your coding environment, or click the **Explain** button

(2) to get the AI assistant to explain a block of code to you.

This is added to the values.yaml file of each component's Helm chart. See kubernetes/helm/components for more details.

To make it easier to identify the source of the metrics once they have been collected by Prometheus, they can be tagged with the name of the microservice that produced the metric. This can be achieved by adding the following configuration to the common configuration file, config-repo/application.yml:

```
management.metrics.tags.application: ${spring.application.name}
```

This will result in each metric that's produced having an extra label named application. It will contain the value of the standard Spring property for the name of a microservice, spring.application.name.

Finally, to ensure that we get metrics from the configured Prometheus endpoints, a test must be added to test-em-all.bash. It looks like this:

```
if [[ $USE_K8S == "true" ]]
then
    # Verify access to Prometheus formatted metrics
    echo "Prometheus metrics tests"
    assertCurl 200 "curl -ks https://health.minikube.me/actuator/prometheus"
fi
```

Note that this test only runs if the test script is run against Kubernetes.

These are all the changes that are required to prepare the microservices to produce performance metrics and make Prometheus aware of what endpoints to use to start collecting them. In the next section, we will build and deploy the microservices.

Building and deploying the microservices

Building, deploying, and verifying the deployment using the test-em-all.bash test script is done in the same way it was done in *Chapter 19, Centralized Logging with the EFK Stack,* in the *Building and deploying the microservices* section. Follow these steps:

1. Build the Docker images from the source with the following commands:

```
cd $BOOK_HOME/Chapter20
eval $(minikube docker-env -u) ./gradlew
 build eval $(minikube
docker-env) docker compose build
```

The eval $(minikube docker-env -u) command ensures that the ./ gradlew build command uses the host's Docker engine and not the Docker engine in the Minikube instance. The build command uses Docker to run test containers.

2. Recreate the namespace, hands-on, and set it as the default namespace:

```
kubectl delete namespace hands-on kubectl
apply -f kubernetes/hands-on-namespace.yml kubectl config set-
context $(kubectl config current-context) --namespace=hands-on
```

3. Resolve the Helm chart dependencies.

   First, we must update the dependencies in the components folder:

```
for f in kubernetes/helm/components/*; do helm dep up $f; donated
```

   Next, we must update the dependencies in the environments folder:

```
for f in kubernetes/helm/environments/*; do helm dep up $f; donated
```

4. Deploy the system landscape using Helm and wait for all deployments to complete:

```
helm install hands-on-dev-env \
    kubernetes/helm/environments/dev-env \
    -n hands-on --wait
```

5. Start the Minikube tunnel, if it's not already running, as follows (see the *Setting up access to Istio services* section, in *Chapter 18,* for a recap if you need one):

```
minikube tunnel
```

Remember that this command requires that your user has sudo privileges and that you enter your password during startup. It takes a couple of seconds for the command to ask for the password, so it is easy to miss!

6. Run the normal tests to verify the deployment:

```
./test-em-all.bash
```

Expect the output to be similar to what we've seen in the previous chapters:

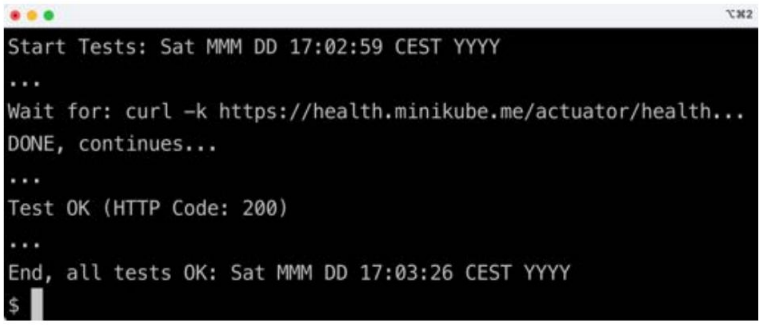


Figure 20.2: All tests are OK

With the microservices deployed, we can move on and start monitoring our microservices using Grafana!

Monitoring microservices using Grafana dashboards

As we already mentioned in the introduction, Kiali provides some very useful dashboards out of the box. In general, they focus on application-level performance metrics such as requests per second, response times, and fault percentages to process requests. As we will see shortly, they are very useful at the application level. But if we want to understand the usage of the underlying hardware resources, we need more detailed metrics, such as Java VM-related ones.

Grafana has an active community that, among other things, shares reusable dashboards. We will try out a dashboard from the community that's tailored to get a lot of valuable Java VM-related metrics from a Spring Boot application, such as our microservices. Finally, we will learn how to build our own dashboards in Grafana. But let's start by exploring the dashboards that come out of the box in Kiali and Grafana.

Before we do that, we need to make two preparations:

1. Install a local mail server for tests and configure Grafana to be able to send alert emails to it. We will use the mail server in the *Setting up alarms in Grafana* section.
2. To be able to monitor some metrics, we will start the load test tool we used in previous chapters.

Installing a local mail server for tests

In this section, we will set up a local test mail server and configure Grafana to send alert emails to the mail server.

Grafana can send emails to any SMTP mail server, but to keep the tests local, we will deploy a test mail server named maildev. Go through the following steps:

1. Install the test mail server in Istio's namespace with the following commands:

```
kubectl -n istio-system create deploy mail-server --image maildev/maildev:2.0.5

kubectl -n istio-system expose deployment mail-server --port=1080,1025 --
type=ClusterIP
kubectl -n istio-system wait --timeout=60s --for=condition=ready pod -l app=mail-server
```

2. To make the mail server's web UI available from the outside of Minikube, a set of Gateway, VirtualService, and DestinationRule manifest files have been added for the mail server in Istio's Helm chart. See the template at kubernetes/helm/environments/istio-system/templates/expose-mail.yml for more details. Run the following helm upgrade command to apply the new manifest files:

```
helm upgrade istio-hands-on-addons kubernetes/helm/environments/
istio-system -n istio-system
```

3. Verify that the test mail server is up and running by visiting its web page at https://mail.minikube.me. Expect a web page such as the following to be rendered:

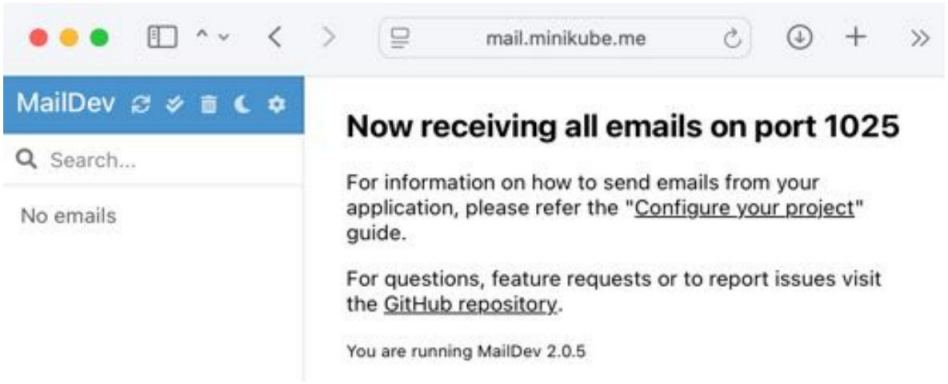


Figure 20.3: Mail server web page

For more information on the mail server, see https://hub.docker.com/r/maildev/maildev.

With the mail server installed, we can configure Grafana to send emails to the server for alerts.

Configuring Grafana

Configuring Grafana can be done by setting up environment variables in its Kubernetes Deployment object. To set Grafana in development mode, enable the alert system, and configure Grafana to send emails to the test mail server, run the following command:

```
kubectl -n istio-system set env deployment/grafana \
        GF_DEFAULT_APP_MODE=development \
        GF_AUTH_BASIC_ENABLED=false \
        GF_AUTH_ANONYMOUS_ENABLED=true \
        GF_AUTH_ANONYMOUS_ORG_ROLE=Admin \
        GF_FEATURE_TOGGLES_ENABLE=alertingSimplifiedRouting,\
alertingQueryAndExpressionsStepMode\
        GF_SMTP_ENABLED=true \
        GF_SMTP_SKIP_VERIFY=true \
        GF_SMTP_HOST=mail-server:1025 \
        GF_SMTP_FROM_ADDRESS=grafana@minikube.me
```

The first four variables are used to configure Grafana for use in a development environment, where no login is required and the admin role is assigned to anonymous users. The GF_FEATURE_TOGGLES_ENABLE variable is used to configure the alerting system, the GF_SMTP_ENABLED variable is used to allow Grafana to send emails, the GF_SMTP_SKIP_VERIFY variable is used to tell Grafana to skip SSL checks with the test mail server, the GF_SMTP_HOST variable points to our mail server, and, finally, the GF_SMTP_FROM_ADDRESS variable specifies what “from” address to use in the email.

Kubernetes will start a new Pod for Grafana that contains the new configuration. Wait for the new Pod to be ready by running the following command:

```
kubectl -n istio-system wait --timeout=60s --for=condition=ready pod -l app.kubernetes.io/
name=grafana
```

Now that Grafana has been configured, we will start the load test tool in the next section.

Starting up the load test

So that we have something to monitor, let's start up the load test using Siege, which we used in previous chapters. Run the following commands to get an access token and then start up the load test, using the access token for authorization:

```
ACCESS_TOKEN=$(curl https://writer:secret-writer@minikube.me/oauth2/token -d
grant_type=client_credentials -d scope="product:read product:write" -ks | jq .access_token -r)

echo ACCESS_TOKEN=$ACCESS_TOKEN

siege https://minikube.me/product-composite/1 -H "Authorization: Bearer $ACCESS_TOKEN" -c1
-d1 -v
```

Remember that an access token is only valid for one hour – after that, you need to get a new one.

Now, we are ready to learn about the dashboards available in Kiali and Grafana and explore the Grafana dashboards that come with Istio.

Using Kiali's built-in dashboards

In *Chapter 18,* we learned about Kiali, but we skipped the part where Kiali shows performance metrics. Now, it's time to get back to that subject!

Execute the following steps to learn about Kiali's built-in dashboards:

Open the Kiali web UI in a web browser by going to https://kiali.minikube.me. Log in with admin/admin if required.

1. To see our deployments, go to the workloads page by clicking on the **Workloads** tab from the menu on the left-hand side.
2. Select the **product-composite** deployment by clicking on it.
3. On the **product-composite** page, select the **Outbound Metrics** tab. You will see a page similar to the following:

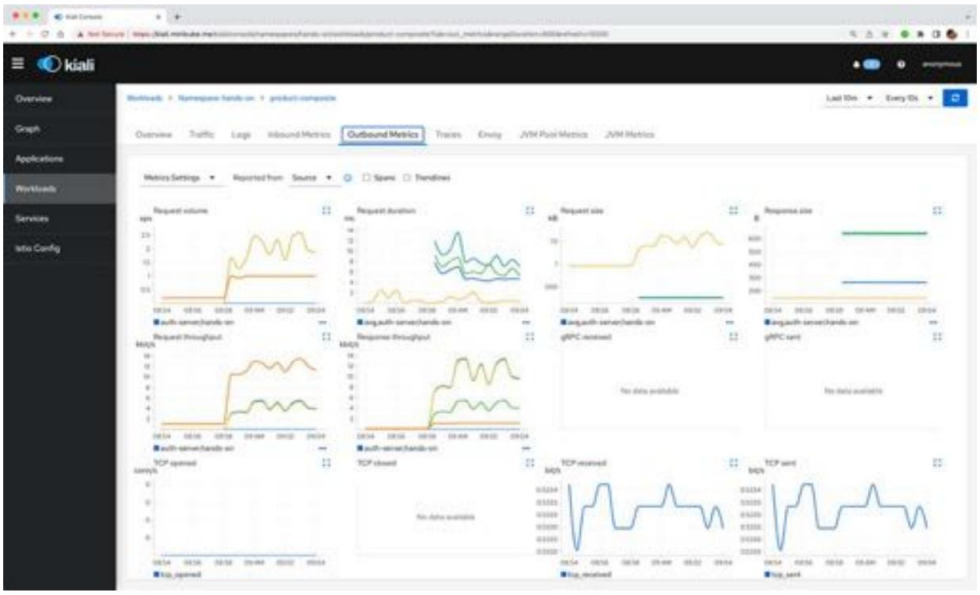


Figure 20.4: Kiali outbound metrics

Kiali will visualize some overall performance graphs that are of great value, and there are more graphs to explore. Feel free to try them out on your own!

4. However, far more detailed performance metrics are available in Grafana. Open the Grafana web UI in a web browser by going to https://grafana.minikube.me.
5. You will be presented with a welcome page with the text **Welcome to Grafana.** To the left, you will see a menu. If you don't see this menu, click on the hamburger menu with ÿ **Home** over the welcome text. Click on **Dashboards** in the menu to get an overview of the available dashboards. You will see a folder named **istio** that contains the dashboards that were installed when Grafana was deployed alongside Istio in *Chapter 18.* Click on this folder to expand it and select the dashboard named **Istio Mesh Dashboard.**

To save some space in the UI, hide the left-hand side menu by clicking on its *Undock menu* icon in the top-right corner of the menu.

You'll see a web page similar to the following:

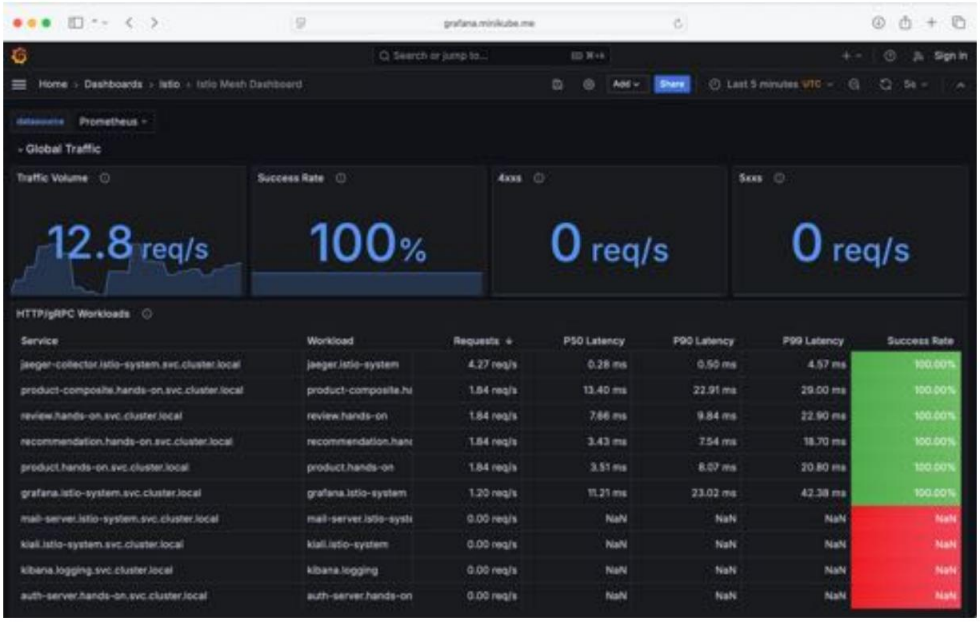


Figure 20.5: Grafana showing Istio Mesh Dashboard

This dashboard gives a very good overview of metrics for the microservices involved in the service mesh, such as request rates, response times, and success rates.

6. There are a lot of detailed performance metrics available. Go back to the **Istio** folder (click on **istio** in the top menu) and select the dashboard named **Istio Workload Dashboard.** Select the **hands-on** namespace and the **product-composite** workload. Finally, expand the **Outbound Services** tab. Collapse the top sections named **General** and **Inbound Workloads** to be able to see the section named **Outbound Workloads.** The web page should look like this:

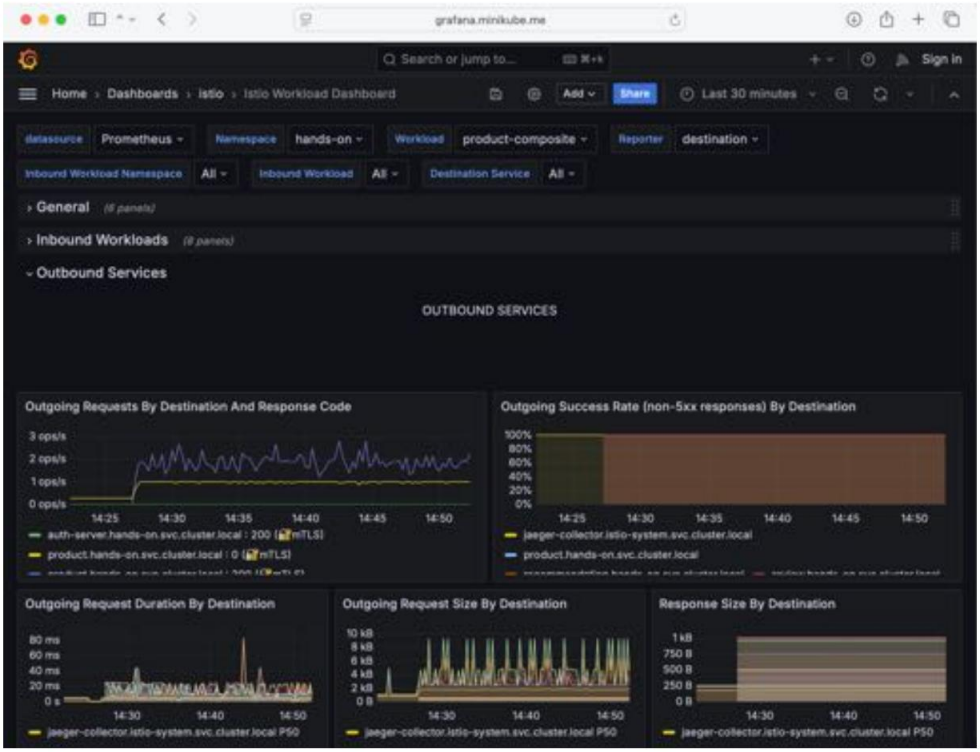


Figure 20.6: Grafana with a lot of metrics for a microservice

This page displays metrics such as response codes, duration, and bytes sent per destination. Feel free to look through the remaining dashboards provided by Istio!

As we've already mentioned, the Istio dashboards give a very good overview at an application level. However, there is also a need to monitor the metrics for hardware usage per microservice. In the next section, we will learn about how existing dashboards can be imported – specifically, a dashboard showing Java VM metrics for a Spring Boot-based application.

Importing existing Grafana dashboards

As we've already mentioned, Grafana has an active community that shares reusable dashboards. They can be explored at https://grafana.com/grafana/dashboards. We will try out a dash- board called **JVM (Micrometer) - Kubernetes - Prometheus by Istio** that's tailored to get a lot of valuable JVM-related metrics from Spring Boot applications in a Kubernetes environment. This dashboard can be found at https://grafana.com/grafana/dashboards/11955. Perform the following steps to import this dashboard:

1. Import the **JVM (Micrometer)** dashboard:

    to. On the Grafana web page, click on **Dashboards** in the menu to the left. Next, click on the blue **New** button and select **Import** from the menu that pops up.

    b. On the **Import** page, enter the dashboard ID 11955 into the **Find and import dashboards…** field and click on the blue **Load** button next to it.

    c. On the **Import** page that's displayed, click on the **Prometheus** drop-down menu and select the **Prometheus** data source.

    d. Now, by clicking on the **Import** button, the **JVM (Micrometer)** dashboard will be imported and rendered.

2. Inspect the **JVM (Micrometer)** dashboard:

    to. To get a good view of the metrics, use the time picker (in the top-right corner) to select **Last 5 minutes,** and select a refresh rate of **5s** in the dropdown to the right.

    b. In the **Application** drop-down menu, which can be found at the top left of the page, select the **product-composite** microservice.

    c. Since we are running a load test using Siege in the background, we will see a lot of metrics. The following is a sample screenshot:

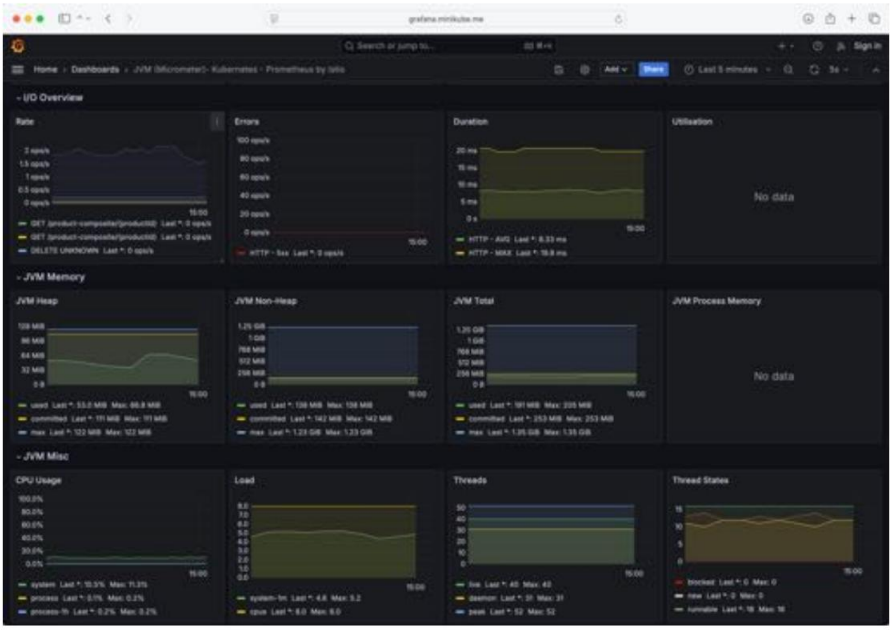


Figure 20.7: Grafana showing Java VM metrics

In this dashboard, we can find all types of relevant Java VM metrics for, among other things, CPU, memory, heap, and I/O usage, as well as HTTP-related metrics such as requests/second, average duration, and error rates. Feel free to explore these metrics on your own!

Being able to import existing dashboards is of great value when we want to get started quickly. However, what's even more important is to know how to create our own dashboards. We will learn about this in the next section.

Developing your own Grafana dashboards

Getting started with developing Grafana dashboards is simple. The important thing for us to understand is what metrics Prometheus makes available for us.

In this section, we will learn how to examine the available metrics. Based on these, we will create a dashboard that can be used to monitor some of the most interesting metrics.

Examining Prometheus metrics

Previously, in the *Changes in source code to collect application metrics* section, we configured Prometheus to collect metrics from our microservices. We can make a call to the same endpoint and see what metrics Prometheus collects. Run the following command:

```
curl https://health.minikube.me/actuator/prometheus -ks
```

Expect a lot of output from the command, as shown in the following example:

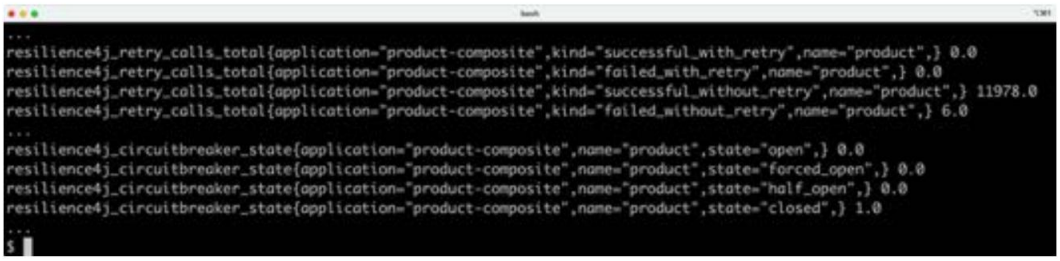


Figure 20.8: Prometheus metrics

Among all of the metrics that are reported, there are two very interesting ones:

- resilience4j_retry_calls: Resilience4j reports on how the retry mechanism operates. It reports four different values for successful and failed requests, combined with and without retries.
- resilience4j_circuitbreaker_state: Resilience4j reports on the state of the circuit breaker.

Note that the metrics have a label named application, which contains the name of the micros-service. This field comes from when we configured the management.metrics.tags.application property, which we did in the *Changes in source code to collect application metrics* section.

These metrics are interesting to monitor. None of the dashboards we have used so far use metrics from Resilience4j. In the next section, we will create a dashboard for these metrics.

Creating the dashboard

In this section, we will learn how to create a dashboard that visualizes the Resilience4j metrics we described in the previous section.

We will set up the dashboard in four stages:

- Creating an empty dashboard.
- Creating a new panel for the circuit breaker metric.
- Creating a new panel for the retry metric.
- Arranging the panels.

Creating an empty dashboard

Perform the following steps to create an empty dashboard:

1. On the Grafana web page, click on **Dashboards** in the menu to the left. Next, click on the blue **New** button and select **New dashboard** from the menu that pops up. A web page named **New dashboard** will be displayed:

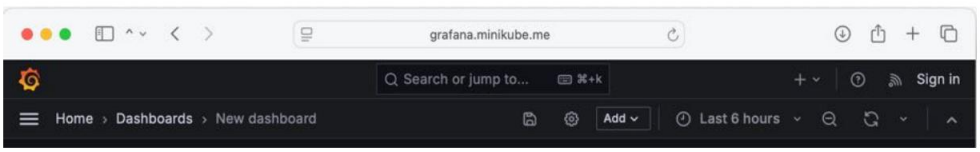


Figure 20.9: Creating a new dashboard in Grafana

2. Click on the **Dashboard settings** button (it has a gear as its icon) in the menu shown in the preceding screenshot. Then, follow these steps:

    to. Specify the name of the dashboard in the **Title** field and set the value to Hands-on Dashboard.

    b. Click on the **Close** button in the top right.

3. Click on the time picker and select **Last 5 minutes** as the range.
4. Click on the refresh rate icon to the right and specify **5s** as the refresh rate.

Creating a new panel for the circuit breaker metric

Perform the following steps to create a new panel for the circuit breaker metric:

1. Click on the blue **+ Add visualization** button and select **Prometheus** as the data source.

   A page will be displayed where the new panel can be configured.

2. On the tab to the right, in the **Panel options** section, set **Title** to Circuit Breaker.
3. Also on the tab to the right, in the **Tooltip** section, set **Tooltip mode** to **All.**
4. In the bottom-left **Query** panel, under the letter A, specify the query as the name of the circuit breaker metric for the **closed** state, as follows:

   to. Set **Metric** to resilience4j_circuitbreaker_state.

   b. Set **Label** to **state** and specify that it shall be equal to **closed.**

   c. Verify that the query at the bottom of the **A** panel is set to resilience4j_circuitbreaker_state{state="closed"}.

5. Expand the **Options** tab and, in the **Legend** drop-down box, select **Custom.** In the **Legend** field, specify {{state}} as the value. This will create a legend in the panel where the names of the different states are displayed.

   The filled-in values should look as follows:

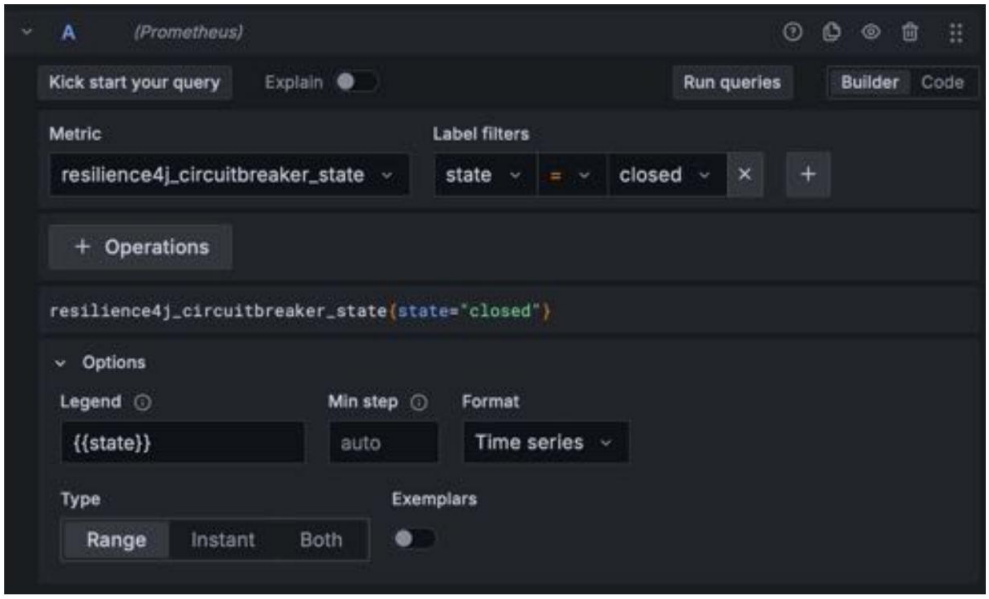


Figure 20.10: Specifying circuit breaker metrics in Grafana

6. Click on the **+ Add query** button at the bottom of the page to enter a new query under **B** for the **open** state. Repeat the steps for query A, but set the state's value to **open.** Verify that the **Raw query** field is set to resilience4j_circuitbreaker_state{state="open"} and set the **Legend** field to {{state}}.
7. Click on the **+ Add query** button at the end time to enter a new query under **C** for the **half_open** state. Set the state's value to half_open, verify that the **Raw query** field is set to resilience4j_circuitbreaker_state{state="half_open"}, and set the **Legend** field to {{state}}.
8. Click on the blue **Apply** button in the top right to get back to the dashboard.

Creating a new panel for the retry metric

Here, we will repeat the same procedure that we went through to add a panel for the preceding circuit breaker metric, but instead, we will specify the values for the retry metrics:

1. Create a new panel by clicking on the **Add** button in the top-level menu, and select **Visualization** from the drop-down menu.
2. Specify Retry as the panel's **Title** value, and set **Tooltip mode** to **All** in the same way as you did for the previous panel.
3. Set **Metric** to resilience4j_retry_calls_total.
4. Since the retry metric is a counter, its value will only go up. An ever-increasing metric is rather uninteresting to monitor. Therefore, a **rate** function is used to convert the retry metric into a rate-per-second metric. The time window specified – that is, 30s – is used by the rate function to calculate the average values of the rate. To apply the rate function, do the following:

    to. Click on the **+ Operations** button.
    b. Click on **Range functions** and select the **Rate** function.
    c. Set **Range** to **30s.**

5. Verify that the query at the bottom of the **A** panel is set to rate(resilience4j_retry_calls_total[30s]).
6. Expand the **Options** tab and, in the **Legend** drop-down box, select **Custom.** in the **legend** field, specify {{kind}} as the value. This will create a legend in the panel where the names of the different kinds of retries are displayed.
7. Note that Grafana immediately starts to render a graph in the editor panel based on the specified values.

8. Click on the blue **Apply** button in the top right to get back to the dashboard.

Arranging the panels

Perform the following steps to arrange the panels on the dashboard:

1. You can resize a panel by dragging its lower right-hand corner to the preferred size.
2. You can also move a panel by dragging its header to the desired position.
3. The following is an example layout of the two panels:

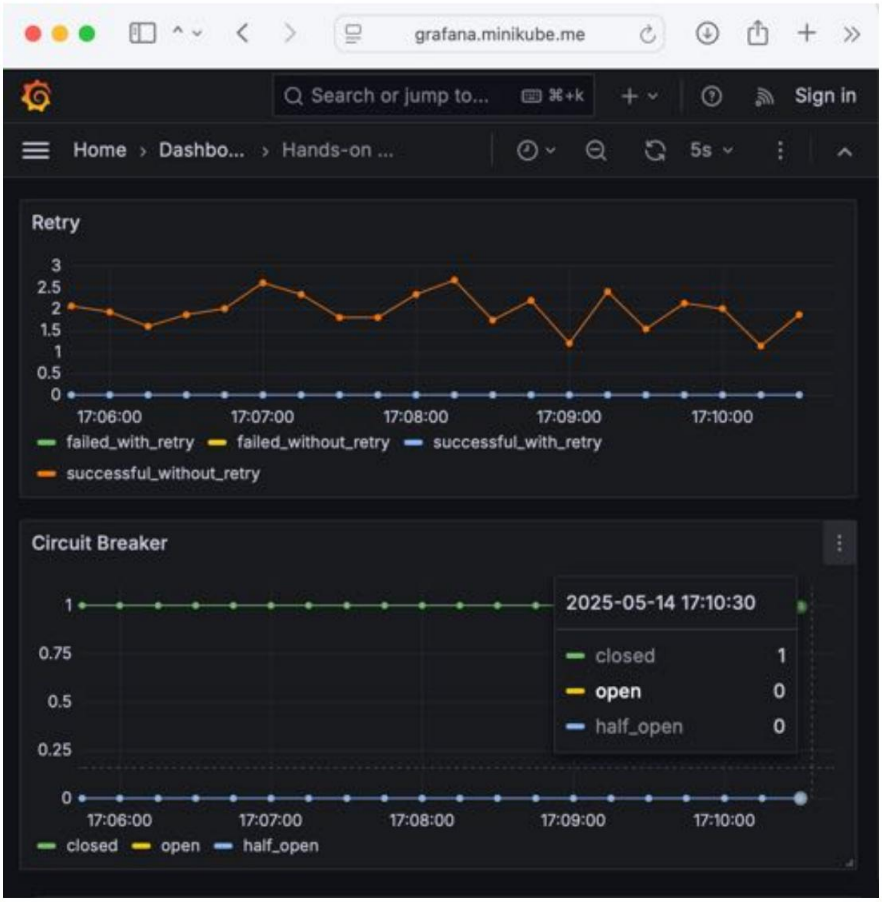


Figure 20.11: Moving and resizing a panel in Grafana

Since this screenshot was taken with Siege running in the background, the **Retry** panel reports successful_without_retry metrics, while the **Circuit Breaker** panel reports that **closed** equals **1** and **open** and **half_open** equal 0, meaning that it is closed and operating normally (something that is about to change in the next section).

4. Finally, click the **Save dashboard** button (a floppy disk icon) at the top of the page. A **Save dashboard** dialog will appear; ensure that its name is **Hands-on Dashboard** and hit the **Save** button.

If you get stuck when configuring the dashboard, take a look at the end of the *Trying out the circuit breaker alarm* section. An easy solution is described there.

With the dashboard created, we are ready to try it out. In the next section, we will try out both metrics.

Trying out the new dashboard

Before we start testing the new dashboard, we must stop the load test tool, Siege. To do this, go to the command window where Siege is running and press *Ctrl + C* to stop it.

Let's start by testing how to monitor the circuit breaker. Afterward, we will try out the retry metrics.

Testing the circuit breaker metrics

If we force the circuit breaker to open, its state will change from **closed** to **open,** and then eventually to the **half-open** state. This should be reported in the circuit breaker panel.

Open the circuit, just like we did in *Chapter 13, Improving Resilience Using Resilience4j,* in the *Trying out the circuit breaker and retry mechanism* section – that is, make some requests to the API in a row, all of which will fail. Run the following commands:

```
ACCESS_TOKEN=$(curl https://writer:secret-writer@minikube.me/oauth2/token -d
 grant_type=client_credentials -d scope="product:read product:write" -ks | jq .access_token -r)


echo ACCESS_TOKEN=$ACCESS_TOKEN

for ((n=0; n<4; n++)); do curl -o /dev/null -skL -w "%{http_code}\n" https://minikube.me/product-
 composite/1?delay=3 -H "Authorization: Bearer $ACCESS_TOKEN" -s; donated
```

We can expect three 500 responses and a final 200 response, indicating three errors in a row, which is what it takes to open the circuit breaker. The last response, 200, indicates a **fail-fast** response from the product-composite microservice when it detects that the circuit is open.

On some rare occasions, I have noticed that the circuit breaker metrics are not reported in Grafana directly after the dashboard is created. If they don't show up after a minute, simply rerun the preceding command to reopen the circuit breaker.

Expect the value for the **closed** state to drop to **0** and the **open** state to take a value of 1, meaning that the circuit is now open. After 10 seconds, the circuit will turn to the half-open state, indicated by the **half-open** metrics having a value of **1** and **open** being set to 0. This means that the circuit breaker is ready to test some requests to see whether the problem that opened the circuit is gone.

Close the circuit breaker again by issuing three successful requests to the API with the following command:

```
for ((n=0; n<4; n++)); do curl -o /dev/null -skL -w "%{http_code}\n" https://minikube.me/product-
 composite/1?delay=0 -H "Authorization: Bearer $ACCESS_TOKEN" -s; donated
```

We will get only 200 responses. Note that the circuit breaker metric goes back to normal again, meaning that the **closed** metric goes back to 1.

After this test, the Grafana dashboard should look as follows:

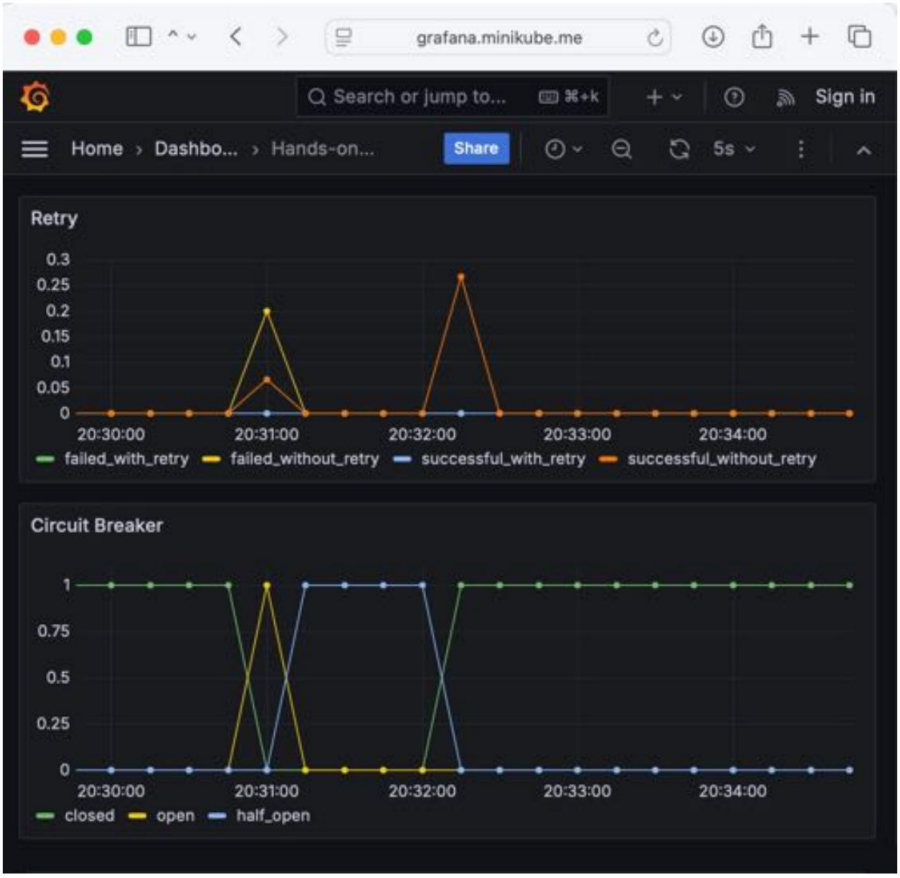


Figure 20.12: Retry and Circuit Breaker in action, as viewed in Grafana

From the preceding screenshot, we can see that the retry mechanism also reports metrics that succeeded and failed. When the circuit was opened, all requests failed without retries. When the circuit was closed, all requests were successful without any retries. This is as expected.

Now that we have seen the circuit breaker metrics in action, let's see the retry metrics in action!

If you want to check the state of the circuit breaker, you can do so with the following command:

```
curl -ks https://health.minikube.me/actuator/health | jq
-r .components.circuitBreakers.details.product.details.state
```

It should report CLOSED, OPEN, or HALF_OPEN, depending on its state.

Testing the retry metrics

To trigger the retry mechanism, we will use the faultPercentage parameter we used in previous chapters. To avoid triggering the circuit breaker, we need to use relatively low values for the parameter. Run the following command:

```
while true; do curl -o /dev/null -s -L -w "%{http_code}\n" -H "Authorization: Bearer
$ACCESS_TOKEN" -k https://minikube.me/product-composite/1?faultPercent=10; sleep 3;
donated
```

This command will call the API once every third second. It specifies that 10% of the requests should fail so that the retry mechanism will kick in and retry the failed requests.

After a few minutes, the dashboard should report various metrics:

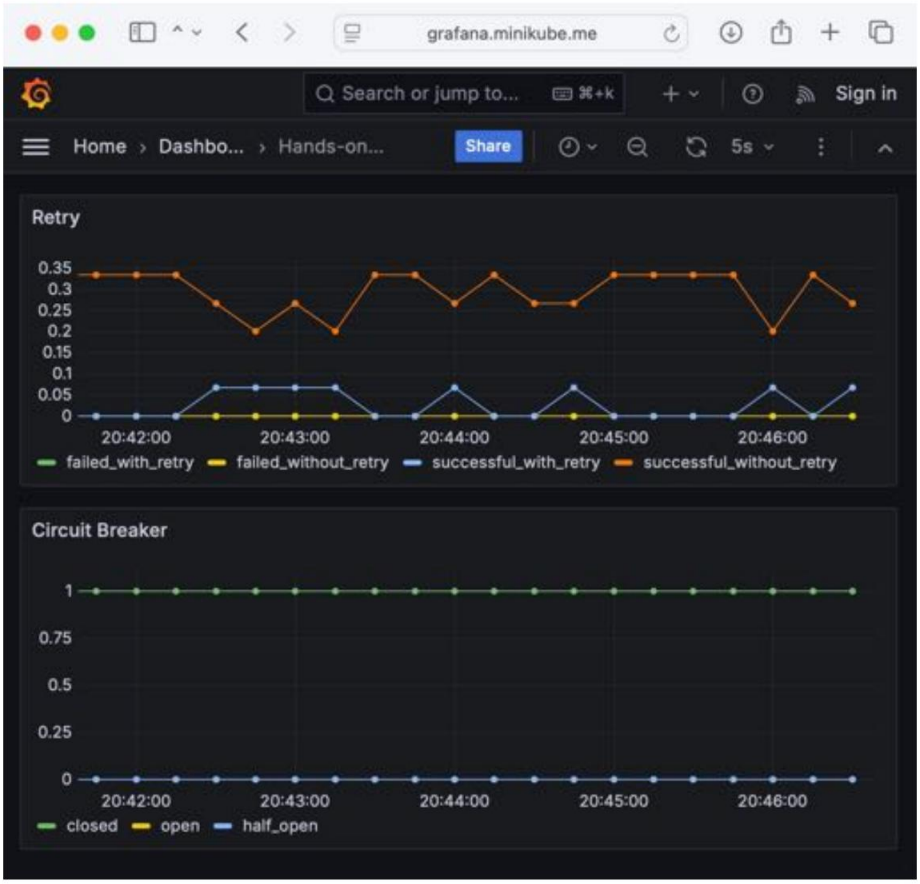


Figure 20.13: Result of retry tests viewed in Grafana

In the preceding screenshot, we can see that most of the requests have been executed successfully, without any retries. Approximately 10% of the requests have been retried by the retry mechanism and successfully executed after the retry.

Before we leave this section on creating dashboards, we will learn how we can export and import dashboards.

Exporting and importing Grafana dashboards

Once a dashboard has been created, we typically want to take two actions:

- Save the definition of the dashboard as source code in a Git repo
- Move the dashboard to other Grafana instances – for example, those used in Q/A and production environments

To perform these actions, we can use Grafana's API for exporting and importing dashboards. Since we only have one Grafana instance, we will perform the following steps:

1. Export the dashboard as a JSON file.
2. Delete the dashboard.
3. Import the dashboard from the JSON file.

Before we perform these steps, we need to understand the two different types of IDs that a dashboard has:

- id, an auto-incremented identifier that is unique only within a Grafana instance.
- uid, a unique identifier that can be used in multiple Grafana instances. It is part of the URL when accessing dashboards, meaning that links to a dashboard will stay the same, so long as the uid value of a dashboard remains the same. When a dashboard is created, a random uid value is created by Grafana.

When we import a dashboard, Grafana will try to update it if the id field is set. To be able to test importing a dashboard in a Grafana instance that doesn't have the dashboard already installed, we need to set the id field to null.

Perform the following steps to export and then import your dashboard:

1. Identify the uid value of your dashboard.

   The uid value can be found in the URL in the web browser where the dashboard is shown. It will look like this:

   ```
   https://grafana.minikube.me/d/YMcDoBg7k/hands-on-dashboard
   ```

2. The uid value in the preceding URL is YMcDoBg7k. In a Terminal window, create a variable with its value. In my case, it will be ID=YMcDoBg7k.
3. Export the dashboard to a JSON file with the following command:

```
curl -sk https://grafana.minikube.me/api/dashboards/uid/$ID | jq '.dashboard.id=null' >
"Hands-on-Dashboard.json"
```

   The curl command exports the dashboard in JSON format. The jq statement sets the id field to null, and the output from the jq command is written to a file named Hands-on-Dashboard.json.

4. Delete the dashboard.
5. In your web browser, select **Dashboards** from the top menu. Identify **Hands-on Dashboard** in the list of dashboards and select it by clicking on the checkbox in front of it. If the dashboard isn't in the list, refresh your web browser. A red **Delete** button will appear; click on it, enter Delete as asked, and then click on the new **Delete** button that is shown in the confirmation dialog box that pops up.
6. Recreate the dashboard by importing it from the JSON file. To do so, run the following command:

```
curl -i -XPOST -H 'Accept: application/json' -H 'Content-Type: application/json' -k \

    'https://grafana.minikube.me/api/dashboards/db' \
    -d @Hands-on-Dashboard.json
```

   Note that the URL used to access the dashboard is still valid – in my case, https://grafana.minikube.me/d/YMcDoBg7k/hands-on-dashboard.

7. Verify that the imported dashboard reports metrics in the same way as before it was deleted and re-imported. Since the request loop that was started in the *Testing the retry metrics* section is still running, the same metrics from that section should be reported.

For more information regarding Grafana's APIs, see https://grafana.com/docs/grafana/v11.2/developers/http_api/dashboard/#get-dashboard-by-uid

Before proceeding to the next section, remember to stop the request loop that we started for the retry test by pressing *Ctrl* + *C* in the Terminal window where the request loop executes!

In the next section, we will learn how to set up alarms in Grafana based on these metrics.

Setting up alarms in Grafana

Being able to monitor the circuit breaker and retry metrics is of great value, but even more important is the capability to define automated alarms on these metrics. Automated alarms highlight us from monitoring the metrics manually.

Grafana comes with built-in support to define alarms and send notifications to a number of channels. In this section, we will define alerts on the circuit breaker and configure Grafana to send emails to the test mail server when alerts are raised. The local test mail server was installed earlier in the *Installing a local mail server for tests* section.

For other types of channels supported by the version of Grafana used in this chap- ter, see https://grafana.com/docs/grafana/v11.2/alerting/configure- notifications/manage-contact-points/#list-of-supported-integrations.

In the next section, we will configure a mail-based contact point that will be used by the alert we set up afterwards.

Configuring a mail-based contact point

Grafana comes with a pre-defined mail-based contact point. To configure and test it, perform the following steps:

1. On the Grafana web page, click on the hamburger menu icon, ÿ, to open the menu and select **Contact points.**
2. Click on the **Edit** button for the **grafana-default-email** contact point.
3. Enter an email address of your choice. Emails will only be sent to the local test mail server, independent of the email address that's specified.

4. The configuration for the contact point should look as follows:

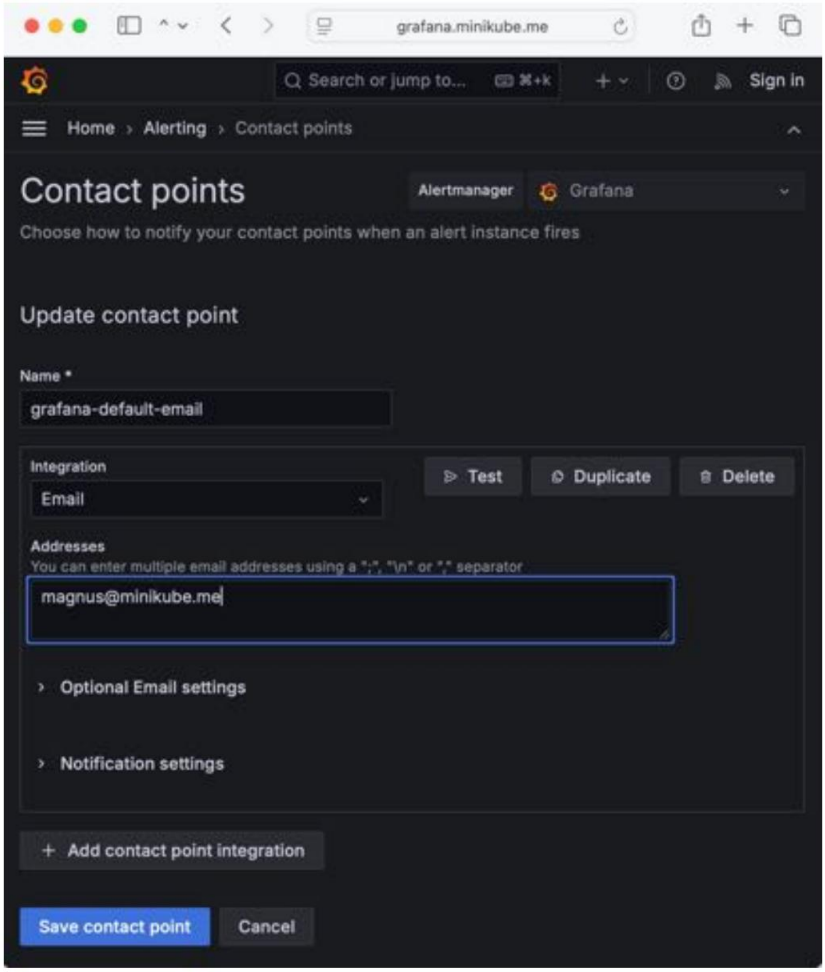


Figure 20.14: Setting up an email-based contact point

5. Click on the **Test** button to send a test email.
6. A **Test contact point** modal panel will be displayed. Click the **Send test notification** button to send the mail, then close the modal panel.
7. Click on the **Save contact point** button.
8. Check the test mail server's web page to ensure that we have received a test email. You should receive the following response:

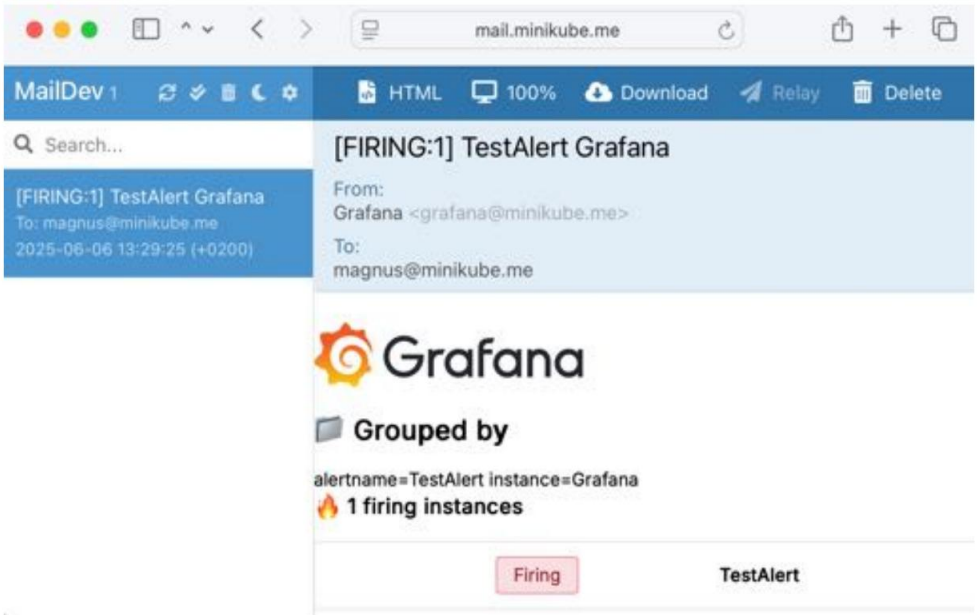


Figure 20.15: Verifying the test mail on the mail server's web page

With a contact point in place, we will modify the default notification policies before we define an alert on the circuit breaker.

Configuring default notification policies

In a development environment, you want alerts to be sent as soon as possible to verify that everything works as expected. The default notification policies within Grafana are configured for a typical production environment, where you typically want to be a bit slower with sending alerts. This is to avoid unnecessary false alerts being sent out for temporary glitches that will be resolved after a short while.

To configure the default notification policies, perform the following steps:

1. On the Grafana web page, click on the hamburger menu icon, ÿ, to open the menu and select **Notification policies** under the **Alerting** submenu.
2. Edit a **Default policy** value by clicking the three dots, …, on the right-hand side and select the **Edit** menu option.
3. Expand the **Timing options** section and set **Group wait** to 1s and **Group interval** to 5s.
4. Click the blue **Update default policy** button to save the changes.

With the default policies updated, we are ready to define an alert on the circuit breaker.

Setting up an alarm on the circuit breaker

To create an alarm on the circuit breaker, we need to create the alert and then add an alert panel to the dashboard, where we can see the current state of the alert.

An error message complaining about an incorrectly configured Loki-based data source will be displayed when setting up the alert. Either ignore the error message or delete the data source by performing these steps:

1. On the Grafana web page, click on the hamburger menu icon, ÿ, to open the menu and select **Data sources.**
2. Click on the Loki-based data source.
3. Scroll to the bottom of the page and click the red **Delete** button.
4. Click on the red **Delete** button shown in the confirmation dialog box once more.

Also, if a warning message saying **Unauthorized** is displayed, it can safely be ignored. It will disappear after a few seconds.

Perform the following steps to create an alert for the circuit breaker:

1. On the Grafana web page, click on the hamburger menu icon, ÿ, to open the menu and select **Alert rules** under the **Alerting** submenu.
2. Click on the blue **New alert rule** button to open the **New alert rule** dialog.
3. In the **1. Enter alert rule name** section, enter circuit_breaker_rule.

4. In the **2. Define query and alert condition** section, set the following values:
    to. Select **Prometheus** as the data source.
    b. In the **Metric** field, enter resilience4j_circuitbreaker_state.
    c. In the **Label filters** field, set **state** as the label and **closed** as its value.
    d. Verify that the full **Metric** value is set to resilience4j_circuitbreaker_state{state="closed"}.
    and. In the **Expressions** section, set **Function** to **Min** and **Threshold** to **IS BELOW 0.5.**
5. In the **3. Set evaluation behavior** section, set the following values:
    to. Set **Folder** to a new folder named rules_folder by clicking on the **+ New folder** button, entering the new name, and clicking the **Create** button.
    b. Set **Evaluation group** to a new group named evaluation_group by clicking on the **+ New evaluation group** button, entering the new name, setting **Evaluation interval** to **10s,** and clicking the **Create** button.
    c. Set **Pending period** to **None.**
6. In the **4. Configure labels and notifications** section, for **Contact point,** select the **graph-na-default-email** contact.
7. Save the alert rule by clicking the blue **Save rule and exit** button in the top-right menu.

The alarm rule should look as follows (to reduce the size, some parts of the settings have been removed from this screenshot):

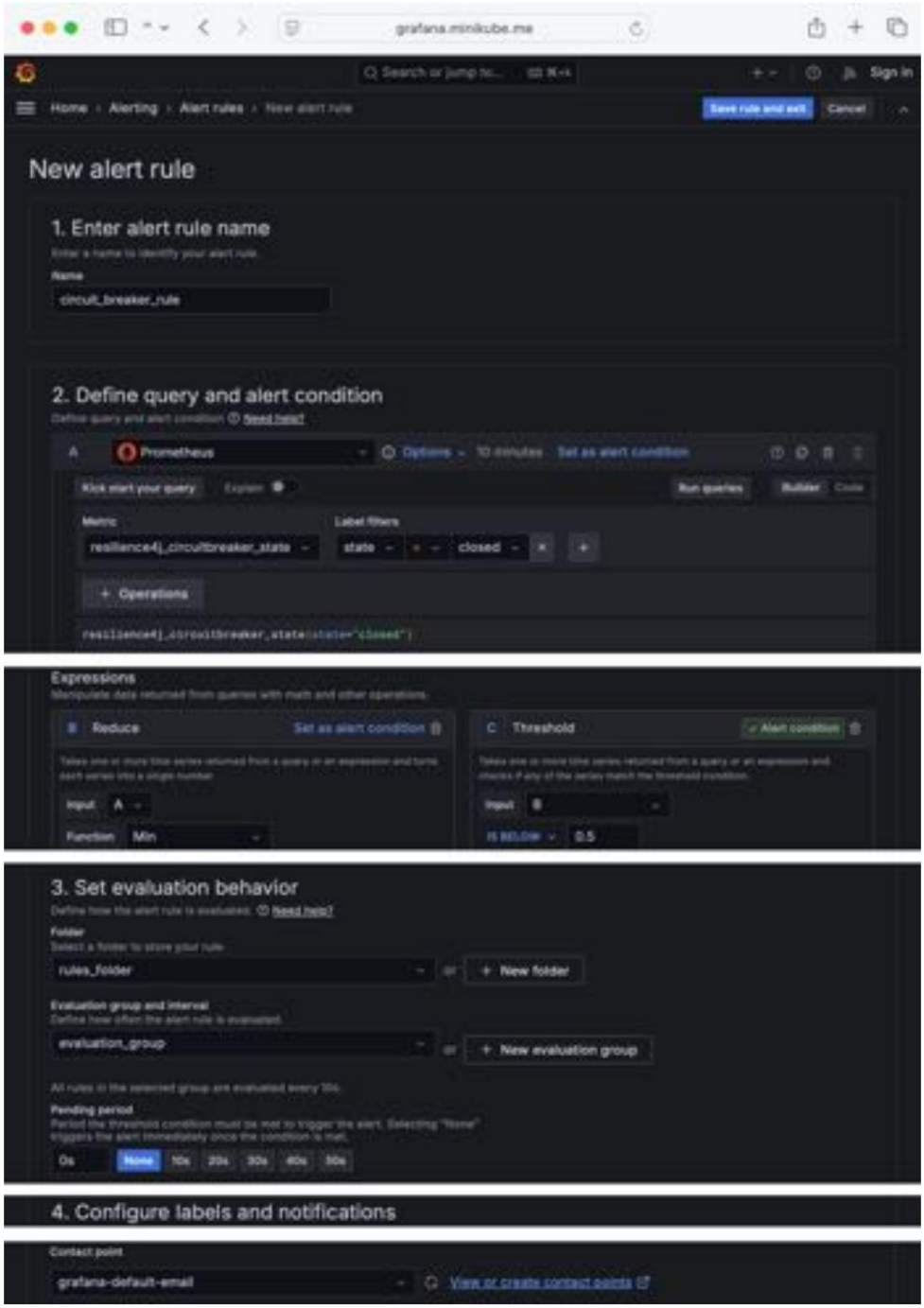


Figure 20.16: Setting up an alarm rule in Grafana, part 1

Now, we need to perform the following steps to add a panel that shows the current state of the alarm:

1. On the Grafana web page, click on the hamburger menu icon, ÿ, to open the menu and click **Dashboards.**
2. Click on **Hands-on Dashboard** to open it.
3. Create a new panel by clicking on the **Add** button in the top-level menu, and select **Visualization** from the drop-down menu.
4. In the top right, change the visualization type from **Time series** to **Alert list.**
5. Specify Circuit Breaker alert status as the panel's **Title.**
6. Select **Grafana** under **Datasource.**
7. Select **rules_folder** under **Folder.**
8. Select all options under **Alert state filter.**

   The settings should look as follows:

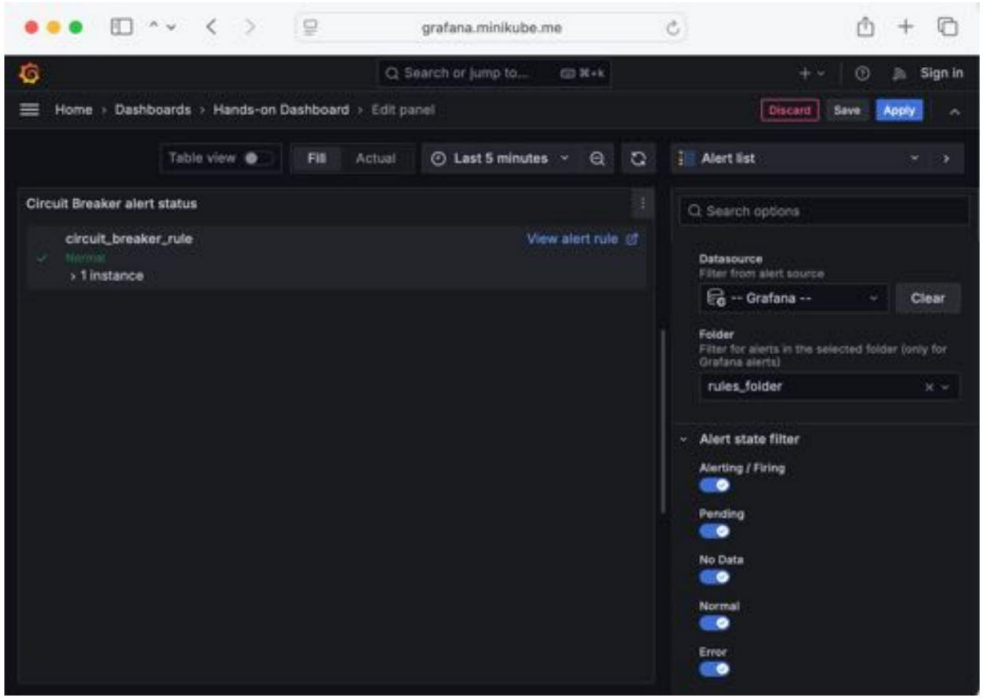


Figure 20.17: Setting up an alarm in Grafana, part 2

9. Click on the blue **Apply** button in the top right.

10. Rearrange the panel to suit your needs.
11. Finally, click the **Save dashboard** button (a diskette icon) at the top of the page. A **Save dashboard** dialog will appear; add a note, such as Added an alert list, and hit the **Save** button.

Here is a sample layout with the alarm list added:

Figure 20.18: Setting up a layout in Grafana with Retry, Circuit Breaker, and alert panels

We can see that **Circuit Breaker alert status** is set to **Normal,** meaning no alert is fired. Now, it's time to try out the alarm!

Trying out the circuit breaker alarm

Here, we will repeat the tests that we ran in the *Testing the circuit breaker metrics* section, but this time, we expect alarms to be raised and emails to be sent as well! Let's get started:

1. Acquire a new access token, if required (valid for one hour):

```
ACCESS_TOKEN=$(curl https://writer:secret-writer@minikube.me/oauth2/
token -d grant_type=client_credentials -d scope="product:read product:write" -ks |
jq .access_token -r)

echo ACCESS_TOKEN=$ACCESS_TOKEN
```

2. Open the circuit breaker, as we did previously:

```
for ((n=0; n<4; n++)); do curl -o /dev/null -skL -w "%{http_code}\n" https://minikube.me/
product-composite/1?delay=3 -H "Authorization: Bearer $ACCESS_TOKEN" -s; donated
```

The dashboard should report the circuit as open, as it did previously. After a few seconds, an alarm should be raised, and an email should also be sent. Expect the dashboard to look like this (you might need to refresh your web page for the alert to appear):

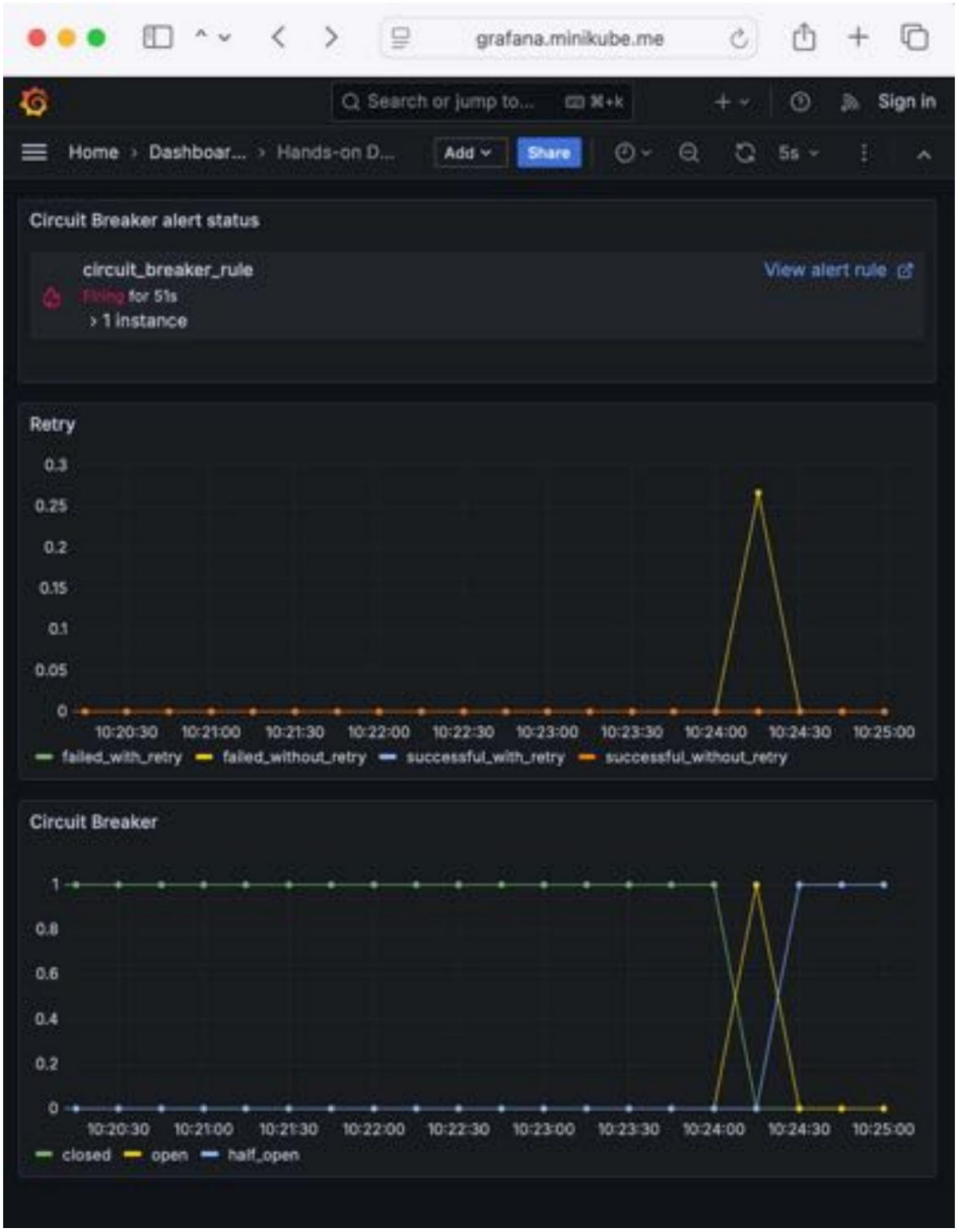


Figure 20.19: Alarm raised in Grafana

Take note of the alarm icon in the header of the circuit breaker panel (a red broken heart). The red line marks the time of the alert event and indicates that an alert has been added to the alert list.

3. In the test mail server, you should see an email, as shown in the following screenshot:

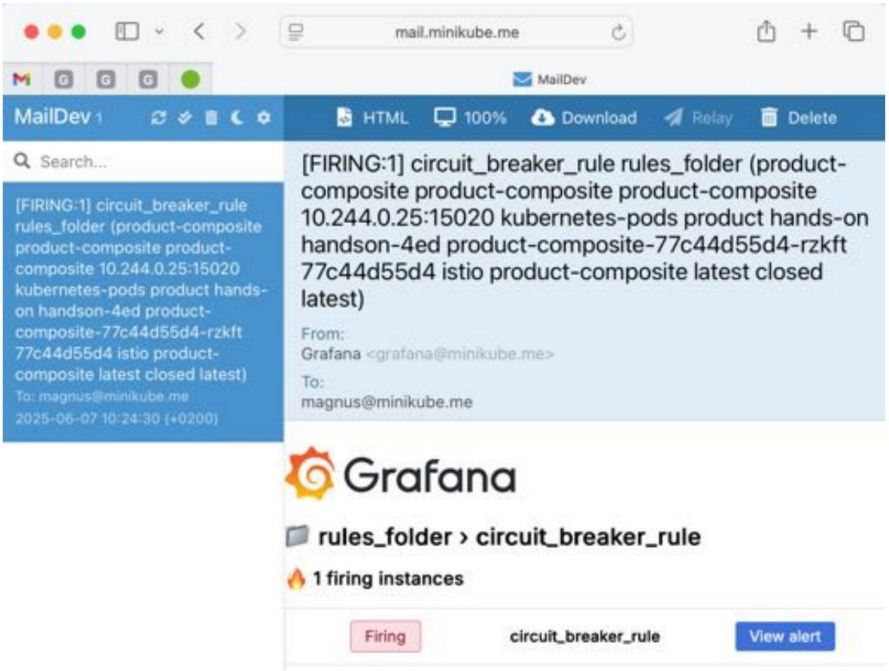


Figure 20.20: Alarm email

4. Great ! We got alarms, just like we expected! Now, close the circuit with the following command, simulating that the problem is gone:

```
for ((n=0; n<4; n++)); do curl -o /dev/null -skL -w "%{http_code}\n" https://minikube.me/
product-composite/1?delay=0 -H "Authorization: Bearer $ACCESS_TOKEN" -s; donated
```

The **closed** metric should go back to normal – that is, **1** – and the alert should turn green again.

The dashboard should now look like this:

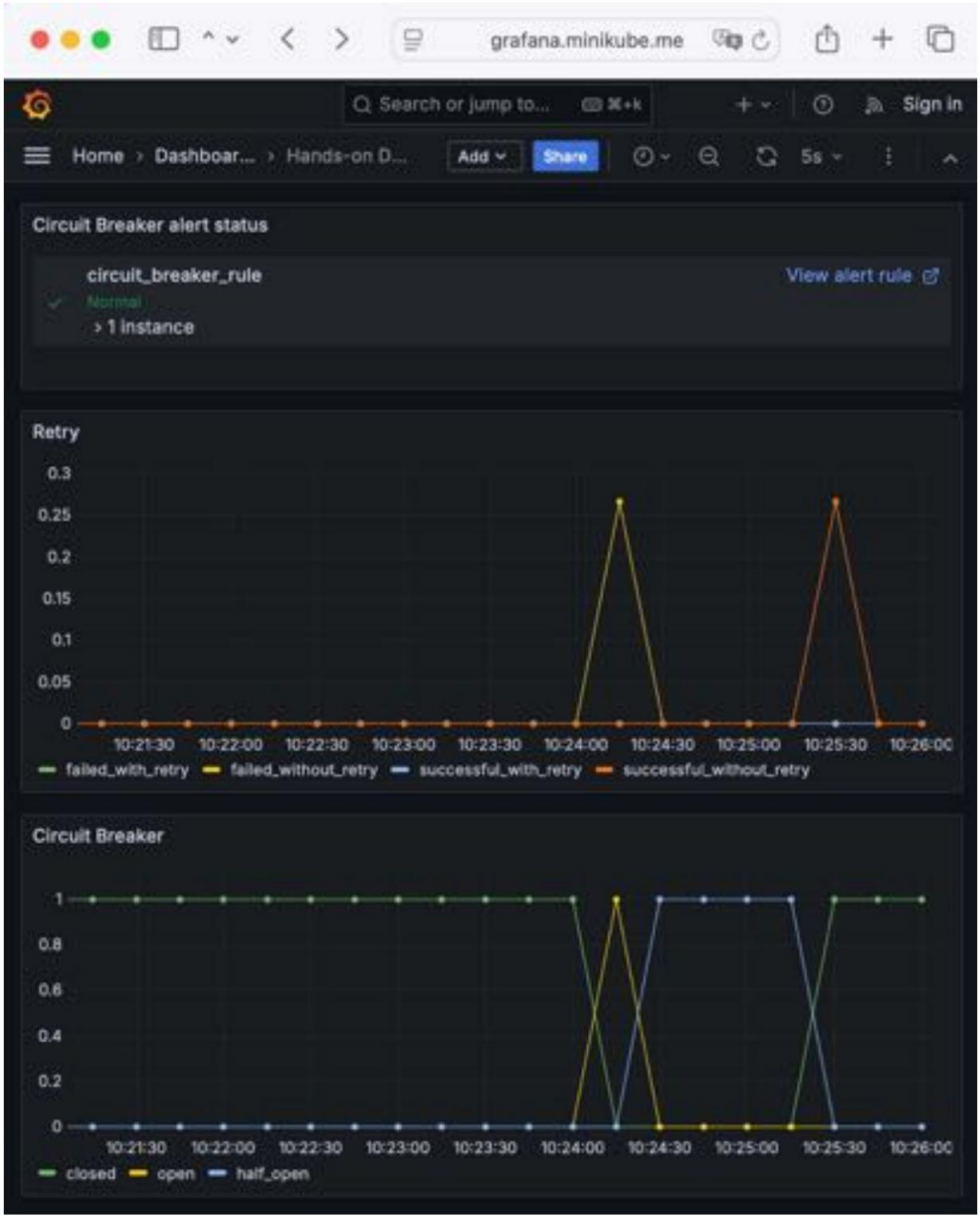


Figure 20.21: Error resolved, as reported in Grafana

Note that the alarm icon in the header of the circuit breaker panel is green again; the green line marks the time of the **OK** event and that it has been added to the alert list.

5. In the test mail server, you should see an email, as shown in the following screenshot:

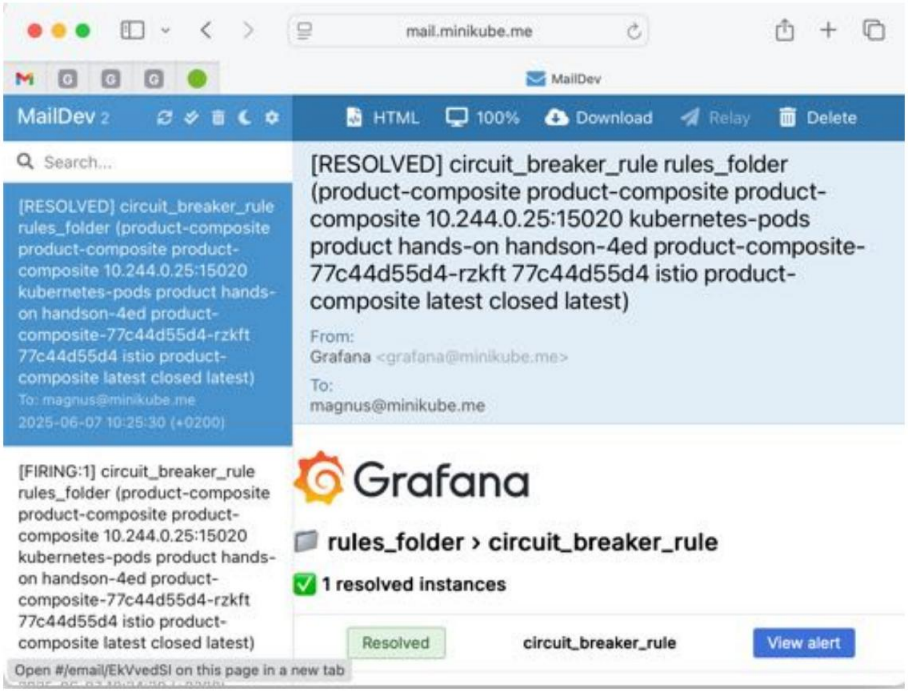


Figure 20.22: Error resolved, as reported in an email

With that, we've learned how to monitor microservices using Prometheus and Grafana.

Summary

In this chapter, we learned how to use Prometheus and Grafana to collect and monitor alerts on performance metrics.

We saw that, to collect performance metrics, we can use Prometheus in a Kubernetes environment.

We then learned how Prometheus can automatically collect metrics from a Pod when a few Prometheus annotations are added to the Pod's definition. To produce metrics in our microservices, we used Micrometer.

After, we learned how to monitor the collected metrics using dashboards in both Kiali and Grafana, which comes with the installation of Istio. We also experienced how to consume dashboards shared by the Grafana community and learned how to develop our own dashboards, where we used metrics from Resilience4j to monitor the usage of its circuit breaker and retry mechanisms. Using the Grafana API, we can export created dashboards and import them into other Grafana instances.

Finally, we learned how to define alerts on metrics in Grafana and how to use Grafana to send out alert notifications. We used a local test mail server to receive alert notifications from Grafana as emails.

The next two chapters should already be familiar to you as they cover how to install the tools used in this book on a Mac or Windows PC. So, head over to the last chapter in this book, which will introduce how we can compile our Java-based microservices into binary executable files using the brand-new **Spring Native** project, still in beta at the time of writing. This will enable the microservices to start up in a fraction of a second, although it should be noted that increased complexity and time are involved when it comes to building them.

Questions

1. What changes did we need to make to the source code in the microservices to make them produces metrics that are consumed by Prometheus?
2. What is the management.metrics.tags.application config parameter used for?
3. If you want to analyze a support case regarding high CPU consumption, which of the dashboards in this chapter would you start with?
4. If you want to analyze a support case regarding slow API responses, which of the dashboards in this chapter would you start with?
5. What is the problem with counter-based metrics such as Resilience4j's retry metrics, and what can be done so that we can monitor them in a useful way?
6. What is going on here?

Figure 20.23: What is going on here?

If you are reading this with screenshots rendered in grayscale, it might be hard to figure out what the metrics say. So, here's some help:

to. The state transitions reported by the circuit breaker are, in order:

1. **half_open** ÿ **open.**
2. **open** ÿ **half_open.**
3. **half_open** ÿ **closed.**

b. The retry mechanism reports the following:

1. An initial burst of requests, where most of them are reported as **failed_without_retry** and a few are reported as **successful_without_retry**
2. A second burst of requests, all reported as **successful_without_retry**

21

Installation Instructions for macOS

In this chapter, we will learn how to set up the tools required to run the commands described in this book on macOS. We will also learn how to gain access to this book's source code.

The following topics will be covered in this chapter:

- Technical requirements
- Installing the necessary tools
- Accessing the source code

If you are using a Windows PC, you should follow the instructions provided in *Chapter 22, Installation Instructions for Microsoft Windows with WSL 2 and Ubuntu.*

Technical requirements

All of the commands described in this book were run on a MacBook Pro with macOS Sequoia and use **bash,** a command shell. All commands have been verified on both an Intel and an Apple silicon-based MacBook Pro.

If you are using another shell, such as **zsh,** I recommend that you switch to **bash** before running any of the commands in this book. To do so, use the following command:

```
/bin/bash
```

Installing the necessary tools

In this section, we will learn how to install and configure the tools that will be used in this book.

Here is a list of the tools we will install, with a link to more information on downloading and installing them, if required:

- **Git:** https://git-scm.com/downloads
- **Docker Desktop for Mac:** https://hub.docker.com/editions/community/docker-ce-desktop-mac/
- **Java:** https://adoptium.net/installation
- **curl:** https://curl.haxx.se/download.html
- **jq:** https://stedolan.github.io/jq/download/
- **Spring Boot CLI:** https://docs.spring.io/spring-boot/3.5/installing.html#getting-started.installing.cli
- **Siege:** https://github.com/JoeDog/siege#where-is-it
- **Helm:** https://helm.sh/docs/intro/install/
- **kubectl:** https://kubernetes.io/docs/tasks/tools/install-kubectl-macos/
- **minikube:** https://minikube.sigs.k8s.io/docs/start/
- **istioctl:** https://istio.io/latest/docs/setup/getting-started/#download

The following versions were used at the time of writing this book:

- **Git:** v2.39.5
- **Docker Desktop for Mac:** v4.40.0:
  - This includes **Docker Engine:** v28.0.4
- **Java:** v24
- **curl:** v8.7.1
- **jq:** v1.7.1
- **Spring Boot CLI:** v3.5.0
- **Siege:** v4.1.7
- **Helm:** v3.17.2

- **kubectl:** v1.32.0
- **minikube:** v1.35.0
- **istioctl:** v1.24.2

To install the specific versions of Java and the Spring Boot CLI, we will use **SDKMan.** For minikube, kubectl, and istioctl, we will use the curl and install commands to install the versions used in this book. For the remaining tools, where the latest version is acceptable, the **Homebrew** pack- age manager (https://brew.sh/) will be used. We will start by installing tools using SDKMan and Homebrew. After that, we will wrap up by installing the remaining tools using the curl and install commands.

When it comes to minikube, kubectl, and istioctl, it is important to install versions that are compatible with each other, specifically when it comes to the versions of Kubernetes that they support. Simply installing and upgrading to the latest ver- sions can lead to situations where incompatible versions are used.

For supported Kubernetes versions when it comes to Istio, see https://istio.io/latest/about/supported-releases/#support-status-of-istio-releases.

For minikube, see https://minikube.sigs.k8s.io/docs/handbook/config/#selecting-a-kubernetes-version.

Installing SDKMan, Java, and the Spring Boot CLI

First, install SDKMan and verify its version by running the following commands:

```
curl -s "https://get.sdkman.io" | Bash
source "$HOME/.sdkman/bin/sdkman-init.sh"
SDK version
```

You should see a response similar to the following:

```
SDKMAN!
script: 5.18.1
native: 0.2.9
```

Next, install the Spring Boot CLI with the following command:

```
SDK install Springboot 3.5.0
```

For Java, we will install two distributions of OpenJDK:

- **Eclipse Temurin,** a plain OpenJDK distribution used in all chapters except for *Chapter 23, Native Compiled Java Microservices*
- **Oracle GraalVM,** an OpenJDK distribution that contains the **Native Image compiler** used in *Chapter 23*

If you are interested in understanding the background of why multiple distributions of OpenJDK exist, I recommend the following blog post:

https://bell-sw.com/announcements/2022/02/24/java-licensing-changes-in-2021/.

Install these two Java distributions by running the following commands:

```
sdk install java 24-tem
sdk install java 24-graalce
```

When asked if the Java distribution should be set as the default, answer yes for Temurin and no for GraalVM. This results in the Temurin distribution being used when opening a new Terminal window.

Installing Homebrew

If you don't have Homebrew installed already, you can install it with the following command:

```
/bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/
install/HEAD/install.sh)"
```

Installing Homebrew also installs the command-line tools for **Xcode,** if they are not already installed, so it might take a while.

Verify the installation of Homebrew with the following command:

```
brew --version
```

You should see a response similar to the following:

```
Homebrew 4.4.27
```

Using Homebrew to install tools

On macOS, curl comes pre-installed, and git was installed as part of the installation of the command-line tools for Xcode, required by Homebrew. Homebrew can be used to install docker, jq, helm, and siege:

```
brew install jq && \
brew install helm && \
brew install siege && \
brew install --cask docker
```

Brew installs the tools in different folders on Intel and Apple silicon-based Macs – in /usr/local and /opt/homebrew, respectively.

Install tools without Homebrew

When it comes to installing minikube, kubectl, and istioctl, we will avoid using brew for improved control over what versions we install. The commands look slightly different on Intel and Apple silicon-based Macs, so I will go through them separately.

Installing tools on an Intel-based Mac

To install the kubectl version used in this book, run the following commands:

```
curl -LO "https://dl.k8s.io/release/v1.32.0/bin/darwin/amd64/kubectl"
sudo install kubectl /usr/local/bin/kubectl
rm kubectl
```

To install the minikube version used in this book, run the following commands:

```
curl -LO https://storage.googleapis.com/minikube/releases/v1.35.0/minikube-darwin-amd64

sudo install minikube-darwin-amd64 /usr/local/bin/minikube
rm minikube-darwin-amd64
```

To install the istioctl version used in this book, run the following commands:

```
curl -L https://istio.io/downloadIstio | ISTIO_VERSION=1.24.2 TARGET_ ARCH=x86_64 sh - sudo
install istio-1.24.2/bin/
istioctl /usr/local/bin/istioctl
rm -r istio-1.24.2
```

Installing tools on an Apple silicon-based Mac

To install the kubectl version used in this book, run the following commands:

```
curl -LO "https://dl.k8s.io/release/v1.32.0/bin/darwin/arm64/kubectl"
sudo install kubectl /usr/local/bin/kubectl
rm kubectl
```

To install the minikube version used in this book, run the following commands:

```
curl -LO https://storage.googleapis.com/minikube/releases/v1.35.0/
minikube-darwin-arm64
sudo install minikube-darwin-arm64 /usr/local/bin/minikube
rm minikube-darwin-arm64
```

To install the istioctl version used in this book, run the following commands:

```
curl -L https://istio.io/downloadIstio | ISTIO_VERSION=1.24.2 TARGET_
ARCH=arm64 sh -
sudo install istio-1.24.2/bin/istioctl /usr/local/bin/istioctl
rm -r istio-1.24.2
```

If you want to use the latest versions, which, as mentioned previously, run the risk of introducing incompatibility issues, you can install minikube, kubectl, and istioctl with Homebrew by running the following commands:

```
brew install kubernetes-cli && \
brew install istioctl && \
brew install minikube
```

With the tools installed, we need to take some post-installation actions before we can verify the installations.

Post-installation actions

We need to perform some actions after installing Docker to make it work properly.

To be able to run the examples in this book, it is recommended that you configure Docker so that it can use most CPUs, except for a few (allocating all CPUs to Docker can make your computer unresponsive when tests are running), and 10 GB of memory, if available.

The initial chapters will work fine with less memory allocated – for example, 6 GB. But the more features we add later in this book, the more memory will be required by the Docker host to be able to run all the microservices smoothly.

Before we can configure Docker, we must ensure that the Docker daemon is running. You can start Docker just like you can start any application on a Mac: by using **Spotlight** or opening the Application folder in **Finder** and starting it from there.

To configure Docker, click on the Docker icon in the status bar and select **Settings…**. Go to the **Resources** tab in the settings for Docker and set **CPUs** and **Memory** accordingly, as illustrated in the following screenshot:

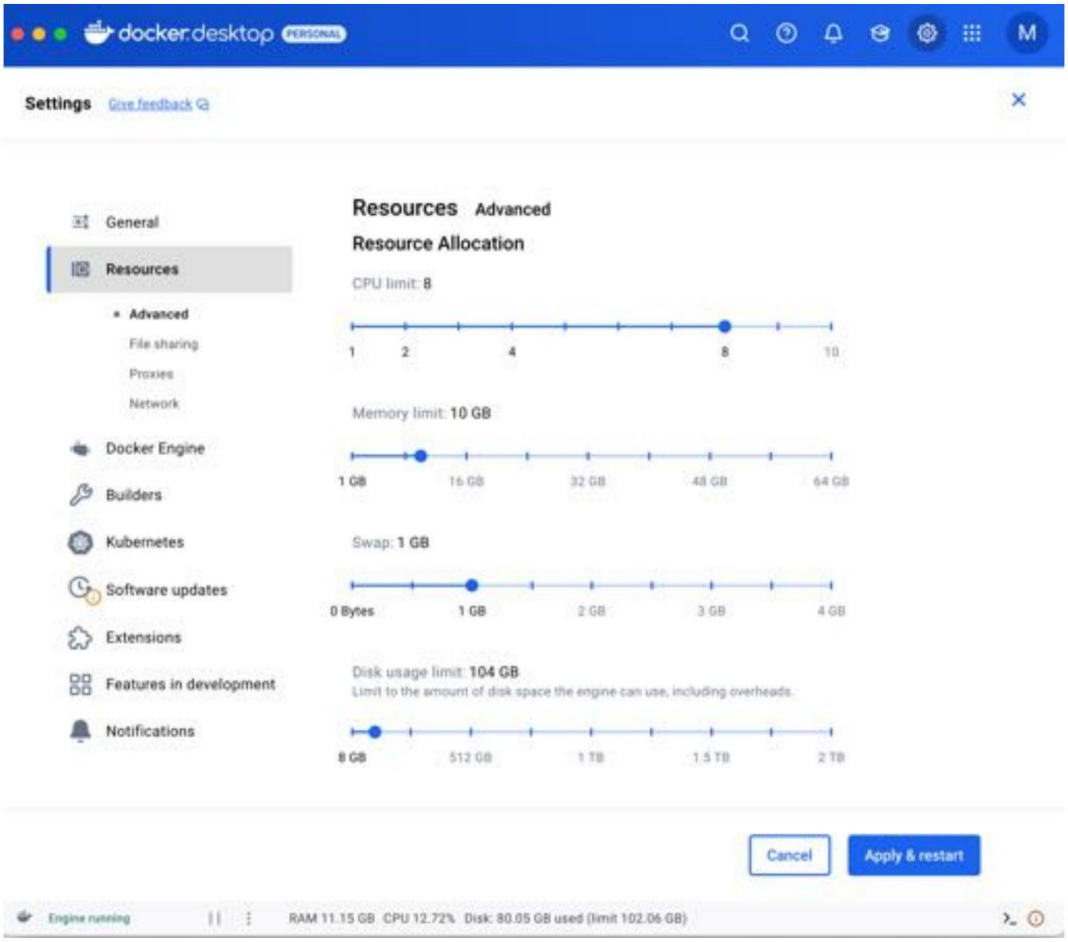


Figure 21.1: Docker Desktop resource configuration

If you don't want to start Docker manually after system startup, you can go to the **General** tab and select the **Start Docker Desktop when you log in** option:

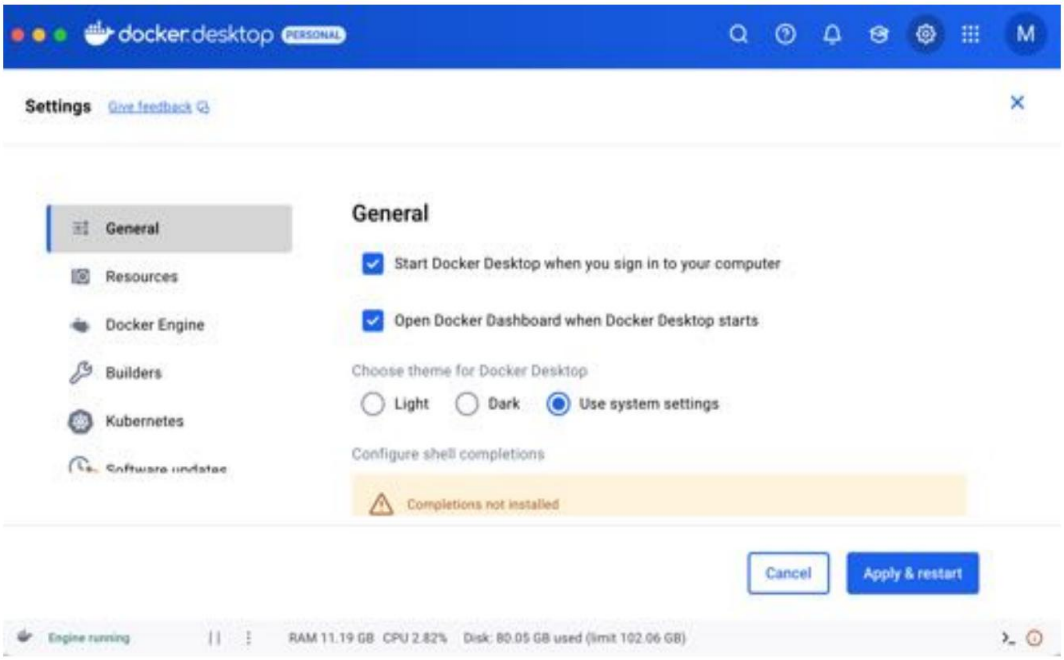


Figure 21.2: Docker Desktop general configuration

Finalize the configuration by clicking on the **Apply & Restart** button.

With the post-installation actions performed, we can verify that the tools are installed as expected.

Verifying the installations

To verify the tool installations, run the following commands to print each tool's version:

```
git version && \
docker version -f json | jq -r .Client.Version && \ java -version 2>&1
| grep "openjdk version" && \ curl --version | grep "curl" | sed 's/
(.*//' && \ jq --version && \

spring --version && \ siege
--version 2>&1 | grep SIEGE && \
helm version --short && \
```

```
kubectl version --client -o json | jq -r .clientVersion.gitVersion && \
minikube version | grep "minikube" && \
istioctl version --remote=false
```

Running these commands will return an output similar to the following:

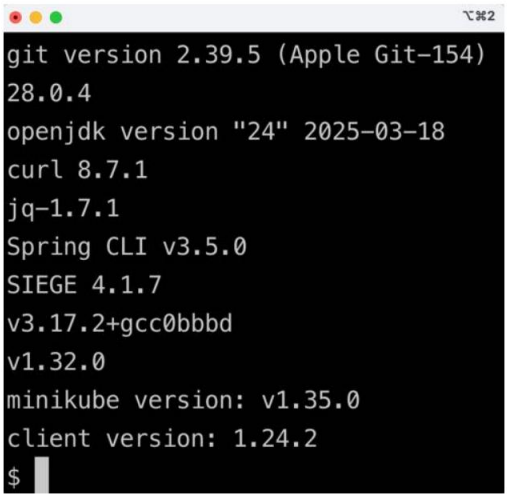


Figure 21.3: Versions used

With the tools installed and verified, let's see how we can access the source code for this book.

Accessing the source code

The source code for this book can be found in this book's GitHub repository:

https://github.com/PacktPublishing/Microservices-with-Spring-Boot-and-Spring-Cloud-Fourth-Edition.

To be able to run the commands described in this book, download the source code to a folder and set up an environment variable, $BOOK_HOME, that points to that folder. Here are some sample commands you can use:

```
export BOOK_HOME=~/Documents/Microservices-with-Spring-Boot-and-Spring-Cloud-Fourth-Edition

git clone https://github.com/PacktPublishing/Microservices-with-Spring-Boot-and-Spring-Cloud-Fourth-Edition.git $BOOK_HOME
```

The Java source code is written for Java SE 8 and uses a Java SE 24 JRE when executed in Docker containers. The following versions of Spring are used:

- **Spring Framework:** 6.2.6
- **Spring Boot:** 3.5.0
- **Spring Cloud:** 2025.0.0

The code examples in each chapter all come from the source code in $BOOK_HOME/ChapterNN, where NN is the number of the chapter. The code examples in this book have been, in many cases, edited to remove irrelevant parts of the source code, such as comments, imports, and log statements.

Using an IDE

I recommend that you work on your Java code using an IDE that supports the development of Spring Boot applications, such as Visual Studio Code, Eclipse, or IntelliJ IDEA Ultimate Edition.

For more information, see https://spring.io/tools. However, you don't need an IDE to be able to follow the instructions in this book.

The structure of the code

Each chapter consists of several Java projects – one for each microservice and Spring Cloud service, plus a couple of library projects used by the other projects. *Chapter 14* contains the largest number of projects; its project structure looks like this:

```
ÿÿÿ api
ÿÿÿ microservices
ÿ ÿÿÿ product-composite-service
ÿ ÿÿÿ product-service
ÿ ÿÿÿ recommendation-service
ÿ ÿÿÿ review-service
ÿÿÿ spring-cloud
ÿ ÿÿÿ authorization-server
ÿ ÿÿÿ config-server
ÿ ÿÿÿ eureka-server
    ÿÿÿ gateway
ÿ ÿÿÿ useful
```

All projects have been built using Gradle and have a file structure that follows Gradle's standard conventions:

```
ÿÿÿ build.gradle
ÿÿÿ settings.gradle
ÿÿÿ src
    ÿÿÿ main
    ÿ ÿÿÿ java
        ÿÿÿ resources
    ÿ ÿÿÿ test
        ÿÿÿ java
        ÿÿÿ resources
```

For more information on how to organize a Gradle project, see

https://docs.gradle.org/current/userguide/organizing_gradle_projects.html.

With this, we have the required tools installed for macOS and the source code for this book down- loaded. In the next chapter, we will learn how to set up the necessary tools in a Windows environment.
ronment.

Summary

In this chapter, we learned how to install, configure, and verify the tools required to run the commands described in this book on macOS. For development, we will use git, docker, java, and spring. To create a Kubernetes environment to deploy our microservice at runtime, we will use minikube, helm, kubectl, and istioctl. Finally, to run tests to verify that the microservices work as expected at runtime, we will use curl, jq, and siege.

We also learned how to access the source code from GitHub and how the source code is structured.

In the next chapter, we will learn how to set up the same tools in an environment based on Microsoft Windows with **Windows Subsystem for Linux v2 (WSL** 2), where we will use a Linux server based on Ubuntu.

22

Installation Instructions for Microsoft Windows with WSL 2 and Ubuntu

In this chapter, we will learn how to set up the tools required to run the commands described in this book on Microsoft Windows. We will also learn how to gain access to this book's source code.

The following topics will be covered in this chapter:

- Technical requirements
- Installing the necessary tools
- Accessing the source code

If you are using a Mac, you should follow the instructions in *Chapter 21, Installation Instructions for macOS.*

Technical requirements

All the commands described in this book have been run on a MacBook Pro using **bash** as the com-mand shell. In this chapter, we will learn how to set up a development environment in Microsoft Windows in which the commands in this book can be run without any changes being required. In a few cases, the commands must be modified so that they can be run in the Windows environment. This is pointed out in each chapter, and the alternative command to be used in Windows is also specified. All commands have been verified to work on **Windows 11.**

The development environment is based on **Windows Subsystem for Linux v2 (WSL** 2), which requires **Windows 10, version 2004** (build 19041) or later. We will use WSL 2 to run a Linux server based on **Ubuntu 24.04,** where we will run all the commands using bash as the command shell.

Microsoft provides integration between Windows and Linux servers that run in WSL 2. Linux files can be accessed from Windows, and vice versa. We will learn how to access files in a Linux server from Visual Studio Code running on Windows. Ports accessible from localhost in the Linux server are also available on localhost in Windows. We will use this integration to access web pages exposed by web applications running on the Linux server from a web browser running in Windows.

For more information on WSL 2, see https://docs.microsoft.com/en-us/windows/wsl/.

Installing the necessary tools

In this section, we will learn how to install and configure the tools. Here is a list of the tools we will install, with a link to more information on downloading and installation, if required.

On Windows, we will install the following tools:

- **Windows Subsystem for Linux v2 (WSL 2):** https://docs.microsoft.com/en-us/windows/wsl/install-win10
- **Ubuntu 24.04 in WSL** 2: https://apps.microsoft.com/detail/9nz3klhxdjp5
- **Windows Terminal:** https://www.microsoft.com/en-us/p/windows-terminal/9n0dx20hk701
- **Docker Desktop for Windows:** https://hub.docker.com/editions/community/docker-ce-desktop-windows/
- **Visual Studio Code** and its extension for **Remote WSL:** https://code.visualstudio.com and https://marketplace.visualstudio.com/items?itemName=ms-vscode-remote.remote-wsl

On the Linux server, we will install the following tools:

- **Git:** https://git-scm.com/downloads
- **Java:** https://adoptium.net/installation
- **curl:** https://curl.haxx.se/download.html
- **jq:** https://stedolan.github.io/jq/download/
- **Spring Boot CLI:** https://docs.spring.io/spring-boot/3.5/installing.html#getting-started.installing.cli
- **Siege:** https://github.com/JoeDog/siege#where-is-it
- **Helm:** https://helm.sh/docs/intro/install/#from-apt-debianubuntu

- **kubectl:** https://kubernetes.io/docs/tasks/tools/install-kubectl-linux/
- **minikube:** https://minikube.sigs.k8s.io/docs/start/
- **istioctl:** https://istio.io/latest/docs/setup/getting-started/#download

The following versions were used at the time of writing this book:

- **Windows Terminal:** v1.22.11141.0
- **Visual Studio Code:** v1.99.3
- **Docker Desktop for Windows:** v4.40.0
- **Docker Engine:** v28.0.4
- **Git:** v2.43.0
- **Java:** v24
- **curl:** v8.5.0
- **jq:** v1.7
- **Spring Boot CLI:** v3.5.0
- **Siege:** v4.0.7
- **Helm:** v3.17.2
- **kubectl:** v1.32.0
- **minikube:** v 1.35.0
- **istioctl:** v1.24.2

We will start by installing the necessary tools on Windows. After that, we will install them on the Linux server running in WSL 2.

Installing tools on Windows

In the Windows environment, we will install WSL 2 together with a Linux server, Windows Terminal, Docker Desktop, and, finally, Visual Studio Code with an extension that provides remote access to files in WSL.

Installing WSL 2 with a default Ubuntu server

Run the following commands to install WSL 2 with an Ubuntu server:

1. Open PowerShell as an administrator and run the following command:

```
wsl –install
```

2. Restart your PC to complete the installation.

3. After restarting, the Ubuntu server will automatically be installed in a new Terminal window.
4. After a while, you will be asked to enter a username and password.
5. When the installation is complete, you can verify the installed version of Ubuntu by running this command:

```
lsb_release -d
```

6. You should see an output similar to the following:

```
Description:    Ubuntu 24.04.1 LTS
```

Installing a new Ubuntu 24.04 server on WSL 2

If you already have WSL 2 installed but not an Ubuntu 24.04 server, you can install it by performing the following steps:

1. Download an installation file from the Microsoft Store:
   - To choose from the available Linux distributions for WSL 2, go to

     https://apps.microsoft.com/

   - To go directly to Ubuntu 24.04, visit

     https://apps.microsoft.com/detail/9nz3klhxdjp5
2. After downloading the installation file, execute it to install Ubuntu 24.04.
3. A console window will open and, after a minute or two, you will be asked for a username and password that will be used on the Linux server.

Installing Windows Terminal

To simplify access to the Linux server, I strongly recommend installing Windows Terminal. It provides support for the following aspects:

- Using multiple tabs
- Using multiple panes within a tab
- Using multiple types of shells – for example, Windows Command Prompt, PowerShell, bash for WSL 2, and the Azure CLI

More options are supported than this. For more information, see

https://docs.microsoft.com/en-us/windows/terminal/.

Windows Terminal can be installed from the Microsoft Store; see https://aka.ms/terminal for more details.

When you start Windows Terminal and click on the *down arrow* in the menu, you will find that it has already been configured so that it starts a Terminal on the Linux server:

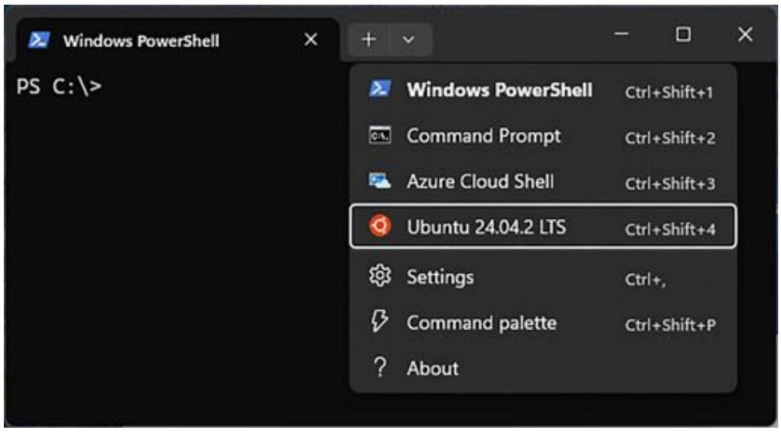


Figure 22.1: Windows Terminal configured for the Linux server in WSL 2

Select **Ubuntu-24.04;** a bash shell will be started. Depending on your settings, your working directory might be set to your home folder in Windows – for example,

/mnt/c/Users/magnus. To get to your home folder via the Linux server, simply use the cd and pwd commands to verify that you are inside your Linux server's filesystem:

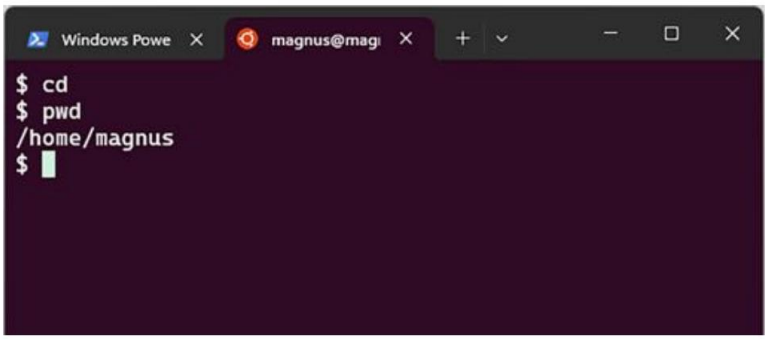


Figure 22.2: Windows Terminal using bash to access files in the Linux server

Installing Docker Desktop for Windows

To install and configure Docker Desktop for Windows, perform the following steps:

1. Download and install Docker Desktop for Windows from

   https://hub.docker.com/editions/community/docker-ce-desktop-windows/.

2. If you are asked to enable WSL 2 during installation, answer **YES.**
3. After the installation is complete, launch Docker Desktop from the **Start** menu.
4. From the **Docker** menu, select **Settings** and, in the **Settings** window, select the **General** tab:
    - Ensure that the **Use the WSL 2 based engine** check box is selected
    - To avoid starting up Docker Desktop manually each time your PC is restarted, I also recommend selecting the **Start Docker Desktop when you log in** check box

    The **General** settings should look like this:

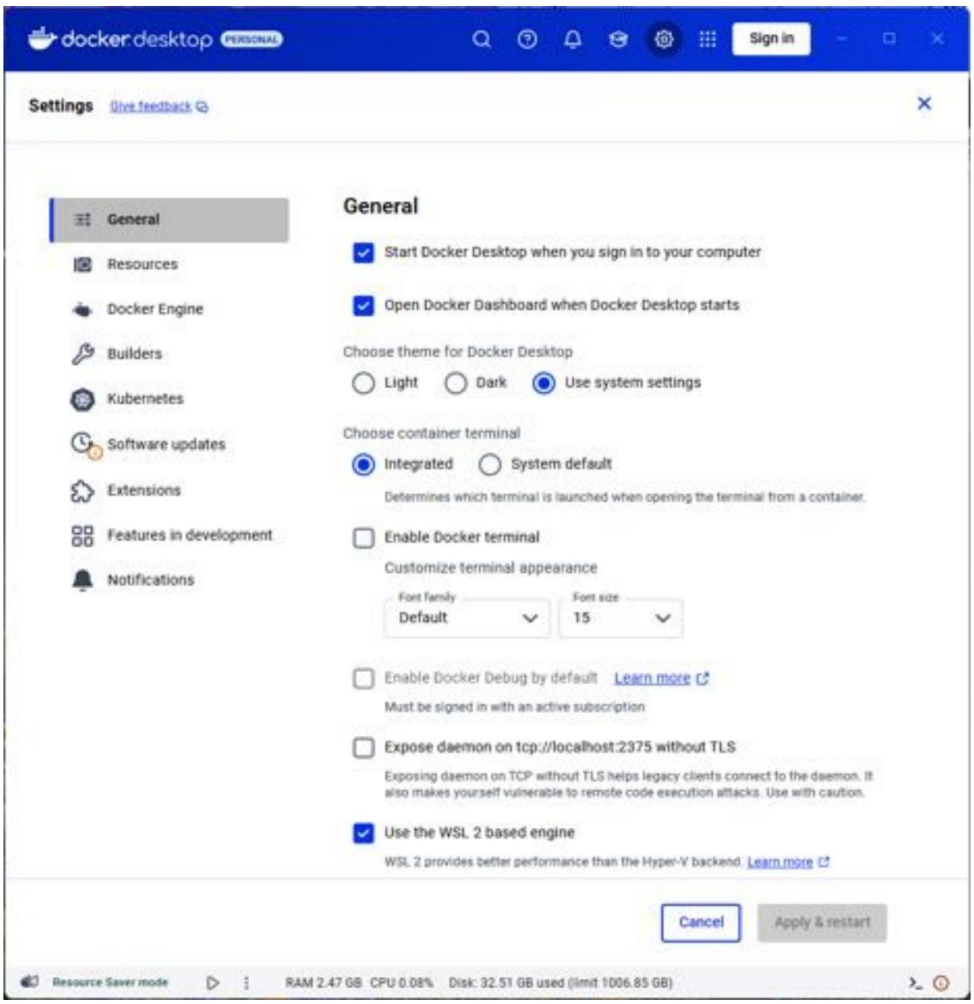


Figure 22.3: Docker Desktop configuration

5. Finalize the configuration by clicking on the **Apply & Restart** button.

Installing Visual Studio Code and its extension for Remote WSL

To simplify editing source code inside the Linux server, I recommend using Visual Studio Code. With its extension for WSL 2, named **Remote WSL,** you can easily work with source code inside the Linux server using Visual Studio Code running on Windows.

To install and configure Visual Studio Code and its extension for Remote WSL, perform the following steps:

1. Download and install Visual Studio Code from https://code.visualstudio.com:

   When asked to **Select Additional Tasks,** select the **Add to PATH** option. This will make it possible to open a folder in Visual Studio Code from within the Linux server with the code command.

2. After the installation is complete, launch Visual Studio Code from the **Start** menu.
3. Install the extension for Remote WSL: https://marketplace.visualstudio.com/items?itemName=ms-vscode-remote.remote-wsl.

If you want to learn more about how Visual Studio Code integrates with WSL 2, see this article: https://code.visualstudio.com/docs/remote/wsl.

Installing tools on the Linux server in WSL 2

Now, it is time to install the tools required on the Linux server in WSL 2.

Launch **Windows Terminal** from the **Start** menu and open a Terminal in the Linux server, as described in the *Installing Windows Terminal* section.

The git and curl tools are already installed in Ubuntu. The remaining tools will be installed using either apt install, sdk install, or a combination of curl and install.

Installing tools using apt install

In this section, we will install jq, siege, Helm, and a couple of dependencies required by the other tools.

Install jq, zip, unzip, and siege with the following commands:

```
sudo apt update
sudo apt install -y jq
```

```
sudo apt install -y zip sudo apt
install -y unzip
sudo apt install -y siege
```

To install Helm, run the following commands:

```
curl -s https://baltocdn.com/helm/signing.asc | \
    gpg --dearmor | sudo tee /usr/share/keyrings/helm.gpg > /dev/null sudo apt-get install apt-
transport-https --yes echo "deb [arch=$(dpkg --print-architecture) \

    signed-by=/usr/share/keyrings/helm.gpg] \ https://
    baltocdn.com/helm/stable/debian/ all main" | \ sudo tee /etc/apt/sources.list.d/
    helm-stable-debian.list
sudo apt-get update sudo
apt install -y helm
```

Installing the Java and Spring Boot CLI using SDKMan To install the Java and Spring Boot CLI, we will use **SDKMan** (https://sdkman.io). Install SD-KMan by running the following commands:

```
curl -s "https://get.sdkman.io" | bash source
"$HOME/.sdkman/bin/sdkman-init.sh"
```

Verify that SDKMan was installed correctly with the following command:

```
SDK version
```

Expect it to return something like this:

```
SDKMAN!
script:5.19.0
native:0.7.4 (linux x86_64)
```

For Java, we will install two distributions of OpenJDK:

- **Eclipse Temurin,** a plain OpenJDK distribution used in all chapters except for *Chapter 23, Native Compiled Java Microservices*
- **Oracle GraalVM,** an OpenJDK distribution that contains the **Native Image compiler** used in *Chapter 23*

If you are interested in understanding the background of why multiple distributions of OpenJDK exist, I recommend reading the following blog post:

https://bell-sw.com/announcements/2022/02/24/java-licensing-changes-in-2021/ .

Install the two Java distributions by running the following commands:

```
sdk install java 24-tem
sdk install java 24-graalce
```

When asked if the Java distribution should be set as the default, answer yes for Temurin and no for GraalVM. This results in the Temurin distribution being used when opening a new Terminal window.

GraalVM's Native Image compiler depends on some libraries that can be installed with the following command:

```
sudo apt install -y build-essential zlib1g-dev
```

Finally, install the Spring Boot CLI:

```
SDK install Springboot 3.5.0
```

Installing the remaining tools using curl and install

Finally, we will install kubectl, minikube, and istioctl using curl to download the executable files. Once downloaded, we will use the install command to copy the files to the proper places in the filesystem, and also ensure that owner and access rights are configured properly. When it comes to these tools, it is important to install versions that are compatible with each other, specifically when it comes to what versions of Kubernetes they support. Simply installing and upgrading to the latest versions can lead to situations where incompatible versions of minikube, Kubernetes, and Istio are used.

For supported Kubernetes versions when it comes to Istio, see

https://istio.io/latest/about/supported-releases/#support-status-of-istio-releases. For minikube, see

https://minikube.sigs.k8s.io/docs/handbook/config/#selecting-a-kubernetes-version.

To install the kubectl version used in this book, run the following commands:

```
curl -LO "https://dl.k8s.io/release/v1.32.0/bin/linux/amd64/kubectl" sudo install -o root -g root -m
0755 kubectl /usr/local/bin/kubectl
rm kubectl
```

To install the minikube version used in this book, run the following commands:

```
curl -LO https://storage.googleapis.com/minikube/releases/v1.35.0/
minikube-linux-amd64
sudo install -o root -g root -m 0755 minikube-linux-amd64 \
    /usr/local/bin/minikube
rm minikube-linux-amd64
```

To install the istioctl version used in this book, run the following commands:

```
curl -L https://istio.io/downloadIstio | ISTIO_VERSION=1.24.2 TARGET_ ARCH=x86_64 sh - sudo
install -o root -g root -m
0755 istio-1.24.2/bin/istioctl \
    /usr/local/bin/istioctl
rm -r istio-1.24.2
```

With the tools now installed, we can verify that they have been installed as expected.

Verifying the installations

To verify the tool installations, run the following commands to print each tool's version:

```
git version && \ docker
version -f json | jq -r .Client.Version && \ java -version 2>&1 | grep "openjdk
version" && \ curl --version | grep "curl" | sed 's/(.*//' && \ jq --version
&& \ spring --version && \ siege --version 2>&1 | grep SIEGE && \ helm
version --short && \
kubectl version --client -o json
| \



    jq -r .clientVersion.gitVersion && \ minikube version |
grep "minikube" && \
istioctl version --remote=false
```

You should see version information similar to the following:

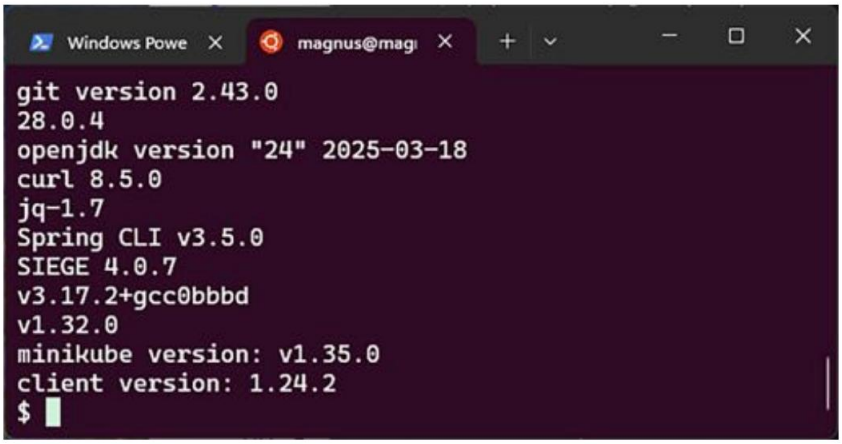


Figure 22.4: Versions used on the Linux server in WSL 2

With the tools installed and verified, let's see how we can access the source code for this book.

Accessing the source code

The source code for this book can be found in this book's GitHub repository at https://github.com/PacktPublishing/Microservices-with-Spring-Boot-and-Spring-Cloud-Fourth-Edition.

To be able to run the commands that are described in this book on the Linux server in WSL 2, download the source code to a folder and set up an environment variable, $BOOK_HOME, that points to that folder. Here are some sample commands:

```
export BOOK_HOME=~/Microservices-with-Spring-Boot-and-Spring-Cloud-Fourth-Edition

git clone https://github.com/PacktPublishing/Microservices-with-Spring-Boot-and-Spring-Cloud-Fourth-Edition.git $BOOK_HOME
```

To verify that you have access to the source code that's been downloaded to the Linux server in WSL 2 from Visual Studio Code, run the following commands:

```
cd $BOOK_HOME
code .
```

Visual Studio Code will open a window from which you can start to inspect the source code. You can also start a Terminal window for running bash commands on the Linux server by going to **Terminal** | **NewTerminal.** The Visual Studio Code window should look something like this:

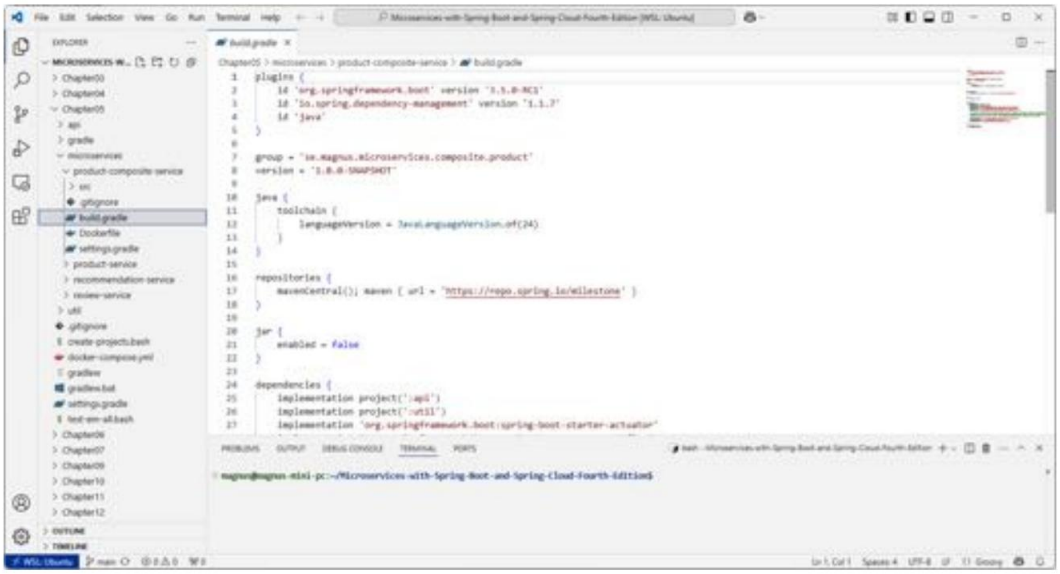


Figure 22.5: Accessing a file on the Linux server from Visual Studio Code

The Java source code is written for Java SE 8 and uses a Java SE 24 JRE when executed in Docker containers. The following versions of Spring are used:

- **Spring Framework:** 6.2.6
- **Spring Boot:** 3.5.0
- **Spring Cloud:** 2025.0.0

The code examples in each chapter all come from the source code in $BOOK_HOME/ChapterNN, where NN is the number of the chapter. The code examples in this book have been, in many cases, edited to remove irrelevant parts of the source code, such as comments, imports, and log statements.

The structure of the code

Each chapter consists of several Java projects – one for each microservice and Spring Cloud service, plus a couple of library projects used by the other projects. *Chapter 14* contains the largest number of projects; its project structure looks like this:

```
ÿÿÿ api
ÿÿÿ microservices
```

```
ÿ ÿÿÿ product-composite-service
ÿ ÿÿÿ product-service
ÿ ÿÿÿ recommendation-service
ÿ ÿÿÿ review-service
ÿÿÿ spring-cloud
ÿ ÿÿÿ authorization-server
ÿ ÿÿÿ config-server
ÿ ÿÿÿ eureka-server
ÿ gateway
ÿÿÿ useful
```

All projects have been built using Gradle and have a file structure that follows Gradle's standard conventions:

```
ÿÿÿ build.gradle
ÿÿÿ settings.gradle
ÿÿÿ src
    ÿÿÿ main
    ÿ ÿÿÿ java
        ÿÿÿ resources
    ÿ ÿÿÿ test
        ÿÿÿ java
        ÿÿÿ resources
```

For more information on how to organize a Gradle project, see https://docs.gradle.org/current/userguide/organizing_gradle_projects.html.

Summary

In this chapter, we learned how to install, configure, and verify the tools required to run the commands described in this book on WSL 2 and Windows. For development, we will use git, docker, java, and spring. To create a Kubernetes environment to deploy our microservice at runtime, we will use minikube, helm, kubectl, and istioctl. Finally, to run tests to verify that the microservices work as expected at runtime, we will use curl, jq, and siege.

We also learned how to access the source code from GitHub and how the source code is structured.

In the next and final chapter, we will learn how to compile microservices natively, reducing their startup time to sub-seconds.

23

Native Compiled Java Microservices

In this chapter, we will learn how to compile the Java source code in our microservices into binary executable files, known as **Native Images.** A Native Image starts up significantly faster compared to using a Java VM and is also expected to consume less memory. We will be introduced to the **Spring AOT** engine introduced in Spring Framework 6 and the **GraalVM** project and its **Native Image compiler,** and learn how to use them.

We will cover the following topics:

- When to natively compile Java source code
- Introducing the GraalVM project and Spring's AOT engine
- Handling problems with native compilation
- Testing and compiling Native Images
- Testing with Docker Compose
- Testing with Kubernetes

Even though Spring Framework 6 and Spring Boot 3 come with **General Availability (GA)** support for building native executables of Spring Boot applications, it must be considered as being in an early stage. At the time of writing this chapter, a lot of pit-falls were discovered while natively compiling the microservices in this book. Since natively compiling the microservices is not required for the rest of the material in this book, this chapter is placed at the end of the book as an extra chapter, describing an exciting but not yet fully mature technology.

Technical requirements

For instructions on how to install the tools used in this book and how to access the source code for this book, see the following:

- *Chapter 21, Installation Instructions for macOS*
- *Chapter 22, Installation Instructions for Microsoft Windows with WSL 2 and Ubuntu*

The code examples in this chapter all come from the source code in $BOOK_HOME/Chapter23.

If you want to view the changes applied to the source code in this chapter so you can natively compile the microservices, you can compare it with the source code for *Chapter 20, Monitoring Microservices.* You can use your favorite diff tool and compare the two folders $BOOK_HOME/Chapter20 and $BOOK_HOME/Chapter23.

When to natively compile Java source code

Java has always been known for its **build-once-run-anywhere** capability, providing excellent cross-platform support. The Java source code is compiled once into bytecode. At runtime, a Java VM transforms the bytecode into executable code for the target platform using a **Just-in-Time** compiler, also known as **JIT** compilation. This takes some time, slowing down the startup of Java programs. Before the era of microservices, Java components typically ran on an application server, such as a Java EE server. After being deployed, the Java component runs for a long time, making the longer startup time less of a problem.

With the introduction of microservices, this perspective changed. With microservices, there comes the expectation of being able to upgrade them more frequently and quickly scale instances for a microservice up and down based on their usage. Another expectation is to be able to **scale to zero,** meaning that when a microservice is not used, it should not run any instances at all. An unused microservice should not allocate any hardware resources and, even more importantly, should not create any runtime cost, for example, in a cloud deployment. To be able to meet these expectations, it is important that a microservice instance can be started quickly.

Also, with the use of containers, the importance of cross-platform support built into the application itself has faded. Instead, Docker can be used to build Docker images that contain support for multiple platforms – for example, Linux on both arm64 and amd64 (also known as x86_64), or Docker images that can be run on Windows and Linux. For more information, see https://docs.docker.com/build/building/multi-platform/. For an example of a Docker image that contains multi-platform support, see the OpenJDK Docker image used in this book: https://hub.docker.com/_/eclipse-temurin/tags.

Given that the startup time for Java programs can be significantly reduced, other use cases also come to mind; for example, developing Java-based **Function-as-a-Service (FaaS)** solutions using AWS Lambda, Azure Functions, or Google Cloud Functions, to mention some of the major platforms. Also, developing CLI tools in Java becomes a feasible option.

Together, these lead to a situation where faster startup becomes a more critical requirement than cross-platform support. This requirement can be achieved by compiling the Java source code into the target platform's binary format at build time, in the same way as C or Go programs are compiled. This is known as **Ahead of Time** compilation or **AOT** compilation. The GraalVM Native Image compiler will be used to perform the AOT compilation.

As we will see in the next section, the GraalVM Native Image compiler comes with a few restrictions – for example, relating to the use of reflection and dynamic proxies. It also takes quite some time to compile Java code into a binary Native Image. This technology has its strengths and weaknesses.

With a better understanding of when it might be of interest to natively compile Java source code, let's learn about the tooling: first, the GraalVM project, and then the Spring AOT engine.

Introducing the GraalVM project

Oracle has worked for several years on a high-performance Java VM and associated tools, known together as the **GraalVM** project (https://www.graalvm.org). It was launched back in April 2018 (https://medium.com/graalvm/graalvm-in-2018-b5fa7ff3b917), but work can be traced back to, for example, a research paper from Oracle Labs in 2013 on the subject: *Maxine: An approach-able virtual machine for, and in, Java;* see https://dl.acm.org/doi/10.1145/2400682.2400689.

Fun fact: The Maxine VM is known as a **metacircular** Java VM implementation, meaning that it is, itself, written in Java.

GraalVM's virtual machine is polyglot, supporting not only traditional Java VM languages such as Java, Kotlin, and Scala but also languages such as JavaScript, C, C++, Ruby, Python, and even programs compiled into WebAssembly.

The part of GraalVM that we will focus on is its **Native Image** compiler, which can be used to compile Java bytecode into a Native Image containing binary executable code for a specific **operating system (OS)** and HW platform – for example, macOS on Apple silicon (arm64) or Linux on Intel (amd64).

The Native Image can run without a Java VM, including binary compiled application classes and other classes required from the application's dependencies. It also includes a runtime system called **Substrate VM,** which handles garbage collection, thread scheduling, and more.

To be able to build a Native Image, the native compiler runs static code analysis based on a closed- world assumption, meaning that all bytecode that can be called at runtime must be reachable during build time. Therefore, it is not possible to load or create classes on the fly at runtime that were not available during the AOT compilation.

To overcome these restrictions, the GraalVM project provides configuration options for the native compiler where we can provide **reachability metadata.** With this configuration, we can describe the use of dynamic features such as reflection and the generation of proxy classes at runtime. For more information, see https://www.graalvm.org/latest/reference-manual/native-image/metadata/. We will learn, later in this chapter, about various ways to create the required reach-ability metadata.

The GraalVM Native Image compiler can be launched using a CLI command, Native Image, or as part of a Maven or Gradle build. GraalVM provides a plugin for both Maven and Gradle. In this chapter, the Gradle plugin will be used.

With GraalVM introduced, it is time to learn about Spring's AOT engine.

Introducing Spring's AOT engine

The Spring team has also worked on supporting the native compilation of Spring applications for some time. In March 2021, after 18 months of work, the experimental **Spring Native** project launched a beta release; see https://spring.io/blog/2021/03/11/announcing-spring-native-beta . Based on the experiences from the Spring Native project, official support for building Native Images was added in Spring Framework 6 and Spring Boot 3; see https://spring.io/blog/2023/02/23/from-spring-native-to-spring-boot-3. To perform the actual native comp-pilation, Spring uses GraalVM's Native Image compiler under the hood.

The most important feature is Spring's new **AOT engine,** which analyzes the Spring Boot application at build time and generates initialization source code and reachability metadata required by the GraalVM Native Image compiler.

The generated initialization source code, also known as **AOT-generated code,** replaces the reflection-based initialization performed when using a Java VM, eliminating most of the requirements of dynamic features such as reflection and the generation of proxy classes at runtime.

When this AOT processing is performed, a closed-world assumption is made in the same way as for the GraalVM Native Image compiler. This means that only Spring beans and classes reachable at build time will be represented in the AOT-generated code and reachability metadata. Special attention must be given to Spring beans that are only created if some profiles are set or if some conditions are met, using @Profile or @ConditionalOnProperty annotations. These profiles and conditions must be set at build time; Otherwise, these Spring beans will not be represented in the Native Image.

To perform the analysis, the AOT engine creates bean definitions for all Spring beans it can be found by scanning the source code of the application. But instead of instantiating the Spring beans, meaning starting the application, it generates the corresponding initialization code that will instantiate the Spring beans when executed. For all use of dynamic features, it will also generate the required reachability metadata.

The process of creating a Native Image from the source code of a Spring Boot application is summarized by the following data flow diagram:

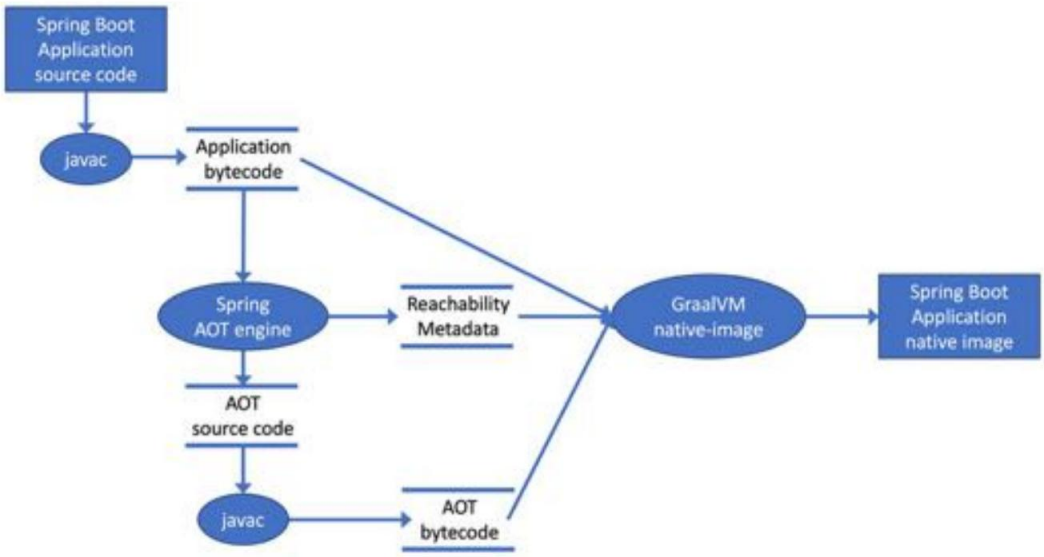


Figure 23.1: Data flow diagram explaining the creation of a Native Image

The creation of a Native Image goes through the following steps:

1. The application's source code is compiled into bytecode by the Java compiler.
2. Spring's AOT engine analyzes the code under a closed-world assumption and generates AOT source code and reachability metadata.
3. The AOT-generated code is compiled into bytecode using the Java compiler.
4. The application's bytecode, together with the reachability metadata and bytecode created by the AOT engine, is sent to GraalVM's Native Image compiler, which creates the Native Image.

For more information on how the creation of GraalVM Native Images is supported in Spring Boot 3, see https://docs.spring.io/spring-boot/docs/current/reference/html/NativeImage.html.

Building a Native Image can be done in two ways:

- Creating a Native Image for the current OS

  The first option is to use Gradle's nativeImage task. It will use the installed GraalVM's Native Image compiler to create an executable file for the current OS and hardware architecture . The nativeImage task is available given that GraalVM's Gradle plugin is declared in the build file.

- Creating a Native Image as a Docker image

  The second option is to use the existing Gradle task, bootBuildImage, to create a Docker image. Given that GraalVM's Gradle plugin is declared in the build file, the bootBuildImage task will create a Docker image that contains the Native Image instead of a Java VM with the application's JAR file that is used otherwise. The Native Image will be built in a Docker container so that it will be built for Linux. This also means that the GraalVM's Native Image compiler does not need to be installed when the bootBuildImage task is used. Under the hood, this task uses **buildpacks,** instead of a Dockerfile, to create the Docker image.

The concept of buildpacks was introduced by Heroku back in 2011. In 2018, the **Cloud Native Buildpacks** project (https://buildpacks.io) was created by Pivotal and Heroku, and later that year, it joined CNCF.

To be a bit more formal, a buildpack creates an **OCI image,** according to the OCI Image Format Specification: https://github.com/opencontainers/image-spec/blob/master/spec.md. Since the OCI specification is based on Docker's image format, the formats are very similar and are both supported by container engines.

To create the OCI images, Spring Boot uses buildpacks from the **Paketo** project; for more information , see https://docs.spring.io/spring-boot/reference/packaging/container-images/cloud-native-buildpacks.html and https://paketo.io/docs/builders.

Creating Native Images with the nativeImage task for the local OS is faster than creating Docker images. Therefore, the nativeImage task can be used for a quick feedback loop when initially trying to build a Native Image successfully. But once that has been worked out, creating Docker images containing the Native Image is the most useful alternative for testing natively compiled microservices, together with either Docker Compose or Kubernetes.

Several tools and projects are available to help eliminate challenges with native compilation. The next section will give an overview of them, and in the *Testing and compiling Native Images* section, we will learn how to use them.

Handling problems with native compilation

Natively compiling Spring Boot applications is, as already mentioned, not yet mainstream. Therefore, you will probably run into problems when trying it out on your own applications. This section will go through a few projects and tools that can be used to handle these problems. Examples of how to use these tools will be provided in the following sections.

The following project and tools can be used to handle problems with the native compilation of Spring Boot applications:

- **Spring AOT smoke tests:** This project contains a suite of tests verifying that the various Spring projects work when natively compiled. Whenever you encounter issues with natively compiling a Spring feature, you should start looking into this project for a working solution. Also, if you want to report a problem with natively compiling a Spring project, you can use tests from this project as a boilerplate to demonstrate the problem in a reproducible way. The project is available at https://github.com/spring-projects/spring-aot-smoke-tests . Test results can be found in Spring's CI environment. For example, the tests of the various Spring Cloud projects can be found here: https://spring-asa-aot-smoke-tests-dashboard.azuremicroservices.io/#cloud.
- **GraalVM reachability metadata repository:** This project contains reachability metadata for various open source projects that do not yet support native compilation themselves. The GraalVM community can submit reachability metadata, which is approved after a review by the project's team. GraalVM's Gradle plugin automatically looks up reachabil-ity metadata from this project and adds it when natively compiling. For more information , see https://graalvm.github.io/native-build-tools/0.10.6/gradle-plugin.html#metadata-support.

- **Testing AOT-generated code with the Java VM:** Since natively compiling a Spring Boot application takes a few minutes, an interesting alternative can be to try out the initialization code generated by the Spring AOT engine on the Java VM. Normally, the AOT-generated code is ignored when using a Java VM, but this can be changed by setting the spring.aot.enabled system property to true. This means that the normal reflection-based initialization of the application is replaced by executing the generated initialization code. This can be used as a quick verification that the generated initialization code works as expected. Another positive effect is that the application starts slightly faster.
- **Providing custom hints:** If an application requires custom reachability metadata for the GraalVM Native Image compiler to create a Native Image, it can be provided as JSON files, as described in the *Introducing the GraalVM project* section. Spring provides an alternative to the JSON files by either using an annotation named @RegisterReflectionForBinding or implementing the RuntimeHintsRegistrar interface in a class, which can be activated by using the @ImportRuntimeHints annotation. The RegisterReflectionForBinding annotation is easier to use, but implementing the RuntimeHintsRegistrar interface gives full control of the hints specified.

  One important benefit of using Spring's custom hints over using GraalVM JSON files is that the custom hints are type-safe and checked by the compiler. If an entity referred to in a GraalVM JSON file is renamed but the JSON file is not updated, that metadata is lost. This will result in the GraalVM Native Image compiler failing to create a Native Image. When using custom hints, the source code will not even compile; Typically, the IDE will complain as soon as the entity is renamed that the custom hint is no longer valid.
- **Running native tests:** Even though testing AOT-generated code with the Java VM can give a quick indication of whether the native compilation will work, we still need to create the Native Image of the application to test it fully. A feedback loop based on creating a Native Image, starting the application, and finally running some tests by hand is very slow and error-prone. A compelling alternative to this process is to run **native tests,** where Spring's Gradle plugin will automatically create a Native Image and then run JUnit tests defined in the application's project using the Native Image. This will still take time due to the native compilation, but the process is fully automated and repeatable. After ensuring the native tests run as expected, they can be placed in a CI build pipeline for automated executio
  Native tests can be started using Gradle with this command:

```
gradle nativeTest
```

- **Using GraalVM's tracing agent:** If it turns out to be hard to determine what reachability metadata and/or custom hints are required to create a working Native Image for an application, GraalVM's tracing agent can help out. If the tracing agent is enabled when running the application in the Java VM, it can gather the required reachability metadata based on how the application uses reflection, resources, and proxies. This is specifically useful if run together with JUnit tests, since gathering the required reachability metadata will be automated and repeatable.

With the tooling introduced and explained for how to handle the main expected challenges, let's see what changes are required in the source code to be able to natively compile the microservices.

Changes in the source code

Before compiling the Java source code in the microservices into native executable images, the source code needs to be updated a bit. To be able to natively compile the microservices, the following changes have been applied to the source code:

- The Gradle build files, build.gradle, have been updated by adding the GraalVM plugin, adjusting some dependencies, and configuring the bootBuildImage command.
- Required reachability metadata and custom hints have been added.
- Build-time property files have been added to ensure that required Spring beans are reachable able during the AOT processing at build time.
- Some properties used at runtime have been added to config-repo to make the natively compiled microservices operate successfully.
- The configuration to be able to run the GraalVM Native Image tracing agent has been added.
- The verification script, test-em-all.bash, has been updated since the Docker images no longer include the wget command.
- Some native tests have been disabled using the @DisabledInNativeImage annotation as described in the *Running native tests* section.
- Two new Docker Compose files for using the Docker images containing the Native Images have been added.
- A system property has been added to the microservices' Dockerfile to simplify toggling the AOT mode. The ENVIRONMENT command has been updated to disable the AOT mode when running with the Java VM. It looks like this:

```
ENTRYPOINT ["java", "-Dspring.aot.enabled=false", "org.
springframework.boot.loader.JarLauncher"]
```

Note that it doesn't work to specify spring.aot.enabled as an environment variable or in a property file; it has to be set as a system property on the java command.

Let's go through the changes one by one and start with the changes applied to the build files.

Updates to the Gradle build files

The changes described in this section have been applied to the build.gradle files in each microservice project unless stated otherwise.

The following updates have been applied:

- To enable Spring AOT tasks, the GraalVM plugin has been added:

```
plugins {
    ...
    id 'org.graalvm.buildtools.native' version '0.10.6'
}
```

- The bootBuildImage task is configured to specify the name of the Docker image that is created. The same naming conventions are used as in earlier chapters, but the name of the image is prefixed with native- to separate it from the existing Docker images. Also, the created date in the image's metadata is set to the current timestamp. By default, it is set to 0, meaning 1 January 1970 in **epoch** time. For the product microservice, the configuration looks like this:

```
tasks.named('bootBuildImage') {
    createdDate = "now"
    imageName = "hands-on/native-product-service"
}
```

- Due to the issue described at https://github.com/spring-projects/spring-boot/issues/33238, the jar task is no longer disabled.

To recap why the jar task was disabled, see the *Implementing our API* section in *Chapter* 3.

These are all the changes required for the build files. In the next section, we will learn about how we need to help the native compiler to compile our source code in some cases.

Providing reachability metadata and custom hints

There are a few cases in the source code where the GraalVM native compiler needs help to be able to compile the source code correctly. The first case is the JSON-based APIs and messages that the microservices use. The JSON parser, Jackson, must be able to create Java objects based on the JSON documents that the microservices receive. Jackson uses reflection to perform this work, and we need to tell the native compiler about the classes to which Jackson will apply reflection.

For example, a native hint for the Product class looks like this:

```
@RegisterReflectionForBinding({ Event.class, ZonedDateTimeSerializer.
class, Product.class})public class ProductServiceApplication {
```

All necessary custom hint annotations have been added to each microservice's main class.

Also, Jackson requires Joda time-related resources to be loaded at startup. To instruct the GraalVM native compiler to keep these resources, an implementation of the RuntimeHintsRegistrar in-terface is provided. The most central parts of the class look like the following:

```
@Configuration
@ImportRuntimeHints(NativeHintsConfiguration.class)
public class NativeHintsConfiguration implements RuntimeHintsRegistrar {

    @Override
    public void registerHints(RuntimeHints hints, ClassLoader classLoader) {
        hints.resources().registerPattern("org/joda/time/tz/data/**");
    }
}
```

From the preceding source code, we can see that a hint is registered for resources found in the org/joda/time/tz/data folder, including its subfolders. The class also provides the necessary configuration by importing itself using the ImportRuntimeHints annotation. Each microservice contains its own hints registration class with the hints required per microservice.

A final corner case is that we must provide reachability metadata for Resilience4J's use of reflection in the declaration of the retry mechanism. The configuration looks like this:

```
resilience4j.retry:
    instances:
        product:
            maxAttempts: 3
            waitDuration: 1000
            retryExceptions:
            - org.springframework.web.reactive.function.client.
WebClientResponseException$InternalServerError
```

This configuration will enable the retry mechanism to retry errors of type InternalServerError. To let the GraalVM Native Image compiler know that reflection must be enabled for this class, it has been added to the product-composite's native hints declaration like this:

```
@RegisterReflectionForBinding({ ...,
  WebClientResponseException.InternalServerError.class })
```

We now know how to provide metadata and custom hints for our own source code. Next, we will learn how we can ensure that Spring beans required at runtime also exist at build time so that the AOT processing can introspect them and generate proper AOT code.

Enabling Spring beans at build time in application.yml files

As mentioned previously, due to the closed-world assumption that the static analysis uses at build time, all Spring beans required at runtime must be reachable at build time. Otherwise, they cannot be activated at runtime. Given that they are reachable at build time, they can be configured at runtime. To summarize, this means that if you are using Spring beans that are only created if some profiles are set or if some conditions are met (using @Profile or @ConditionalOnProperty annotations), you must ensure that these profiles and conditions are met at build time.

For example, the possibility to specify a separate management port at runtime when using a natively compiled microservice is only possible if the management port was set to a random port (different from the standard port) at build time. Therefore, each microservice has an application.yml file in its src/main/resources folder that specifies the following:

```
# Required to make the Spring AOT engine generate the appropriate infrastructure for a
separate management port at build time
management.server.port: 9009
```

With this specified at build time, when the Native Image is created, the management port can be set to any value at runtime using property files in the config-repo folder.

Following is a list of all properties set at build time in the application.yml files to avoid these types of problems for various Spring beans used by the four microservices:

```
# Required to make Springdoc handling forward headers correctly when natively compiled

server.forward-headers-strategy: framework

# Required to make the Spring AOT engine generate the appropriate infrastructure for a
separate management port, Prometheus, and K8S probes
at build time
management.server.port: 9009
management.endpoint.health.probes.enabled: true
management.endpoints.web.exposure.include:
health,info,circuitbreakerevents,prometheus

# Required to make the Spring AOT engine generate a ReactiveJwtDecoder for the OIDC issuer

spring.security.oauth2.resourceserver.jwt.issuer-uri: http://someissuer

# See https:// github.com/ springdoc/ springdoc-openapi/
issues/ 1284#issuecomment-1279854219
springdoc.enable-native-support: true

# Native Compile: Point out that RabbitMQ is to be used when performing the native compilation

spring.cloud.stream.defaultBinder: rabbit

# Native Compile: Required to disable the health check of RabbitMQ when using Kafka

# management.health.rabbit.enabled: false

# Native Compile: Required to disable the health check of Kafka when using RabbitMQ

management.health.kafka.enabled: false

# Native Compile: Required to get the circuit breaker's health check to work properly

management.health.circuitbreakers.enabled: true
```

There can also exist cases where natively compiled microservices also require slightly different configurations at runtime; this will be covered in the next section.

Updated runtime properties

In one case, a runtime property also needs to be updated when using natively compiled images. That is the connection string for the MySQL database used by the review microservice. Since not all character sets are represented in the Native Image by default, we must specify one as available in the Native Image. We will use the UTF-8 character set. This is done for all MySQL connection properties in the config-repo/review.yml review config file. It looks like this:

```
spring.datasource.url: jdbc:mysql://localhost/review-db?
useUnicode=true&connectionCollation=utf8_general_
ci&characterSetResults=utf8&characterEncoding=utf-8
```

With the required changes of properties both at build time and runtime covered, let's learn how to configure the GraalVM Native Image tracing agent.

Configuration of the GraalVM Native Image tracing agent

In cases where it is hard to determine what reachability metadata and/or custom hints are required, we can use the GraalVM Native Image tracing agent. As previously mentioned, it can, at runtime, detect the usage of reflection, resources, and proxies and create the required reachability metadata based on that.

To enable the tracing agent to observe the execution of JUnit tests, the following jvmArgs can be added to the build.gradle file in the test section:

```
tasks.named('test') {
    useJUnitPlatform()
    jvmArgs "-agentlib:Native Image-agent=access-filter-file=src/test/
resources/access-filter.json,config-output-dir=src/main/resources/META-INF/Native Image"

}
```

Since the tracing agent is not required to make native compilation work for the microservices in this book, this configuration is commented out in the build files.

The Native Image-agent=access-filter-file parameter specifies a file listing Java packages and classes that the tracing agent should exclude, typically test-related classes that we have not used for at runtime. For example, for the product microservice, the src/test/resources/access-filter.json file looks like this:

```
{ "rules":
    [
        {"excludeClasses": "org.apache.maven.surefire.**"},
        {"excludeClasses": "net.bytebuddy.**"},
        {"excludeClasses": "org.apiguardian.**"},
        {"excludeClasses": "org.junit.**"},
        {"excludeClasses": "org.gradle.**"},
        {"excludeClasses": "org.mockito.**"},
        {"excludeClasses": "org.springframework.test.**"},
        {"excludeClasses": "org.springframework.boot.test.**"},
        {"excludeClasses": "org.testcontainers.**"},
        {"excludeClasses": "se.magnus.microservices.core.product.
MapperTests"},
        {"excludeClasses": "se.magnus.microservices.core.product.
MongoDbTestBase"},
        {"excludeClasses": "se.magnus.microservices.core.product.
PersistenceTests"},
        {"excludeClasses": "se.magnus.microservices.core.product.
ProductServiceApplicationTests"}
    ]
}
```

The folder specified by the config-output-dir parameter will contain the generated configuration files. The specified folder, src/main/resources/META-INF/Native Image, is where the GraalVM native compiler looks for reachability metadata.

Finally, let's learn how the verification script has been adopted to be able to test the Native Images.

Updates to the test-em-all.bash verification script

In the previous chapters, eclipse-temurin was used as the base image for the Docker images. The verification script, test-em-all.bash, uses the wget command that comes with that base image for running circuit breaker tests inside the product-composite container.

The verification script runs the wget command inside the product-composite container since the endpoints used to verify the functionality of the circuit breaker are not exposed outside of the internal network in Docker.

With natively compiled microservices, the Docker image will no longer contain utility tools such as the wget command. To overcome this problem, the wget commands are executed from the auth-server's container, whose Docker image is still based on eclipse-temurin and therefore contains the required wget command. Since the tests of the circuit breaker are executed from auth-server, the hostname, localhost, is replaced with product-composite.

For details, see the verification script: test-em-all.bash.

With the required changes in the source code explained, let's learn how to use the various tools mentioned in the previous sections to test and create Native Images for the microservices.

Testing and compiling Native Images

Now, it is time to try out the tools for testing and building Native Images!

The following tools will be covered in this section:

- Running the tracing agent
- Executing native tests
- Creating a Native Image for the current OS
- Creating a Native Image as a Docker image

Since the first three tools require GraalVM and its Native Image compiler locally, we must switch to the GraalVM-based Java compiler with this command:

```
sdk use java 24-graalce
```

Next, we will go through the tools one by one; let's start with the tracing agent!

Running the tracing agent

For the microservices in this book, the tracing agent is not required. But it can be interesting to see how to use it in other cases. If, for example, one of your microservices requires help from the tracing agent to generate the required reachability metadata.

If you want to try out the tracing agent, you can do so with the following steps:

1. Activate the jvmArgs parameter in the section of the build.gradle file for the selected microservice by removing the preceding comment characters, //.
2. Run a gradle test command – in this case, for the product service:

```
cd $BOOK_HOME/Chapter23
./gradlew :microservices:product-service:test --no-daemon
```

This is a normal gradle test command, but to avoid running out of memory, we disable the use of the Gradle daemon. By default, the daemon is limited to using 512 MB for its heap, which is insufficient for the tracing agent in most cases.

After the tests are complete, you should find a file named reachability-metadata.json in the microservices/product-service/src/main/resources/META-INF/Native Image folder.

To learn more about the content of this file, see https://www.graalvm.org/latest/reference-manual/native-image/metadata/.

After browsing through the generated file, wrap up by re-adding the comment before the jvmArgs parameter in the build file to disable the tracing agent and remove the created file.

Next, on how to use native tests!

Running native tests

Running native tests can, as described earlier, be very useful for automating the process of finding-ing issues with creating Native Images. Unfortunately, a couple of problems currently prevent us from using native tests with the microservices in this book. Therefore, some tests have been disabled when running native tests using the @DisabledInNativeImage annotation. The error that is thrown when running the native test is described in a comment next to the annotation for each disabled test. When these problems are resolved in the future, the @DisabledInNativeImage - annotations can be removed step by step, and more and more tests will be run by the native test command.

To run the native tests on all four microservices, run this command:

```
./gradlew nativeTest
```

To test a specific microservice, run a command like this:

```
./gradlew :microservices:product-service:nativeTest
```

After each test of a microservice, the native testing tool creates a test report that looks like this:

```
Test run finished after 3441 ms
...
[                13 tests found                  ]
```

```
[ [ 1 test skipped ]
[ [ 12 tests started ]
[ 0 tests aborted ]
[ [ 12 successful tests ]
[ 0 tests failed ]
```

As seen in the preceding report, 12 native tests ran successfully, and one test was skipped.

After covering the test agent and native tests, let's see how we can create Native Images.

Creating a Native Image for the current OS

The first option for creating a Native Image is to use Gradle's nativeImage command. It will use the installed GraalVM Native Image compiler to create an executable file for the current OS and hardware architecture.

Since we will only test our microservices using Docker and Kubernetes, we will not use Native Images created by this command. But to try it out on the product-composite microservice, run the following commands:

1. Create the Native Image with the following command:

   ```
   ./gradlew microservices:product-composite-service:nativeCompile
   ```

   The executable file will be created in the build/native/nativeCompile folder with the name product-composite-service.

   The executable file can be inspected using the file command:

   ```
   file microservices/product-composite-service/build/native/
   nativeCompile/product-composite-service
   ```

   It will respond with something like this:

   ```
   …product-composite-service: Mach-O 64-bit executable arm64
   ```

   Here, Mach-O indicates that the file is compiled for macOS, and arm64 indicates it is compiled for Apple silicon.

2. To try it out, we need to start up the resources it needs manually. In this case, it is only RabbitMQ that is required to make it start up successfully. Start it up with this command:

   ```
   docker compose up -d rabbitmq
   ```

3. The Native Image can now be started in the terminal by specifying the same environment variables that are supplied in the docker-compose files:

```
SPRING_RABBITMQ_USERNAME=rabbit-user-prod \
SPRING_RABBITMQ_PASSWORD=rabbit-pwd-prod \
SPRING_CONFIG_LOCATION=file:config-repo/application.yml,file:config-repo/product-composite.yml \
microservices/product-composite-service/build/native/nativeCompile/product-composite-service
```

   It should start up quickly and print something like the following in the log output:

```
Started ProductCompositeServiceApplication in 0.543 seconds
```

4. Try it out by calling its liveness probe:

```
curl localhost:4004/actuator/health/liveness
```

5. Expect it to answer with the following:

```
{"status":"UP"}
```

6. Stop the execution by pressing *Ctrl* + *C* and stop the RabbitMQ container with this command:

```
docker compose down
```

Even though this is the fastest way to create a Native Image, it is not very useful for the scope of this book. Instead, it needs to be built for Linux and placed in a Docker container. Let's jump into the next section and learn how to do that.

Creating a Native Image as a Docker image

Now, it is time to build Docker images containing Native Images of our microservices. Go through the following steps:

1. This is a very resource-demanding process. Therefore, first, ensure that Docker Desktop is allowed to consume at least 10 GB of memory to avoid out-of-memory faults.

If a build fails with an error message that looks like <container-name> exited with code 137, you have run out of memory in Docker.

2. If your computer has less than 32 GB of memory, it could be a good idea to stop the minikube instance at this time to avoid running out of memory on the computer. Use the following command:

```
minikube stop
```

3. Ensure that the Docker client talks to Docker Desktop and not to the minikube instance:

```
eval $(minikube docker-env -u)
```

4. Run the following command to compile the product service:

```
./gradlew :microservices:product-service:bootBuildImage --no-daemon
```

Expect it to take some time. The command will start a Docker container to perform the native compilation. The first time it runs, it will also download the GraalVM native compiler to be used in Docker, making the compilation time even longer. On my MacBook, the first compilation takes a few minutes, mainly depending on my network's capacity; after that, it takes around just a minute or two.

Expect a lot of output during the compilation, including all sorts of warning and error messages. A successful compilation ends with a log output like this:

```
Successfully built image 'docker.io/hands-on/native-product-
service:latest'
```

Natively compile the three remaining microservices with the following commands:

```
./gradlew :microservices:product-composite-service:bootBuildImage --no-daemon

./gradlew :microservices:recommendation-service:bootBuildImage --no-
daemon
./gradlew :microservices:review-service:bootBuildImage --no-daemon
```

5. To verify that the Docker images were successfully built, run the following command:

```
dockerimages | grep "hands-on/native"
```

Expect output like this:

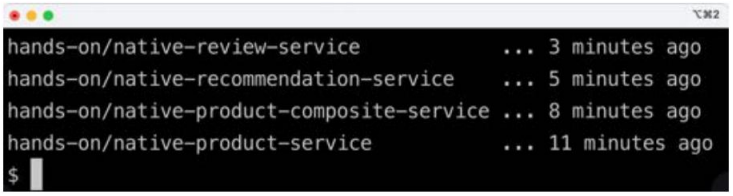


Figure 23.2: Docker images containing the natively compiled executables

Now that we've created the Docker images containing the natively compiled executables, we are ready to try them out! We will start with Docker Compose and, after that, try them out with Kubernetes.

Testing with Docker Compose

We are ready to try out the natively compiled microservices. To use the Docker images that contain the natively compiled microservices, two new Docker Compose files have been created, docker-compose-native.yml and docker-compose-partitions-native.yml. They are copies of docker-compose.yml and docker-compose-partitions.yml, where the build option has been removed from the definitions of the microservices. Also, the names of the Docker images to use have been changed, so the ones we created in the previous section are used, with names that start with native-.

In this chapter, we will only use docker-compose-native.yml.; feel free to try out docker-compose-partitions-native.yml on your own.

We'll first get a benchmark using the Java VM-based microservices to compare the startup times and initial memory consumption. We will run the following tests:

- Use the Java VM-based microservices with AOT mode disabled
- Use the Java VM-based microservices with AOT mode enabled
- Use the Docker images that contain the natively compiled microservices

To avoid port collisions, we first must stop the minikube instance with this command:

```
minikube stop
```

Testing Java VM-based microservices with AOT mode disabled

We will start the tests by ignoring the AOT-generated code, starting the Java VM-based micros-services as we did in the previous chapters. Run through the following commands to test the Java VM-based microservices:

1. Start by compiling the source code and building the Java VM-based Docker images in Docker Desktop:

```
cd $BOOK_HOME/Chapter23
eval $(minikube docker-env -u)
./gradlew build
docker compose build
```

2. Use the default Docker Compose file for the Java VM-based microservices:

```
unset COMPOSE_FILE
```

3. Start all containers, except the microservices' containers:

```
docker compose up -d mysql mongodb rabbitmq auth-server gateway
```

Wait for the containers to start up until the CPU load goes down.

4. Start up the microservices using the Java VM:

```
docker compose up -d
```

Wait for the microservices to start up, again monitoring the CPU load.

5. To find out how long it took to start the microservices, we can look for a log output con-staining: Started. Run the following command:

```
docker compose logs product-composite product review recommendation | grep ":Started"
```

Expect an output like this:

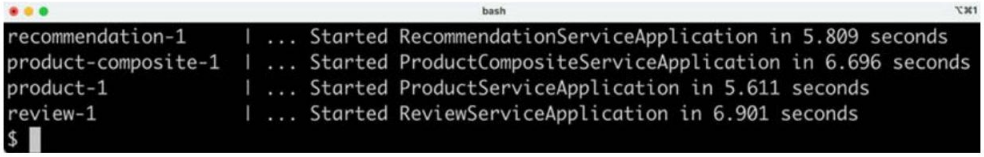


Figure 23.3: Startup times for Java VM-based microservices

In the output, we can see startup times varying from 5.5 to 7 seconds. Remember that all four microservice instances were started simultaneously, resulting in longer startup times compared to if they were started up one by one.

6. Run through the tests to verify that the system landscape works as expected:

```
USE_K8S=false HOST=localhost PORT=8443 HEALTH_URL=https://
localhost:8443 ./test-em-all.bash
```

7. Expect the output we have seen in previous chapters from the tests:

```
Start Tests: Sat MMM DD 09:36:19 CEST YYYY
...
Wait for: curl -k https://health.minikube.me/actuator/health...
DONE, continues...
...
Test OK (HTTP Code: 200)
...
End, all tests OK: Sat MMM DD 09:36:40 CEST YYYY
$
```

Figure 23.4: Output from the test script

8. Finally, to find out how much memory is used after starting up and running the tests, run the following command:

```
docker stats --no-stream
```

Expect a response like this:

```
CONTAINER ID   NAME                                              CPU %     MEM USAGE ...
c2fd8c3cf276   chapter23-review-1                                0.14%     307.7MiB  ...
ecb84348619f   chapter23-recommendation-1                        0.13%     251.4MiB  ...
d276f9d9c4bf   chapter23-product-composite-1                     0.16%     293.6MiB  ...
0aa87dcc0145   chapter23-product-1                               0.19%     245.2MiB  ...
8a37a5918b2f   chapter23-jaeger-collector.istio-system-1         0.13%     198.4MiB  ...
6ee062ef5073   chapter23-gateway-1                               0.12%     232.9MiB  ...
efc90cd09472   chapter23-auth-server-1                           0.20%     164.2MiB  ...
32cf0ffa6870   chapter23-mongodb-1                               1.17%     164.7MiB  ...
7ead773af325   chapter23-mysql-1                                 0.85%     394.6MiB  ...
b46099e0b2ab   chapter23-rabbitmq-1                              7.96%     122.8MiB  ...
$
```

Figure 23.5: Memory usage for Java VM-based microservices

From the preceding output, we can see that the microservices consume around 240-310 MB.

Bring down the system landscape:

```
docker compose down
```

Now we know how long the microservices take to start up without using the AOT-generated code; let's test them in AOT mode.

Testing Java VM-based microservices with AOT mode enabled

Now we will enable AOT mode, using the AOT-generated code to start the Java VM-based microservices . We expect them to start a bit faster in AOT mode. Run through the following commands:

1. Start all containers, except the microservices' containers:

```
docker compose up -d mysql mongodb rabbitmq auth-server gateway
```

2. Enable AOT mode by editing each microservice's Dockerfile and set “-Dspring.aot.enabled=true" in the ENVIRONMENT command so it looks like this:

```
ENTRYPOINT ["java", "-Dspring.aot.enabled=true", "org.springframework.boot.loader.JarLauncher"]
```

3. Rebuild the microservices:

```
docker compose build
```

4. Start the microservices:

```
docker compose up -d
```

5. Check AOT mode:

```
docker compose logs product-composite product review recommendation | grep "Starting AOT-processed"
```

Expect four lines containing “Starting AOT-processed".

6. Check the startup times:

```
docker compose logs product-composite product review recommendation | grep ":Started"
```

Expect the same type of output as when running without AOT mode in the preceding section , but with slightly shorter startup times. In my case, the startup times vary from 4.5 to 5.5 seconds. Compared to the normal Java VM startup times, this is 1 to 1.5 seconds faster.

7. Run test-em-all.bash:

```
USE_K8S=false HOST=localhost PORT=8443 HEALTH_URL=https://
localhost:8443 ./test-em-all.bash
```

   Expect an output like when running without AOT mode in the preceding section.

8. Revert the changes in the Dockerfiles and perform a rebuild of the Docker images to disable the AOT mode.

9. Bring down the system landscape:

```
docker compose down
```

In this test, we verified that Java VM-based microservices start up a bit faster using the AOT-gen-erated code. Now it's time to try out the natively compiled microservices.

Testing natively compiled microservices

Now, we are ready to repeat the same procedure, but this time using the Docker images with the natively compiled microservices:

1. Change to the new Docker Compose file:

```
export COMPOSE_FILE=docker-compose-native.yml
```

2. Start all containers, except for the microservices' containers:

```
docker compose up -d mysql mongodb rabbitmq auth-server gateway
```

   Wait for the containers to start up until the CPU load goes down.

3. Start up the microservices using the Java VM:

```
docker compose up -d
```

   Wait for the microservices to start up, again monitoring the CPU load.

4. To find out how long it took to start the natively compiled microservices, run the same command we ran previously:

```
docker compose logs product-composite product review recommendation | grep ":Started"
```

Expect output like this:

Figure 23.6: Startup times for natively compiled microservices

In the preceding output, we can see startup times varying from 0.2-0.5 seconds. Consider that all microservices instances were started up at the same time, these are rather impressive figures compared to the 5.5 to 7 seconds it took for the Java VM-based tests!

5. Run through the tests to verify that the system landscape works as expected:

```
USE_K8S=false HOST=localhost PORT=8443 HEALTH_URL=https://
localhost:8443 ./test-em-all.bash
```

Expect the same output as from the preceding test using the Java VM-based Docker images.

6. Finally, to find out how much memory is used after starting up and running the tests, run the following command:

```
docker stats --no-stream
```

Expect a response like this:

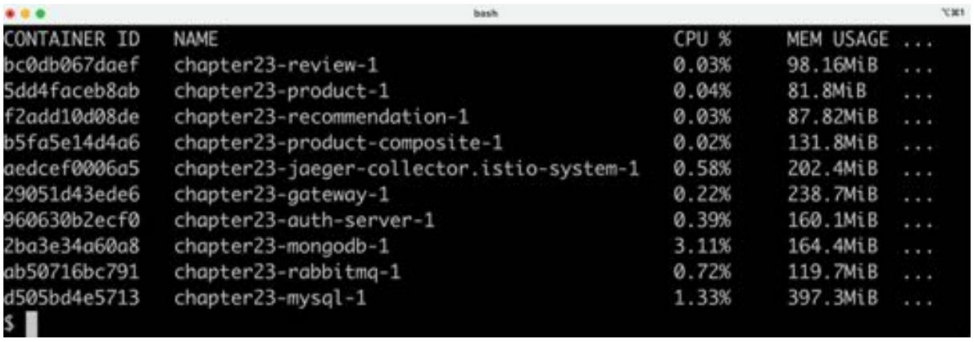


Figure 23.7: Memory usage for natively compiled microservices

From the preceding output, we can see that the microservices consume around 80-130 MB. Again, this is a noticeable reduction compared to the 240-310 MB that the Java VM containers used!

7. Bring down the system landscape:

```
docker compose down
```

To get a better understanding of both the memory and CPU consumption of the natively compiled microservices, a more realistic load test needs to be performed, but that is beyond the scope of this book.

After seeing how much faster and less memory-consuming the natively compiled microservices are when starting up compared to Java VM-based alternatives, let's see how we can run them using Kubernetes.

Testing with Kubernetes

To be able to deploy the natively compiled microservices in Kubernetes, a new environment Helm chart has been added, which has been configured to use the Docker images that contain the natively compiled microservices. The Helm charts can be found in the following folders:

```
kubernetes/helm/
ÿÿÿ environments
    ÿÿÿ dev-env-native
```

Another thing we need to consider before deploying the natively compiled microservices to Kubernetes is how to provision the Docker images. We don't want to run the lengthy native compilation commands again to get new Docker images created in the minikube instance. If we used a Docker registry in this book, we could have pushed the images to the registry, but we haven't. Instead, we will extract the Docker images from Docker Desktop and import them into the minikube instance, as a workaround for not using a Docker registry.

Move the Docker images from Docker Desktop to the minikube instance with the following commands:

1. Export the Docker images from Docker Desktop:

```
eval $(minikube docker-env -u)

docker save hands-on/native-product-composite-service:latest -o native-product-
composite.tar

docker save hands-on/native-product-service:latest -o native-product.tar
```

```
docker save hands-on/native-recommendation-service:latest -o native-recommendation.tar

docker save hands-on/native-review-service:latest -o native-review.
tar
```

2. Start up the minikube instance again:

```
minikube start
```

3. In a separate terminal, start the minikube tunnel command:

```
minikube tunnel
```

Note that this command requires that your user has sudo privileges and that you enter your password during startup. It can take a couple of seconds before the command asks for the password, so it is easy to miss!

4. Import the Docker images into the minikube instance:

```
eval $(minikube docker-env)
docker load -i native-product-composite.tar
docker load -i native-product.tar
docker load -i native-recommendation.tar
docker load -i native-review.tar
```

5. Finally, delete the exported .tar files:

```
rm native-product-composite.tar native-product.tar native-recommendation.tar native-review.tar
```

Building, deploying, and verifying the deployment on Kubernetes is done in the same way as in the previous chapters. Run the following commands:

1. Build the Docker image for auth-server with the following command:

```
docker compose build auth-server
```

2. Recreate the namespace, hands-on, and set it as the default namespace:

```
kubectl delete namespace hands-on
kubectl apply -f kubernetes/hands-on-namespace.yml
```

```
kubectl config set-context $(kubectl config current-context) --namespace=hands-on
```

3. Resolve the Helm chart dependencies with the following commands.

   First, we update the dependencies in the components folder:

```
for f in kubernetes/helm/components/*; do helm dep up $f; donated
```

4. Next, we update the dependencies in the environments folder:

```
for f in kubernetes/helm/environments/*; do helm dep up $f; donated
```

5. We are now ready to deploy the system landscape using Helm. Run the following command and wait for all the deployments to complete:

```
helm upgrade -install hands-on-dev-env-native \
  kubernetes/helm/environments/dev-env-native \
  -n hands-on --wait
```

In the previous chapters, we used the helm install command. The helm upgrade -install command used here is a better alternative for scripting since it performs an insert if the chart is not installed, but an upgrade
if the chart is already installed. It's a bit like an upsert command in the relational database world.

6. Run the normal tests to verify the deployment with the following command:

```
./test-em-all.bash
```

   Depending on how long you waited, the tests might fail with an error message like this:

```
- Response Body: Jwks doesn't have key to match kid or something from Jwt
```

   This error is caused by the Istio daemon, istiod, caching the JWKS public keys from the auth server in the previous deployment. Refer to *Chapter 18, Introduction to Microservices,* in the *Deploying v1 and v2 versions of the microservices with routing to the v1 version* section, for more information. Allow around a minute to pass after the helm update command is executed, and the tests should run without any errors. Expect the output to be like what we've already seen in the previous tests.

7. Check the startup time for one of the Pods. To measure the actual startup time for a specific microservice, let's delete it and then measure the time it takes to start up once Kubernetes recreates it:

```
kubectl delete pod -l app=product-composite
kubectl logs -l app=product-composite --tail=-1 | grep ":Started"
```

Expect a response like this:

Figure 23.8: Startup time when running as a Pod in Kubernetes

Expect a startup time around what we noticed when using Docker Compose – 0.4 seconds in the preceding example. Since we also start an Istio proxy as a sidecar, there might be some extra delay.

8. Check the Docker images used with the following command:

```
kubectl get pods -o jsonpath="{.items[*].spec.containers[*].image}" | xargs -n1 | grep hands-
on
```

Expect the following response:

Figure 23.9: Docker images with natively compiled code

From the output, we can see that all containers, except auth-server, use Docker images with the native name prefix, meaning we are running natively compiled executables inside the Docker containers.

This completes this chapter on using Spring's AOT engine and the GraalVM project to create natively compiled executables for our microservices.

Summary

In this chapter, we were introduced to the new *Spring AOT engine* and underlying *GraalVM* project, along with its Native Image compiler. After declaring GraalVM's plugin in the build file and providing the Native Image compiler with some reachability metadata and custom hints, it can be used to create Native Images. Spring Boot's buildBootImage Gradle task packages these stand- alone executable files into ready-to-use Docker images.

The main benefit of compiling Java-based source code into Native Images is a significantly faster startup time and less memory usage. In a test where we started up the microservice instances at the same time, we observed 0.2-0.5 seconds startup times for the natively compiled microservices, compared with the 5.5 to 7 seconds required for the Java VM-based microservices for the same test. Also, the natively compiled microservices required less than half of the memory compared to the Java VM-based microservices after running through the verifications in the test-em-all. bash script.

Most libraries and frameworks in this book already support GraalVM's Native Image compiler. For those that don't, *GraalVM Reachability Metadata Repository* can help out by providing reachability metadata from the community. GraalVM's build plugin can automatically detect and download reachability metadata from this repository. As a last resort, the GraalVM Native Image *tracing agent* can be used to create reachability metadata to help the native compiler. The tracing agent is configured to run together with the existing JUnit tests, creating reachability metadata based on the execution of the tests.

If we find it problematic to get the Native Image creation to work properly for a specific Spring feature, we can reach out to the *Spring AOT Smoke Tests* project for working examples per Spring feature. To verify that the microservices will work once natively compiled, we can run the unit tests in a Native Image using Spring Boot's Gradle task, nativeTest.

We have also seen how easy it is to replace the Docker images running a Java VM with Docker images containing natively compiled images. By running the Java VM Docker images in *AOT mode,* the startup times can be reduced to a bit and, at the same time, ensure that the generated AOT code works as expected. Finally, we tested the natively compiled images, both using Docker Compose and Kubernetes.

With this, we have reached the end of the book. I hope it has helped you learn how to develop microservices using all the amazing features of Spring Boot, Spring Cloud, Kubernetes, and Istio, and that you feel encouraged to try them out!

Questions

1. How are Spring's AOT engine and the GraalVM projects related to each other?
2. How is the tracing agent used?
3. What is the difference between JIT and AOT compilation?
4. What is AOT mode, and how can it be beneficial to use?
5. What is a native custom hint?
6. What is a native test, and why is it useful?
7. How are initial memory usage and startup times affected by natively compiling Java code?

24

Unlock Your Book's Exclusive Benefits

Your copy of this book comes with the following exclusive benefits:

Next-gen Packt Reader

AI assistant (beta)

DRM-free PDF/ePub downloads

Use the following guide to unlock them if you haven't already. The process takes just a few minutes and needs to be done only once.

How to unlock these benefits in three easy steps

Step 1

Have your purchase invoice for this book ready, as you'll need it in *Step* 3. If you received a physical invoice, scan it on your phone and have it ready as either a PDF, JPG, or PNG.

For more help on finding your invoice, visit https://www.packtpub.com/unlock-benefits/help.

Note: Did you buy this book directly from Packt? You don't need an invoice. After completing Step 2, you can jump straight to your exclusive content.

Step 2

Scan this QR code or go to packtpub.com/unlock.

On the page that opens (which will look similar to Figure X.1 if you're on desktop), search for this book by name. Make sure you select the correct edition.

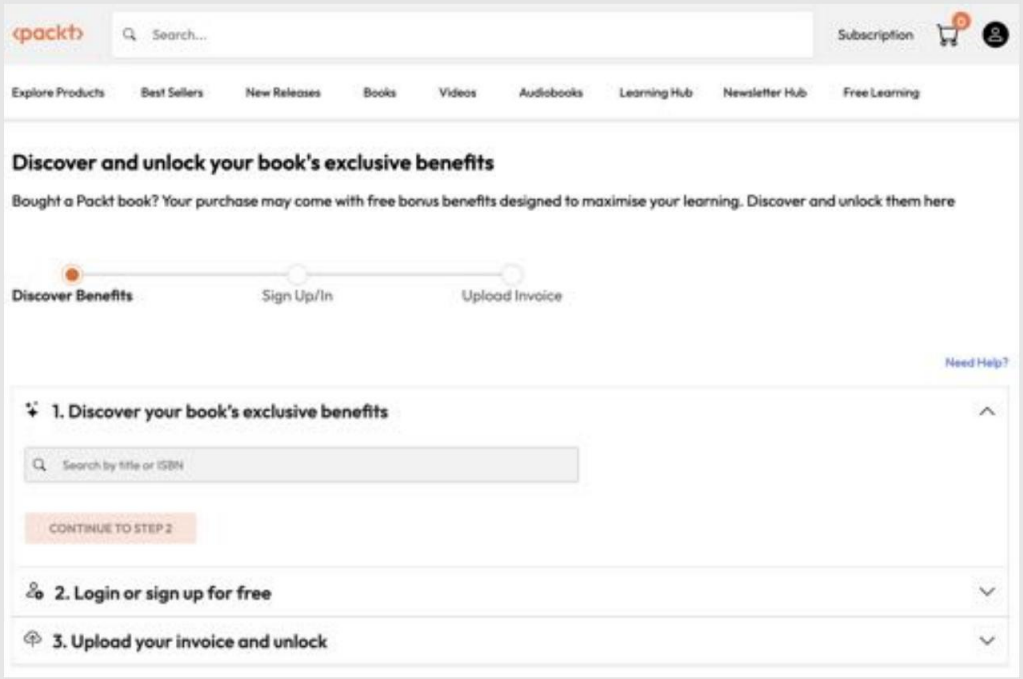


Figure X.1: Packt unlock landing page on desktop

Step 3

Once you've selected your book, sign in to your Packt account or create a new one for free. Once you're logged in, upload your invoice. It can be in PDF, PNG, or JPG format and must be no larger than 10 MB. Follow the rest of the instructions on the screen to complete the process.

Need help?

If you get stuck and need help, visit https://www.packtpub.com/unlock-benefits/help for a detailed FAQ on how to find your in- voices and more.

The following QR code will take you to the help page directly:

Note: If you are still facing issues, reach out to customercare@packt.com.

Unlock this book's exclusive benefits now

Scan this QR code or go to packtpub.com/unlock, then search this book by name.

Note: Keep your purchase invoice ready before you start.

Other Books You May Enjoy

If you enjoyed this book, you may be interested in these other books by Packt:

Software Architecture with Spring

Wanderson Xesquevixos

ISBN: 978-1-83588-060-9

- Translate complex business needs into clear and implementable design
- Design resilient systems with common architectural styles
- Transform monolithic applications into microservices following best practices • Implement event-driven architecture with Kafka
- Monitor, trace, and ensure robust testing, security, and performance
- Identify bottlenecks and optimize performance using patterns, caching, and database strategies
- Automate development workflows with CI/CD pipelines, using Jenkins to deploy the application to Kubernetes

Spring System Design in Practice

Rodrigo Santiago

ISBN: 978-1-80324-901-8

- Implement microservices for scalable, resilient web systems
- Break down business goals into well-structured product requirements
- Weigh tradeoffs between asynchronous writing vs. synchronous services and SQL vs. NoSQL storage
- Accelerate service development and reliability through the adoption of test-driven de- development
- Identify and eliminate hidden performance bottlenecks to maximize speed and efficiency
- Achieve real-time processing and responsiveness in distributed environments

Packt is searching for authors like you

If you're interested in becoming an author for Packt, please visit authors.packt.com and apply today. We have worked with thousands of developers and tech professionals, just like you, to help them share their insight with the global tech community. You can make a general application, apply for a specific hot topic that we are recruiting an author for, or submit your own idea.

Share your thoughts

Now you've finished *Microservices with Spring Boot and Spring Cloud, Fourth Edition* we'd love to hear your thoughts! If you purchased the book from Amazon, please click here to go straight to the Amazon review page for this book and share your feedback or leave a review on the site that you purchased it from.

Your review is important to us and the tech community and will help us make sure we're delivering excellent quality content.

Index

Symbols

B

C

D

F

G

H

I

J

K

L

M

N

Q

R

S

T